

INF05010 – Algoritmos avançados

Notas de aula

Marcus Ritt

13 de março de 2026

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica

Versão 1119c0 compilada em 13 de março de 2026. A obra está licenciada sob uma [Licença Creative Commons](#) (Atribuição-Uso Não-Comercial-Não a obras derivadas 4.0 ).

Agradecimentos Agradeço aos estudantes dessa disciplina por críticas e comentários e em particular o Rafael de Santiago por diversas correções e sugestões.

Conteúdo

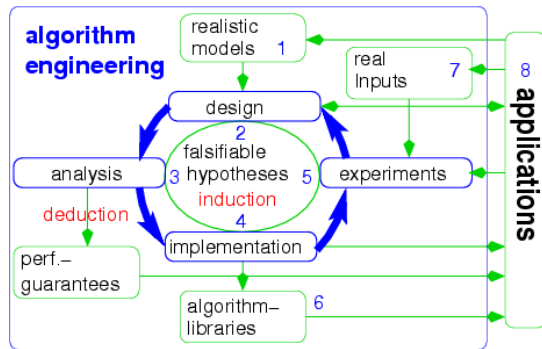
1. Algoritmos em grafos	5
1.1. Representação de grafos	5
1.1.1. Amostragem de grafos aleatórios	6
1.2. Caminhos e ciclos Eulerianos	7
1.3. Árvores geradores	8
1.4. Caminhos mais curtos	9
1.4.1. Tópicos	12
1.4.2. Mais sobre caminhos mais curtos	17
1.4.3. Notas	20
1.5. Filas de prioridade e heaps	21
1.5.1. Heaps binários	21
1.5.2. Heaps binomiais	24
1.5.3. Heaps Fibonacci	29
1.5.4. Rank-pairing heaps	33
1.5.5. Heaps ocos	41
1.5.6. Árvores de van Emde Boas	47
1.5.7. Exercícios	56
1.6. Fluxos em redes	58
1.6.1. O algoritmo de Ford-Fulkerson	60
1.6.2. O algoritmo de Edmonds-Karp	63
1.6.3. O algoritmo “caminho mais gordo” (“fattest path”)	65
1.6.4. O algoritmo push-relabel	66
1.6.5. Variantes do problema	70
1.6.6. Aplicações	74
1.6.7. Outros problemas de fluxo	80
1.6.8. Exercícios	80
1.7. Emparelhamentos	81
1.7.1. Aplicações	84
1.7.2. Grafos bi-partidos	85
1.7.3. Emparelhamentos em grafos não-bipartidos	96
1.7.4. Notas	102
1.7.5. Exercícios	103
2. Tabelas hash	105
2.1. Hashing com listas encadeadas	105

2.2.	Hashing com endereçamento aberto	109
2.3.	Cuco hashing	111
2.4.	Filtros de Bloom	113
3.	Algoritmos de aproximação	117
3.1.	Problemas, classes e reduções	117
3.2.	Medidas de qualidade	119
3.3.	Técnicas de aproximação	119
3.3.1.	Algoritmos gulosos	119
3.3.2.	Aproximações com randomização	124
3.3.3.	Programação linear	125
3.4.	Esquemas de aproximação	126
3.5.	Aproximando o problema da árvore de Steiner mínima	128
3.6.	Aproximando o PCV	129
3.7.	Aproximando problemas de cortes	130
3.8.	Aproximando empacotamento unidimensional	134
3.8.1.	Um esquema de aproximação assintótico para min-EU	139
3.9.	Aproximando problemas de sequenciamento	141
3.9.1.	Um esquema de aproximação para $P \parallel C_{\max}$	143
3.10.	Exercícios	145
4.	Algoritmos randomizados	147
4.1.	Teoria de complexidade	148
4.1.1.	Amplificação de probabilidades	149
4.1.2.	Relação entre as classes	150
4.2.	Seleção	153
4.3.	Corte mínimo	155
4.4.	Teste de primalidade	159
4.5.	Notas	163
4.6.	Exercícios	163
5.	Complexidade e algoritmos parametrizados	165
6.	Outros algoritmos	169
6.1.	O problema de soma de intervalos	169
6.2.	Amostragem aleatória	171
6.2.1.	Amostragem com reposição	171
6.2.2.	Amostragem sem reposição	172
6.3.	Problemas de contagem	176

A. Conceitos matemáticos	179
A.1. Probabilidade discreta	180
B. Algoritmos adicionais	187
B.1. Soluções do problema da mochila com Programação Dinâmica .	187
B.2. Seleção	187
C. Técnicas para a análise de algoritmos	191
C.1. Análise de recorrências	191
Bibliografia	193
Índice	199

Introdução

A disciplina “Algoritmos avançados” foi criada para combinar a teoria e a prática de algoritmos. Muitas vezes a teoria de algoritmos e a prática de implementações eficientes são ensinadas separadamente, em particular no caso de algoritmos avançados. Porém a experiência mostra que encontramos muitos obstáculos no caminho de um algoritmo teoricamente eficiente para uma implementação eficiente. Além disso, o projeto de algoritmos novos não termina com uma implementação eficiente, mas é alimentado pelos resultados experimentais para produzir melhores algoritmos. A figura abaixo mostra o ciclo típico da área emergente de *engenharia de algoritmos*.



Engenharia de algoritmos (*Algorithm Engineering* s.d.).

Seguindo essa filosofia, o nosso objetivo é tanto entender a teoria de algoritmos, demonstrando a sua correteza e analisando a sua complexidade, quanto dominar a prática de algoritmos, a sua implementação e avaliação experimental. Isso é refletido numa sequência alternada de aulas teóricas e práticas.

1. Algoritmos em grafos

1.1. Representação de grafos

Um grafo pode ser representado diretamente de acordo com a sua definição por n estruturas que representam os vértices, m estruturas que representam os arcos e ponteiros entre as estruturas. Um vértice possui ponteiros para todos os arcos incidentes saíntes ou entrantes, e um arco possui ponteiros para o início e término. A representação direta possui várias desvantagens. Por exemplo não temos acesso direto aos vértices para inserir um arco.

Duas representações simples são listas (ou vetores) não-ordenadas de vértices ou arestas. Uma outra representação simples de um grafo G com n vértices é uma *matriz de adjacência* $M = (m_{ij}) \in \mathbb{B}^{n \times n}$. Para vértices u, v o elemento $m_{uv} = 1$ caso exista um arco entre u e v . Para representar grafos não-direcionados mantemos $m_{uv} = m_{vu}$, i.e., M é simétrica. A representação permite um teste de adjacência em $O(1)$. Percorrer todos os vizinhos de um dado vértice v custa $O(n)$. O custo alto de espaço de $\Theta(n^2)$ restringe o uso de uma matriz de adjacência para grafos pequenos¹.

Uma representação mais eficiente é por *listas* ou *vetores* de adjacência. Neste caso armazenamos para cada vértice os vizinhos em uma lista ou um vetor. As listas ou vetores mesmos podem ser armazenados em uma lista ou um vetor global. Com isso a representação ocupa espaço $\Theta(n + m)$ para m arestas.

Uma escolha comum é um vetor de vértices que armazena listas de vizinhos. Essa estrutura permite uma inserção e deleção simples de arcos. Para facilitar a deleção de um vértice em grafos não-direcionados, podemos armazenar junto com o vizinho u do vértice v a posição do vizinho v do vértice u . A representação dos vizinhos por vetores é mais eficiente, e por isso preferível caso a estrutura do grafo é estática (Black Jr. e Martel, 1998; Park et al., 2004).

Caso escolhamos armazenar os vértices em uma lista dupla, que armazena uma lista dupla de vizinhos, em que os vizinhos são representados por posições da primeira lista, obtemos uma *lista dupla de arcos* (ingl. doubly connected arc list, DCAL). Essa estrutura permite uma inserção e remoção tanto de vértices quanto de arcos.

Suponha que $V = [n]$. Uma outra representação compacta e eficiente conhecida como *forward star* para grafos estáticos usa um *vetor de arcos* $a_1, \dots, a_m$.

¹ Ainda mais espaço consome uma *matriz de incidência* entre vértices e arestas em $\mathbb{B}^{n \times m}$.

Tabela 1.1.: Operações típicas em grafos.

Operação	Lista de arestas	Lista de vértices	Matriz de adjacência	Lista de adjacência
Inserir aresta	$O(1)$	$O(n + m)$	$O(1)$	$O(1)$ ou $O(n)$
Remover aresta	$O(m)$	$O(n + m)$	$O(1)$	$O(n)$
Inserir vértice	$O(1)$	$O(1)$	$O(n^2)$	$O(1)$
Remover vértice	$O(m)$	$O(n + m)$	$O(n^2)$	$O(n + m)$
Teste $uv \in E$	$O(m)$	$O(n + m)$	$O(1)$	$O(\Delta)$
Percorrer vizinhos	$O(m)$	$O(\Delta)$	$O(n)$	$O(\Delta)$
Grau de um vértice	$O(m)$	$O(\Delta)$	$O(n)$	$O(1)$

Mantemos a lista de arestas ordenada pelo começo do arco. Uma permutação σ nos dá as arestas em ordem do término. (O uso de uma permutação serve para reduzir o consumo de memória.) Para percorrer eficientemente os vizinhos de um vértice armazenamos o índice s_v do primeiro arco sainte na lista de arestas ordenado pelo começo e o índice e_v do primeiro arco entrante na lista de arestas ordenada pelo término com $s_{n+1} = e_{n+1} = m + 1$ por definição. Com isso temos $N^+(v) = \{a_{s_v}, \dots, a_{s_{v+1}-1}\}$ com $\delta_v^+ = s_{v+1} - s_v$, e $N^-(v) = \{a_{\sigma(e_v)}, \dots, a_{\sigma(e_{v+1}-1)}\}$ com $\delta_v^- = e_{v+1} - e_v$. A representação precisa espaço $O(n + m)$.

Tabela 1.1 mostra a complexidade de operações típicas nas diferentes representações.

1.1.1. Amostragem de grafos aleatórios

Um modelo elementar de grafos aleatórios é de Erdős e Rényi. Na variante $G_{n,p}$ temos um grafo com n vértices, e cada uma das possíveis $M = \binom{n}{2}$ arestas é gerada com probabilidade p ; na variante $G_{n,m}$ cada uma das $\binom{M}{m}$ seleções de m das M arestas tem a mesma probabilidade. (Tudo que segue funciona também no caso de grafos direcionados, tomando $M = n(n - 1)$.)

Para amostrar de acordo com $G_{n,p}$ podemos simplesmente percorrer todas as M arestas candidatas e adicionar cada uma com probabilidade p em tempo $O(M)$ e espaço $O(m)$. Neste caso o número de arestas é variável, de acordo com uma distribuição binomial $B(m, p)$ com valor esperado de arestas $m = pM$ e desvio padrão de $\sqrt{mp(1 - p)} = \Theta(n\sqrt{p(1 - p)}) = \Theta(n)$. Uma alternativa mais rápida pode ser amostrar o número de arestas de acordo com $B(m, p)$ e depois usar o modelo $G_{n,m}$.

Para amostrar de acordo com $G_{n,m}$ podemos usar um algoritmo de amostra-

gem sem reposição (ver 6.2.2) para selecionar as arestas em tempo e espaço $O(m)$. (Uma forma simples, mas menos eficiente é aplicar a *amostragem por rejeição*: repetidamente selecionar uma aresta aleatória dos M e rejeitar arestas já selecionadas. O tempo esperado de amostrar a i -ésima aresta é $M/(M-i)$ e logo o tempo esperado é

$$\begin{aligned} E[T] &= \frac{M}{M-0} + \frac{M}{M-1} + \cdots + \frac{M}{M-m+1} \\ &= M(H_M - H_{M-m}) \leq M(\ln M - \ln(M-m)). \end{aligned}$$

Com $m = pM$ obtemos $M - m = (1-p)M$ e logo $E[T] = M \ln \frac{1}{1-p}$.

1.2. Caminhos e ciclos Eulerianos

Um *caminho Euleriano* passa por todas as arestas do grafo exatamente uma vez. Um caminho Euleriano fechado é um ciclo Euleriano. Um grafo é *Euleriano* caso ele possua um ciclo Euleriano que passa por cada vértice (pelo menos uma vez).

Proposição 1.1

Um grafo não-direcionado $G = (V, E)$ é Euleriano sse G é conectado e cada vértice tem grau par.

Prova. Por indução sobre o número de arestas. A base da indução é um grafo com um vértice e nenhuma aresta que satisfaz a proposição. Suponha que os grafos com $\leq m$ arestas satisfazem a proposição e temos um grafo G com $m+1$ arestas. Começa por um vértice v arbitrário e procura um caminho que nunca passa duas vezes por uma aresta até voltar para v . Isso sempre é possível porque o grau de cada vértice é par: entrando num vértice sempre podemos sair. Removendo este caminho do grafo, obtemos uma coleção de componentes conectados com menos que m arestas, e pela hipótese da indução existem ciclos Eulerianos em cada componente. Podemos obter um ciclo Euleriano para o grafo original pela concatenação desses ciclos Eulerianos. ■

Pela prova temos o seguinte algoritmo com complexidade $O(|E|)$ para encontrar um ciclo Euleriano na componente de $G = (V, E)$ que contém $v \in V$:

Algoritmo 1.1 (Caminho Euleriano)

```

1  Euler( $G = (V, E), v \in V$ ) :=
2    if  $|E| = 0$  return  $v$ 
3    procura um caminho começando em  $v$ 
```

1. Algoritmos em grafos

```
4      sem repetir arestas voltando para  $v$ 
5      seja  $v = v_1, v_2, \dots, v_n = v$  esse caminho
6      remove as arestas  $v_1v_2, v_2v_3, \dots, v_{n-1}v_n$  de  $G$ 
7      para obter  $G_1$ 
8      return  $\text{Euler}(G_1, v_1) + \dots + \text{Euler}(G_{n-1}, v_{n-1}) + v_n$ 
9
10     // Usamos + para concatenação de caminhos.
11     //  $G_i$  é  $G_{i-1}$  com as arestas do
12     // caminho  $\text{Euler}(G_{i-1}, v_{i-1})$  removidos, i.e
13     //  $G_i := (V, E(G_{i-1}) \setminus E(\text{Euler}(G_{i-1}, v_{i-1})))$ 
```

Algoritmo 1.1 é de Hierholzer (1873).

1.3. Árvores geradores

Exemplo 1.1

Árvore geradora mínima através do algoritmo de Prim.

Algoritmo 1.2 (Árvore geradora mínima)

Entrada Um grafo conexo não-direcionado ponderado $G = (V, E, c)$

Saída Uma árvore $T \subseteq E$ de menor custo total.

```
1   $V' := \{v_0\}$  para um  $v_0 \in V$ 
2   $T := \emptyset$ 
3  while  $V' \neq V$  do
4      escolha  $e = \{u, v\}$  de custo mínimo
5      entre  $V'$  e  $V \setminus V'$  (com  $u \in V', v \in V \setminus V'$ )
6       $V' := V' \cup \{v\}$ 
7       $T := T \cup \{e\}$ 
8  end while
```

Algoritmo 1.3 (Prim refinado)

Implementação mais concreta:

```
1   $T := \emptyset$ 
2  for  $u \in V \setminus \{v\}$  do
3      if  $u \in N(v)$  then
4           $\text{value}(u) := c_{uv}$ 
5           $\text{pred}(u) := v$ 
```

```

6   else
7       value(u) := ∞
8   end if
9   insert(Q, (value(u), u)) { pares (chave, elemento) }
10 end for
11 while Q ≠ ∅ do
12     v := deletemin(Q)
13     T := T ∪ {pred(v)v}
14     for u ∈ N(v) do
15         if u ∈ Q e cvu < value(u) then
16             value(u) := cuv
17             pred(u) := v
18             update(Q, u, cvu)
19         end if
20     end for
21 end while

```

Custo? $n \times \text{insert} + n \times \text{deletemin} + m \times \text{update}$.

◇

Observação 1.1

Implementação com vetor de distâncias: $\text{insert} = O(1)$ ², $\text{deletemin} = O(n)$, $\text{update} = O(1)$, e temos custo $O(n + n^2 + m) = O(n^2 + m)$. Isso é assintoticamente ótimo para grafos densos, i.e. $m = \Omega(n^2)$.

◇

Observação 1.2

Implementação com lista ordenada: $\text{insert} = O(n)$, $\text{deletemin} = O(1)$, $\text{update} = O(n)$, e temos custo $O(n^2 + n + mn) = O(mn)$ ³.

◇

Observação 1.3

Implementação com uma lista de $\sqrt{n}$ blocos de $\sqrt{n}$ elementos, insert , deletemin e update podem ser implementados em tempo $O(\sqrt{n})$, logo o algoritmo de Prim e de Dijkstra tem complexidade $O(m\sqrt{n})$.

◇

1.4. Caminhos mais curtos

Um problema fundamental em grafos é encontrar caminhos mais curtos entre pares de vértices. O algoritmo de Dijkstra resolve o problema das distâncias

²Com chaves compactas $[1, n]$.

³Na hipótese razoável que $m \geq n$.

1. Algoritmos em grafos

de um vértice origem para todos os demais em grafos com distâncias não-negativas.

Exemplo 1.2

Caminhos mais curtos com o algoritmo de Dijkstra

Algoritmo 1.4 (Dijkstra)

Entrada Um grafo direcionado $G = (V, A)$ com pesos $d_e \geq 0$ nos arcos $a \in A$, e um vértice $s \in V$.

Saída A distância mínima d_v entre s e cada vértice $v \in V$.

```
1   $d_s := 0; d_v := \infty, \forall v \in V \setminus \{s\}$ 
2   $\text{visited}(v) := \text{false}, \forall v \in V$ 
3   $Q := \emptyset$ 
4   $\text{insert}(Q, (s, 0))$ 
5  while  $Q \neq \emptyset$  do
6     $v := \text{deletemin}(Q)$ 
7     $\text{visited}(v) := \text{true}$ 
8    for  $u \in N^+(v)$  do
9      if not  $\text{visited}(u)$  then
10        if  $d_u = \infty$  then
11           $d_u := d_v + d_{vu}$ 
12           $\text{insert}(Q, (u, d_u))$ 
13        else if  $d_v + d_{vu} < d_u$ 
14           $d_u := d_v + d_{vu}$ 
15           $\text{update}(Q, (u, d_u))$ 
16        end if
17      end if
18    end for
19  end while
```

Observação 1.4

A fila de prioridade contém pares de vértices e distâncias. O algoritmo se aplica igualmente a um grafo não-direcionado. $\diamond$

Proposição 1.2

O algoritmo de Dijkstra possui complexidade

$$O(n) + n \times \text{deletemin} + n \times \text{insert} + m \times \text{update}.$$

Prova. O pré-processamento (1-3) tem custo $O(n)$. O laço principal é dominado por no máximo n operações insert, n operações deletemin, e m operações

update. A complexidade concreta depende da implementação desses operações. ■

Proposição 1.3

O algoritmo de Dijkstra é correto.

Prova. Seja $\text{dist}(s, x)$ a menor distância entre s e x . Provaremos por indução que para cada vértice v selecionado na linha 6 do algoritmo $d_v = \text{dist}(s, v)$. Como base isso é correto para $v = s$. Seja $v \neq s$ um vértice selecionado na linha 6, e supõe que existe um caminho $P = s \cdots xy \cdots v$ de comprimento menor que d_v , tal que y é o primeiro vértice que não foi processado (i.e. selecionado na linha 6) ainda. (É possível que $y = v$.) Sabemos que

$$\begin{aligned} d_y &\leq d_x + d_{xy} && \text{porque } x \text{ já foi processado} \\ &= \text{dist}(s, x) + d_{xy} && \text{pela hipótese } d_x = \text{dist}(s, x) \\ &\leq d(P) && \text{dist}(s, x) \leq d_P(s, x) \text{ e } P \text{ passa por } xy \\ &< d_v, && \text{pela hipótese} \end{aligned}$$

uma contradição com a minimalidade do elemento extraído na linha 6. (Notação: $d(P)$: distância total do caminho P ; $d_P(s, x)$: distância entre s e x no caminho P .) ■ ◇

Observação 1.5

Podemos ordenar n elementos usando um heap com n operações “insert” e n operações “deletemin”. Pelo limite de $\Omega(n \log n)$ para ordenação via comparação, podemos concluir que o custo de “insert” mais “deletemin” é $\Omega(\log n)$. Portanto, pelo menos uma das operações é $\Omega(\log n)$. ◇

O caso médio do algoritmo de Dijkstra Dado um grafo $G = (V, E)$ e um vértice inicial arbitrário suponha que temos um conjunto $C(v)$ de pesos positivos com $|C(v)| = |N^-(v)|$ para cada $v \in V$. Atribuiremos permutações dos pesos em $C(v)$ aleatoriamente para os arcos entrantes em v .

Proposição 1.4 (Noshita (1985))

O algoritmo de Dijkstra chama update em média $n \log(m/n)$ vezes neste modelo.

Prova.

Para um vértice v os arcos que podem levar a uma operação update em v são de forma (u, v) com $\text{dist}(s, u) \leq \text{dist}(s, v)$. Suponha que existem k arcos $(u_1, v), \dots, (u_k, v)$ desse tipo, ordenados por $\text{dist}(s, u_i)$ não-decrescente. Independente da atribuição dos pesos aos arcos, a ordem de processamento sempre

1. Algoritmos em grafos

é $u_1, u_2, \dots, u_k$. O arco (u_i, v) leva a uma operação update caso

$$\begin{aligned} \text{dist}(s, u_i) + d_{u_i v} &< \min_{j: j < i} (\text{dist}(s, u_j) + d_{u_j v}). \\ &< \min_{j: j < i} (\text{dist}(s, u_i) + d_{u_j v}). \\ &< \text{dist}(s, u_i) + \min_{j: j < i} d_{u_j v}. \end{aligned}$$

Logo

$$d_{u_i v} < \min_{j: j < i} \text{dist}(s, u_j) - \text{dist}(s, u_i) + d_{u_j v} < \min_{j: j < i} d_{u_j v},$$

i.e. $d_{u_i v}$ é um mínimo local na sequência dos pesos dos k arcos. O número esperado de mínimos locais de uma permutação aleatória é $H_k - 1 \leq \ln k$. Como $k \leq \delta^-(v)$ temos um limite superior para o número de operações update em todos vértices de

$$\sum_{v \in V} \ln \delta^-(v) = n \sum_{v \in V} (1/n) \ln \delta^-(v) \leq n \ln \sum_{v \in V} (1/n) \delta^-(v) = n \ln m/n.$$

A desigualdade é justificada pela equação (A.6) observando que $\ln n$ é côncava. ■

Com isso, a complexidade média do algoritmo de Dijkstra é

$$O(m + n \times \text{deletemin} + n \times \text{insert} + n \ln(m/n) \times \text{update}).$$

Usando uma fila de prioridade implementada por um heap binário que executa todas as operações em $O(\log n)$ a complexidade média do algoritmo de Dijkstra é $O(m + n \log m/n \log n)$.

1.4.1. Tópicos

Fast marching method

A equação Eikonal (do grego eikon, imagem)

$$\begin{aligned} ||\nabla T(x)|| F(x) &= 1, & x \in \Omega, \\ T|_{\partial\Omega} &= 0, \end{aligned}$$

define o tempo de chegada de uma superfície que inicia no tempo 0 na fronteira $\partial\Omega$ de um subconjunto aberto $\Omega \subseteq \mathbb{R}^3$ e se propaga com velocidade $F(x) > 0$ na direção normal⁴. O fast marching method resolve a equação Eikonal por

⁴O método também funciona para $F(x) < 0$, mas não para $F(x)$ com sinais diferentes.

discretizar o espaço regularmente, aproximar as derivadas do gradiente $\|\nabla T\|$ por diferenças finitas e propagar os valores com um método igual ao algoritmo de Dijkstra.

Com

$$\nabla T = (\partial T / \partial x_1, \partial T / \partial x_2, \partial T / \partial x_3)$$

temos

$$\|\nabla T\|^2 = (\partial T / \partial x_1)^2 + (\partial T / \partial x_2)^2 + (\partial T / \partial x_3)^2 = 1/F^2.$$

Definindo as diferenças finitas

$$D^{+x_1}T = T(x_1 + 1, x_2, x_3) - T(x); \quad D^{-x_1}T = T(x) - T(x_1 - 1, x_2, x_3)$$

podemos aproximar

$$\partial T / \partial x_1 \approx T_{x_1} = \max\{D^{-x_1}T, -D^{+x_1}T, 0\}$$

e com aproximações similares para as direções y e z obtemos uma equação quadrática em $T(x)$

$$\|\nabla T\|^2 \approx T_{x_1}^2 + T_{x_2}^2 + T_{x_3}^2 = 1/F^2 \quad (1.1)$$

Na solução dessa equação valores ainda desconhecidos de T são ignorados. O fast marching method define $T = 0$ para os pontos iniciais em $\partial\Omega$ e coloca-os numa fila de prioridade. Repetidamente o ponto de menor tempo é extraído da fila, os vizinhos ainda não visitados são atualizados de acordo com (1.1) e entram na fila, caso ainda não fazem parte. (Na terminologia do fast marching method, os pontos com distância já conhecida são “vivos” (*alive*), os pontos na fila formam a “faixa estreita” (*narrow band*), os restantes pontos são “distantes” (*far away*).)

Busca informada

O algoritmo de Dijkstra encontra o caminho mais curto de um vértice origem $s \in V$ para todos os outros vértices num grafo ponderado $G = (V, E, d)$. Caso estejamos interessados somente no caminho mais curto para um único vértice destino $t \in V$, podemos parar o algoritmo depois de processar t . Isso é uma aplicação muito comum, por exemplo na busca da rota mais curta em sistemas de navegação. Uma *busca informada* processa vértices que estimadamente são mais próximos do destino com preferência. O objetivo é processar menos vértices antes de encontrar o destino. Um dos algoritmos mais conhecidos de

1. Algoritmos em grafos

busca informada é o algoritmo A^* . Para cada vértice $v \in V$ com distância $g(v)$ da origem s , ele usa uma função heurística $h : V \rightarrow \mathbb{R}_{\geq 0}$ que estima a distância para o destino t e processa os vértices em ordem crescente do custo total estimado

$$f(v) = g(v) + h(v). \quad (1.2)$$

O desempenho do algoritmo A^* depende da qualidade de heurística h . Ele pode, diferente do algoritmo de Dijkstra, processar vértices múltiplas vezes, caso ele descubra um caminho mais curto para um vértice já processado. Isso é a principal diferença com o algoritmo de Dijkstra. Uma outra modificação é que substituímos o campo “visited” usado no algoritmo Dijkstra 1.4 por um conjunto V de vértices já visitados, porque o A^* é frequentemente aplicado em grafos com um número grande de vértices, que são explorados passo a passo sem armazenar todos os vértices do grafo na memória.

```
1   $g(s) := 0$ 
2   $f(s) := g(s) + h(s)$ 
3   $C := \emptyset$  { vértices já visitados }
4   $Q := \emptyset$ 
5  insert( $Q, (s, f(s))$ )
6  while  $Q \neq \emptyset$  do
7       $v := \text{deletemin}(Q)$ 
8       $C := C \cup \{v\}$ 
9      if  $v = t$  { destino encontrado }
10         return  $t$ 
11     for  $u \in N^+(v)$  do
12         if  $u \in Q$  then { ainda aberto: atualiza }
13              $g(u) := \min(g(v) + d_{vu}, g(u))$ 
14              $f(u) := g(u) + h(u)$ 
15             update( $Q, (u, f(u))$ )
16         else if  $u \in C$  then
17             if  $g(v) + d_{vu} < g(u)$  then
18                 { caminho menor p/ vértice já processado }
19                  $C := C \setminus \{u\}$ 
20                  $g(u) := g(v) + d_{vu}$ 
21                  $f(u) := g(u) + h(u)$ 
22                 insert( $Q, (u, f(u))$ )
23             end if
24         else { novo vértice }
25              $g(u) := g(v) + d_{vu}$ 
26              $f(u) := g(u) + h(u)$ 
```



```

27         insert(Q, (u, f(u)))
28     end if
29 end for
30 end while

```

Observação 1.6

O algoritmo de Dijkstra e a busca A^* funcionam de forma idêntica quando substituímos o vértice destino $t \in V$ por um conjunto de vértices destino $T \subseteq V$. $\diamond$

Existe uma formulação alternativa, equivalente ao algoritmo A^* . Ao invés de sempre processar o vértice aberto de menor valor f podemos processar sempre o vértice aberto de menor distância $\hat{g}$ num grafo com pesos modificados $\hat{d}_{uv} = d_{uv} - h(u) + h(v)$. Com pesos modificados obtemos para a distância total de um caminho uv arbitrário P

$$\begin{aligned}
 \hat{g}(u, v) &= \sum_{(u', v') \in P} \hat{d}_{u'v'} = \sum_{(u', v') \in P} d_{u'v'} - h(u') + h(v') \\
 &= h(v) - h(u) + \sum_{(u', v') \in P} d_{u'v'} = h(v) - h(u) + g(u, v).
 \end{aligned}$$

Com $\hat{g}(u) = \hat{g}(s, u)$ obtemos

$$\begin{aligned}
 f(u) \leq f(v) &\iff g(u) + h(u) \leq g(v) + h(v) \\
 &\iff \hat{g}(u) + h(s) \leq \hat{g}(v) + h(s) \\
 &\iff \hat{g}(u) \leq \hat{g}(v).
 \end{aligned}$$

Logo a ordem de processamento por menor $\hat{g}$ ou por menor valor f é equivalente.

Para garantir a otimalidade de uma solução a heurística h tem que ser *admissível*. Caso h seja *consistente* o algoritmo A^* não somente retorna a solução ótima, mas processa cada vértice somente uma vez.

Definição 1.1 (Admissibilidade e consistência)

Seja $\text{dist}(v, t)$ a distância mínima do vértice v ao destino t . Uma heurística h é *admissível* caso h seja um limitante inferior à distância mínima, i.e.

$$h(v) \leq \text{dist}(v, t). \quad (1.3)$$

Uma heurística é *consistente* caso o seu valor diminua de acordo com os pesos do grafo: para um arco $(u, v) \in A$

$$h(v) \geq h(u) - d_{uv}. \quad (1.4)$$

1. Algoritmos em grafos

Na representação alternativa (1.3), o critério de consistência (1.4) é equivalente com $\hat{d}_{uv} = d_{uv} - h(u) + h(v) \geq 0$. Com isso temos diretamente o

Teorema 1.1

Caso h seja consistente o algoritmo A^* nunca processa um vértice mais que uma vez.

Prova. Neste caso $\hat{d}_{uv} \geq 0$. Logo todas distâncias são positivas e o algoritmo A^* é equivalente ao algoritmo de Dijkstra. Por um argumento similar ao da proposição 1.3 o A^* nunca processa um vértice duas vezes. ■

Lema 1.1

Caso h seja consistente e $h(t) = 0$ (i.e. reconhece o destino t), h é admissível.

Prova. Seja $P = v_0v_1 \dots v_k$ um caminho de $v_0 = u$ a $v_k = t$. Então

$$d(P) = \sum_{i \in [k]} d_{v_{i-1}, v_i} \stackrel{(1.4)}{\geq} \sum_{i \in [k]} h(v_{i-1}) - h(v_i) = h(u) - h(t) = h(u).$$

Em particular, para um caminho P^* ótimo de u a t temos $h(u) \leq d(P^*) = \delta(u)$. ■

Teorema 1.2

Caso exista uma solução mínima e h seja admissível o algoritmo A^* encontra a solução mínima.

Prova. Seja $P^* = v_0v_1 \dots v_k$ um caminho ótimo de $v_0 = s$ a $v_k = t$. Caso A^* não tenha terminado, t ainda não foi explorado. Logo existe um vértice aberto de menor índice v_i em P^* . Agora supõe que o próximo vértice explorado é t , mas o valor de t não é ótimo, i.e. $f(t) > d(P^*)$. Mas então $f(v_i) = g(v_i) + h(v_i) \leq g(v_i) + \delta(v_i) = d(P^*) < f(t)$, porque h é admissível, em contradição com a exploração de t . ■

Exemplo 1.3

Figura 1.1 mostra um grafo com três funções heurísticas h diferentes. A heurística no grafo da esquerda não é admissível em u (marcado por $\uparrow$). O A^* expande s , v e depois t e termina com a distância sub-ótima 5 para chegar em t . A heurística no grafo do meio é admissível, mas não consistente: $h(u) \leq h(v) + 1$ não é satisfeito. O A^* expande s , v , u , v , t , i.e. o vértice v é processado duas vezes. Finalmente a heurística no grafo da direita é consistente (e por isso admissível). O A^* expande cada vértice uma vez, na ordem s , u , t (ou s , u , v , t). ◇

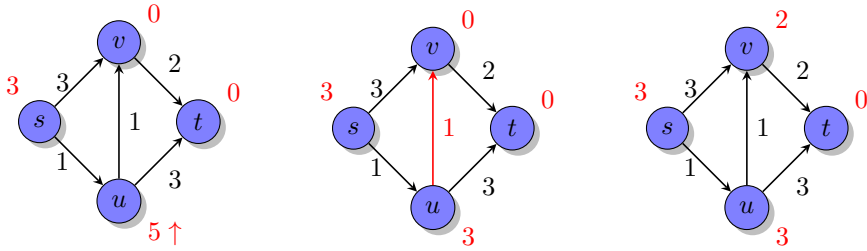


Figura 1.1.: Esquerda: Heurística não-admissível. A^* produz o valor sub-ótimo 5. Centro: Heurística admissível, mas inconsistente (arco vermelho). A^* visita v duas vezes. Direita: Heurística admissível e consistente. A^* visita cada vértice somente uma vez.

Exemplo 1.4

A Figura 1.2 compara uma busca com o algoritmo de Dijkstra com uma busca com o A^* num grafo geométrico com 5000 vértices e uma aresta entre vértices de distância no máximo 0.02. Vértices não explorados são pretos, vértices explorados claros. A clareza corresponde à ordem de exploração.

◇

1.4.2. Mais sobre caminhos mais curtos

Define um arco $a = uv \in A$ como *relaxado* caso $d_v \leq d_u + d_{uv}$, senão *tenso*. Para relaxar temos a operação

1 $\text{relax}(a) := d_v := \min\{d_v, d_u + d_{uv}\}.$

Similarmente, define $t_v := \min_{u \in N^-(v)} d_u + d_{uv}$. Podemos definir um vértice v como *relaxado* caso $d_v \leq t_v$, e senão *tenso*. Para relaxar podemos aplicar

1 $\text{relax}(v) := d_v := t_v.$

Com isso temos dois algoritmos simples que melhoram (super-)estimativas d_v das distâncias $\text{dist}(s, v)$, inicialmente $d_s = 0$, e $d_v = \infty$, para todos $v \neq s$.

- Dijkstra: em ordem de d_v , relaxa $a \in N^+(v)$; tempo $O(n \log n + m)$.
- Bellman-Ford: repete até n vezes: relaxa todos $a \in A$; tempo $O(nm)$.

O algoritmo de Bellman-Ford também funciona para pesos negativos, na ausência de ciclos negativos. (E neste caso é um dos melhores algoritmos atualmente para todas as distâncias de uma origem.)

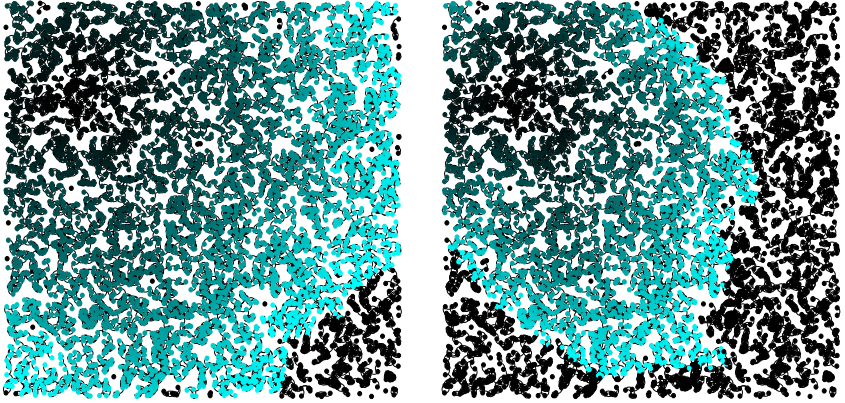


Figura 1.2.: Comparação de uma busca com o algoritmo de Dijkstra (esquerda) e o A^* (direita).

Potenciais Chama p_v , $v \in V$ um potencial caso

$$d_{uv} \geq p_v - p_u, \quad a = uv \in A. \quad (1.5)$$

Teorema 1.3

Um potencial existe sse todo circuito (ciclo direcionado) tem comprimento não-negativo.

Prova. “ $\Rightarrow$ ”: Considere o circuito $C = (v_0, v_1, \dots, v_m)$, $v_m = v_0$. Então

$$d(C) = \sum_{i \in [m]} d_{v_{i-1}, v_i} \geq \sum_{i \in [m]} p_{v_i} - p_{v_{i-1}} = p_m - p_0 = 0.$$

“ $\Leftarrow$ ”: seleciona algum $s \in V$, define $p_v := \text{dist}(s, v)$. Isso claramente satisfaz (1.5). ■

Logo: podemos definir

$$\tilde{d}_{uv} := d_{uv} - (p_v - p_u) \geq 0, \quad (1.6)$$

uma transformação que mantém caminhos mais curtos.

Agora: como podemos encontrar circuitos negativos?

Teorema 1.4

Um circuito negativo pode ser encontrado em tempo $O(nm)$.

Prova. Roda Bellman-Ford para obter distâncias $d^0, d^1, \dots, d^n$. Assume $d^{n-1} \neq d^n$, com testemunha $t \in V$, i.e. $d_t^n < d_t^{n-1}$. Logo existe uma st -caminhada P de distância $d(P) = d_t^n$ e de comprimento $|P| = n$. Como ela tem n arcos, contém um circuito C . Remove C de P para obter uma caminhada P' com menos que n arcos. Como

$$d(P') \geq d^{n-1}(t) > d^n(t) = d(P),$$

temos $d(C) < 0$. Caso $d^{n-1} = d^n$ nenhum circuito negativo é alcançável. ■

Teorema 1.5

Um potencial pode ser encontrado em tempo $O(nm)$ caso não tem circuitos negativos.

Prova. Adiciona um vértice s e arcos sv para todo $v \in V$ com $d_{sv} = 0$, roda Bellman-Ford e define $p_v := d_v$. Como não tem circuitos negativos $d_v = \text{dist}(s, v)$ e logo $d_{uv} \geq \text{dist}(s, v) - \text{dist}(s, u) = p_v - p_u$. ■

Caminhos mais curtos entre todos pares de vértices. Seja $d_k(s, t)$ a distância entre s e t usando somente vértices $\{s, t, v_1, \dots, v_k\}$ para alguma ordem de vértices $v_1, v_2, \dots, v_n$ e define $d_0(s, t) = d_{st}$ caso $st \in A$ e ∞ caso contrário. O algoritmo de *Floyd-Warshall* computa

$$d_{k+1}(s, t) := \min\{d_k(s, t), d_k(s, v_{k+1}) + d_k(v_{k+1}, t)\};$$

isso custa tempo $O(n^2)$ por iteração, logo não mais que $O(n^3)$ em total.

Com potenciais, podemos melhorar a complexidade (Johnson 1973): encontra um potencial p , aplica a transformação (1.6) e roda o algoritmo de Dijkstra n vezes. Isso custa somente $O(n(n \log n + m)) = O(nm + n^2 \log n)$ e caso o grafo tem $m = \Omega(n \log n)$ arcos temos custo $O(nm)$.

O método de Dial Assume distâncias inteiras e que temos um limite superior $\Delta \geq \max_{v \in V} \text{dist}(s, v)$. Neste caso podemos substituir a fila de prioridade no algoritmo de Dijkstra por $\Delta + 1$ “baldes” $L_0, \dots, L_\Delta$ (implementados como listas) onde balde L_i contém os vértices de distância $d_v = i$. Mantendo o número do menor balde não-vazio μ , é simples de ver que

- podemos atualizar μ em tempo amortizado $O(1)$ sobre todas n iterações (porque μ só aumenta para pesos não negativos);
- podemos atualizar os baldes em tempo constante sobre atualizações de distâncias.

Logo: temos uma complexidade de $O(m + \Delta) = O(m + nD)$, onde $D := \max_{a \in A} d_a$.

1.4.3. Notas

O algoritmo (assintoticamente) mais rápido para árvores geradoras mínimas usa *soft heaps* e possui complexidade $O(m\alpha(m, n))$, com α a função inversa de Ackermann (Chazelle, 2000; Kaplan e Zwick, 2009).

Karger propôs uma variante de heaps de Fibonacci que substituem a marca “cut” usado nos cortes em cascata por uma decisão randômica: com probabilidade 0.5 continua cortando, senão para. Além disso o heap é construído novamente com probabilidade $1/n$ depois de cada operação. Com isso “deletemin” possui complexidade esperada amortizada $\Theta(\log^2 n / \log \log n)$ (Li e Peebles, 2015).

Armazenar e atravessar árvores em ordem de van Emde Boas usando índices, similar à ordem por busca em largura é possível (Brodal et al., 2001). O consumo de memória das árvores de van Emde Boas pode ser reduzido para $O(n)$ (Dementiev et al., 2004; Cormen et al., 2009).

Mais sobre o fast marching method se encontra em Sethian (1999). Uma aplicação interessante é a solução do caixeiro viajante contínuo (Andrews e Sethian, 2007).

1.5. Filas de prioridade e heaps

Uma fila de prioridade mantém um conjunto de chaves com prioridades de forma que atualizar prioridades e acessar o elemento de menor prioridade seja eficiente. Ela possui aplicações em algoritmos para calcular árvores geradoras mínimas, caminhos mais curtos de um vértice para todos outros (algoritmo de Dijkstra) e em algoritmos de ordenação (heapsort).

1.5.1. Heaps binários

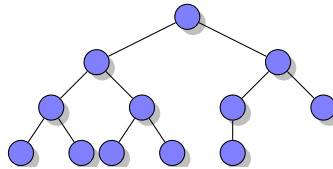
Teorema 1.6

Uma fila de prioridade pode ser implementada com custo $\text{insert} = O(\log n)$, $\text{deletemin} = O(\log n)$, $\text{update} = O(\log n)$. Portanto, uma árvore geradora mínima pode ser calculada em tempo $O(n \log n + m \log n)$.

Um *heap* é uma árvore com chaves nos vértices que satisfazem um critério de ordenação.

- *min-heap*: as chaves dos filhos são maiores ou iguais que a chave do pai;
- *max-heap*: as chaves dos filhos são menores ou iguais que a chave do pai.

Um *heap* binário é um heap em que cada vértice possui no máximo dois filhos. Implementaremos uma fila de prioridade com um heap binário *completo*. Um heap completo fica organizado de forma que possui folhas somente no último nível, da esquerda para direita. Isso garante uma altura de $O(\log n)$.



Positivo: Achar a chave com valor mínimo (operação *findmin*) custa $O(1)$. Como implementar a inserção? Ideia: Colocar na última posição e restabelecer a propriedade do min-heap, caso a chave seja menor que a do pai.

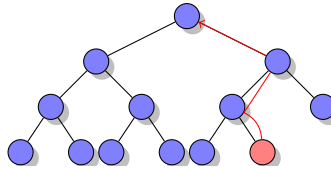
```

1  insert( $H, c$ ) :=
2  insere  $c$  na última posição  $p$ 
3  heapify-up( $H, p$ )
4
5  heapify-up( $H, p$ ) :=
6  if root( $p$ ) return

```

1. Algoritmos em grafos

```
7   if key(parent(p)) > key(p) then
8       swap(key(parent(p)), key(p))
9       heapify-up(H, parent(p))
10  end if
```



Lema 1.2

Seja T um min-heap. Decremente a chave do nó p . Após $\text{heapify-up}(T, p)$ temos novamente um min-heap. A operação custa $O(\log n)$.

Prova. Por indução sobre a profundidade k de p . Caso $k = 1$: p é a raiz, após o decremento já temos um min-heap e heapify-up não altera ele. Caso $k > 1$: Seja c a nova chave de p e d a chave de $\text{parent}(p)$. Caso $d \leq c$ já temos um min-heap e heapify-up não altera ele. Caso $d > c$ heapify-up troca c e d e chama $\text{heapify-up}(T, \text{parent}(p))$ recursivamente. Podemos separar a troca em dois passos: (i) copia d para p . (ii) copia c para $\text{parent}(p)$. Após passo (i) temos um min-heap T' e passo (ii) diminui a chave de $\text{parent}(p)$ e como a profundidade de $\text{parent}(p)$ é $k - 1$ obtemos um min-heap após a chamada recursiva, pela hipótese da indução.

Como a profundidade de T é $O(\log n)$, o número de chamadas recursivas também é, e como cada chamada tem complexidade $O(1)$, heapify-up tem complexidade $O(\log n)$. ■

Como remover? A ideia básica é a mesma: troca a chave com a menor chave dos filhos. Para manter o heap completo, colocaremos primeiro a chave da última posição na posição do elemento removido.

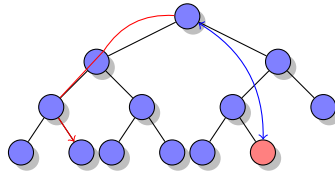
```
1  delete(H, p) :=
2      troca última posição com p
3      heapify-down(H, p)
4
5  heapify-down(H, p) :=
6      if p não possui filhos return
7      if p possui um filho then
8          if key(left(p)) < key(p) then swap(key(left(p)), key(p))
9          return
10     end if
```



```

11 { p possui dois filhos }
12 if key(p)>key(left(p)) or key(p)>key(right(p)) then
13     if (key(left(p))<key(right(p)) then
14         swap(key(left(p)),key(p))
15         heapify-down(H,left(p))
16     else
17         swap(key(right(p)),key(p))
18         heapify-down(H,right(p))
19     end if
20 end if

```



Lema 1.3

Seja T um min-heap. Incremente a chave do nó p . Após $\text{heapify-down}(T, p)$ temos novamente um min-heap. A operação custa $O(\log n)$.

Prova. Por indução sobre a altura k de p . Caso $k = 1$, p é uma folha e após o incremento já temos um min-heap e `heapify-down` não altera ele. Caso $k > 1$: Seja c a nova chave de p e d a chave do menor filho f . Caso $c \leq d$ já temos um min-heap e `heapify-down` não altera ele. Caso $c > d$ `heapify-down` troca c e d e chama `heapify-down( $T, f$ )` recursivamente. Podemos separar a troca em dois passos: (i) copia d para p . (ii) copia c para f . Após passo (i) temos um min-heap T' e passo (ii) aumenta a chave de f e como a altura de f é $k - 1$, obtemos um min-heap após a chamada recursiva, pela hipótese da indução. Como a altura de T é $O(\log n)$ o número de chamadas recursivas também, e como cada chamada tem complexidade $O(1)$, `heapify-down` tem complexidade $O(\log n)$. ■

Última operação: atualizar a chave.

```

1  update( $H, p, v$ ) :=
2      if  $v < \text{key}(p)$  then
3           $\text{key}(p) := v$ 
4          heapify-up( $H, p$ )
5      else
6           $\text{key}(p) := v$ 
7          heapify-down( $H, p$ )
8      end if

```

1. Algoritmos em grafos

Sobre a implementação Uma árvore binária completa pode ser armazenada em um vetor v que contém as chaves. Um pontador p a um elemento é simplesmente o índice no vetor. Caso o vetor contém n elementos e possui índices a partir de 0 podemos definir

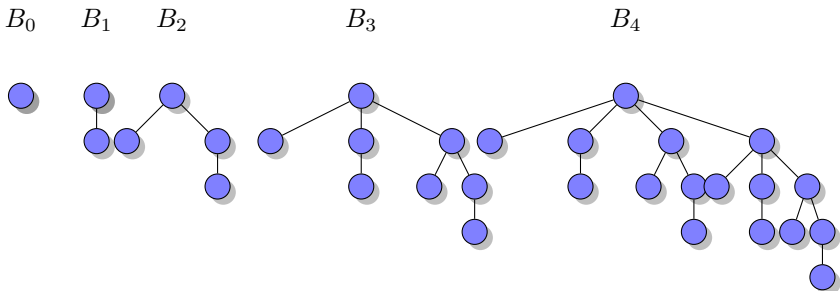
```
1 root( $p$ ) := return  $p = 0$ 
2 parent( $p$ ) := return  $\lfloor (p - 1)/2 \rfloor$ 
3 key( $p$ ) := return  $v[p]$ 
4 left( $p$ ) := return  $2p + 1$ 
5 right( $p$ ) := return  $2p + 2$ 
6 numchildren( $p$ ) := return  $\max(\min(n - \text{left}(p), 2), 0)$ 
```

Outras observações:

- Para chamar update, temos que conhecer a posição do elemento no heap. Para um conjunto de chaves compactos $[0, n]$ isso pode ser implementado usando um vetor pos, tal que $\text{pos}[c]$ é o índice da chave c no heap.
- A fila de prioridade não possui teste $u \in Q$ (linha 15 do algoritmo 1.3) eficiente. O teste pode ser implementado usando um vetor visited, tal que $\text{visited}[u]$ sse $u \in Q$.

1.5.2. Heaps binomiais

Um heap binomial é um coleção de *árvores binomiais* que satisfazem a ordenação de um heap. A árvore binomial B_0 consiste de um único vértice. A árvore binomial B_i possui uma raiz com filhos $B_0, \dots, B_{i-1}$. O posto de B_k é k . Um heap binomial contém no máximo uma árvore binomial de cada posto.



Lema 1.4

Uma árvore binomial tem as seguintes características:

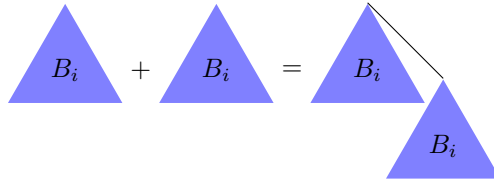
1. B_n possui 2^n vértices, 2^{n-1} folhas (para $n > 0$), e tem altura $n + 1$.

2. O nível k de B_n (a raiz tem nível 0) tem $\binom{n}{k}$ vértices. (Isso explica o nome.)

Prova. Exercício. ■

Observação 1.7

Podemos combinar dois B_i obtendo um B_{i+1} e mantendo a ordenação do heap: Escolhe a árvore com menor chave na raiz, e torna a outra filho da primeira. Chamaremos essa operação “link”. Ela tem custo $O(1)$ (veja observações sobre a implementação).



◇

Observação 1.8

Um B_i possui 2^i vértices. Um heap com n chaves consiste em $O(\log n)$ árvores. Isso permite juntar dois heaps binomiais em tempo $O(\log n)$. A operação é semelhante à soma de dois números binários com “carry”. Comece juntando os B_0 . Caso tenha zero, continue; caso tenha um, inclua no heap resultante. Caso tenha dois, o heap resultante não recebe um B_0 . Defina como “carry” o link dos dois B_0 ’s. Continue com os B_1 . Se tem zero, um ou dois, proceda como no caso dos B_0 . Caso tenha três, incluindo o “carry”, inclua um no resultado e defina como “carry” o link dos dois restantes. Continue dessa forma com as restantes árvores. Para heaps h_1, h_2 chamaremos essa operação $\text{meld}(h_1, h_2)$. ◇

Com a operação meld , podemos definir as seguintes operações:

- $\text{makeheap}(c)$: Retorne um B_0 com chave c . Custo: $O(1)$.
- $\text{insert}(h, c)$: $\text{meld}(h, \text{makeheap}(c))$. Custo: $O(\log n)$.
- $\text{getmin}(h)$: Mantendo um link para a árvore com o menor custo: $O(1)$.
- $\text{deletemin}(h)$: Seja B_k a árvore com o menor chave. Remove a raiz. Define dois heaps: h_1 é h sem B_k , h_2 consiste dos filhos de B_k , i.e. $B_0, \dots, B_{k-1}$. Retorne $\text{meld}(h_1, h_2)$. Custo: $O(\log n)$.

1. Algoritmos em grafos

- $\text{updatekey}(h, p, c)$: Como no caso do heap binário completo com custo $O(\log n)$.
- $\text{delete}(h, c)$: $\text{decreasekey}(h, c, -\infty)$; $\text{deletemin}(h)$

Em comparação com um heap binário completo ganhamos nada no caso pessimista. De fato, a operação insert possui complexidade pessimista $O(1)$ *amortizada*. Um insert individual pode ter custo $O(\log n)$. Do outro lado, isso acontece raramente. Uma análise amortizada mostra que em média sobre uma série de operações, um insert só custa $O(1)$. Observe que isso não é uma análise da complexidade média, mas uma análise da complexidade pessimista de uma série de operações.

Análise amortizada

Exemplo 1.5

Temos um contador binário com k bits e queremos contar de 0 até $2^k - 1$. Análise “tradicional”: um incremento tem complexidade $O(k)$, porque no caso pior temos que alterar k bits. Portanto todos incrementos custam $O(k2^k)$. Análise amortizada: “Poupamos” operações extras nos incrementos simples, para “gastá-las” nos incrementos caros. Concretamente, setando um bit, gastamos duas operações, uma para setar, outra seria “poupada”. Incrementando, usaremos as operações “poupadas” para zerar bits. Desta forma, um incremento custa $O(1)$ e temos custo total $O(2^k)$.

Uma outra forma da análise amortizada é através uma *função potencial* φ , que associa a cada estado de uma estrutura de dados um valor positivo (a “poupança”). O custo amortizado de uma operação que transforma uma estrutura e_1 em uma estrutura e_2 é $c - \varphi(e_1) + \varphi(e_2)$, com c o custo de operação. No exemplo do contador, podemos usar como $\varphi(i)$ o número de bits na representação binário de i . Agora, se temos um estado e_1

$$\underbrace{11 \dots 1}_p 0 \quad \underbrace{\dots}_q$$

p bits um q bits um

com $\varphi(e_1) = p + q$, o estado após de um incremento é

$$\underbrace{00 \dots 0}_0 1 \quad \underbrace{\dots}_q$$

com $\varphi(e_2) = 1 + q$. O incremento custa $c = p + 1$ operações e portanto o custo amortizado é

$$c - \varphi(e_1) + \varphi(e_2) = p + 1 - p - q + 1 + q = 2 = O(1).$$

◇

Resumindo: Dado um série de chamadas de uma operação com custos $c_1, \dots, c_n$ o custo amortizado da operação é $\sum_{1 \leq i \leq n} c_i/n$. Caso temos m operações diferentes, o custo amortizado da operação que ocorre nos índices $J \subseteq [1, m]$ é $\sum_{i \in J} c_i/|J|$.

As somas podem ser difíceis de avaliar diretamente. Um método para simplificar o cálculo do custo amortizado é o *método potencial*. Acha uma *função potencial* φ que atribui cada estrutura de dados antes da operação i um valor não-negativo $\varphi_i \geq 0$ e normaliza ela tal que $\varphi_1 = 0$. Atribui um custo amortizado

$$a_i = c_i - \varphi_i + \varphi_{i+1}$$

a cada operação. A soma dos custos não ultrapassa os custos originais, porque

$$\sum a_i = \sum c_i - \varphi_i + \varphi_{i+1} = \varphi_{n+1} - \varphi_1 + \sum c_i \geq \sum c_i$$

Portanto, podemos atribuir a cada tipo de operação $J \subseteq [1, m]$ o custo amortizado $\sum_{i \in J} a_i/|J|$. Em particular, se cada operação individual $i \in J$ tem custo amortizado $a_i \leq F$, o custo amortizado desse tipo de operação é F .

Exemplo 1.6

Queremos implementar uma tabela dinâmica para um número desconhecido de elementos. Uma estratégia é reservar espaço para n elementos, manter a última posição livre p , e caso $p > n$ alocar uma nova tabela de tamanho maior. Uma implementação dessa ideia é

```

1  insert(x) :=
2    if p > n then
3      aloca nova tabela de tamanho t = max{2n, 1}
4      copia os elementos x_i, 1 ≤ i < p para nova tabela
5      n := t
6    end if
7    x_p := x
8    p := p + 1

```

com valores iniciais $n := 0$ e $p := 0$. O custo de insert é $O(1)$ caso existe ainda espaço na tabela, mas $O(n)$ no pior caso.

Uma análise amortizada mostra que a complexidade amortizada de uma operação é $O(1)$. Seja Cn o custo das linhas 3–5 e D o custo das linhas 7–8. Escolhe a função potencial $\varphi(n) = 2Cp - Dn$. A função φ é satisfaz os critérios de um potencial, porque $p \geq n/2$, e inicialmente temos $\varphi(0) = 0$. Com isso o custo amortizado caso tem espaço na tabela é

$$\begin{aligned} a_i &= c_i - \varphi(i-1) + \varphi(i) \\ &= D - (2C(p-1) - Dn) + (2Cp - Dn) = C + 2C = O(1). \end{aligned}$$

1. Algoritmos em grafos

Caso temos que alocar uma nova tabela o custo é

$$\begin{aligned} a_i &= c_i - \varphi(i-1) + \varphi(i) = D + Cn - (2C(p-1) - Dn) + (2Cp - 2Dn) \\ &= C + Dn + 2C - Dn = O(1). \end{aligned}$$

◇

Custo amortizado do heap binomial Nosso potencial no caso do heap binomial é o número de árvores no heap. O custo de `getmin` e `updatekey` não altera o potencial e por isso permanece o mesmo. `makeheap` cria uma árvore que custa mais uma operação, mas permanece $O(1)$. `deletemin` pode criar $O(\log n)$ árvores novas, porque o heap contém no máximo um $B_{\lceil \log n \rceil}$ que tem $O(\log n)$ filhos, e permanece também com custo $O(\log n)$. Finalmente, `insert` reduz o potencial para cada link no `meld` e portanto agora custa somente $O(1)$ amortizado, com o mesmo argumento que no exemplo 1.5.

Desvantagem: a complexidade (amortizada) assintótica de calcular uma árvore geradora mínima permanece $O(n \log n + m \log n)$.

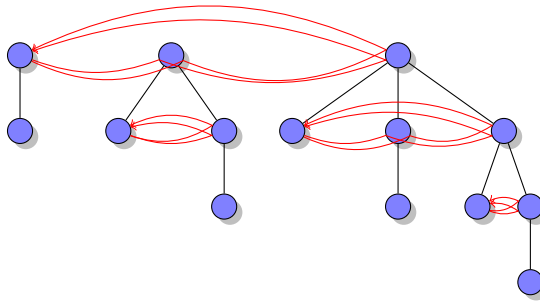
Meld preguiçosa Ao invés de reorganizar os dois heaps em um `meld`, podemos simplesmente concatená-los em tempo $O(1)$. Isso pode ser implementado sem custo adicional nas outras operações. A única operação que não tem complexidade $O(1)$ é `deletemin`. Agora temos uma coleção de árvores binomiais não necessariamente de posto diferente. O `deletemin` reorganiza o heap, tal que obtemos um heap binomial com árvores de posto único novamente. Para isso, mantemos um vetor com as árvores de cada posto, inicialmente vazio. Sequencialmente, cada árvore no heap, será integrado nesse vetor, executando operações `link` só for necessário. O tempo amortizado de `deletemin` permanece $O(\log n)$.

Usaremos um potencial φ que é o dobro do número de árvores. Supondo que antes do `deletemin` temos t árvores e executamos l operações `link`, o custo amortizado é

$$(t+l) - 2t + 2(t-l) = t-l.$$

Mas $t-l$ é o número de árvores depois do `deletemin`, que é $O(\log n)$, porque todas árvores possuem posto diferente.

Sobre a implementação Uma forma eficiente de representar heaps binomiais, é em forma de apontadores. Além dos apontadores dos filhos para os pais, cada pai possui um apontador para um filho e os filhos são organizados em uma lista encadeada dupla. Mantemos uma lista encadeada dupla também das raízes. Desta forma, a operação `link` pode ser implementada em $O(1)$.



1.5.3. Heaps Fibonacci

Um heap Fibonacci é uma modificação de um heap binomial, com uma operação `decreasekey` de custo $O(1)$. Com isso, uma árvore geradora mínima pode ser calculada em tempo $O(m + n \log n)$. Para conseguir `decreasekey` em $O(1)$ não podemos mais usar `heapify-up`, porque `heapify-up` custa $O(\log n)$.

Primeira tentativa:

- `delete(h, p)`: Corta p de h e executa um `meld` entre o resto de h e os filhos de p . Uma alternativa é implementar `delete(h, p)` como `decreasekey(h, p, -\infty)` e `deletemin(h)`.
- `decreasekey(h, p)`: A ordenação do heap pode ser violada. Corta p e execute um `meld` entre o resto de h e p .

Problema com isso: após de uma série de operações `delete` ou `decreasekey`, a árvore pode se tornar “esparso”, i.e. o número de vértices não é mais exponencial no posto da árvore. A análise da complexidade das operações como `deletemin` depende desse fato para garantir que temos $O(\log n)$ árvores no heap. Consequência: Temos que garantir, que uma árvore não fica “podado” demais. Solução: Permitiremos cada vértice perder no máximo dois filhos. Caso o segundo filho é removido, cortaremos o próprio vértice também. Para cuidar dos cortes, cada nó mantém ainda um valor booleana que indica, se já foi cortado um filho. Observe que um corte pode levar a uma série de cortes e por isso se chama de corte em cascatas (ingl. *cascading cuts*). Um corte em cascata termina na pior hipótese na raiz. A raiz é o único vértice em que permitiremos cortar mais que um filho. Por isso não mantemos flag na raiz.

Implementações Denotamos com h um heap, c uma chave e p um elemento do heap. `minroot(h)` é o elemento do heap que corresponde com a raiz da chave mínima, e `cut(p)` é uma marca que verdadeiro, se p já perdeu um filho.

1. Algoritmos em grafos

```
1  insert( $h, c$ ) :=
2    meld(makeheap( $c$ ))
3
4  getmin( $h$ ) :=
5    return minroot( $h$ )
6
7  delete( $h, p$ ) :=
8    decreasekey( $h, p, -\infty$ )
9    deletemin( $h$ )
10
11 meld( $h_1, h_2$ ) :=
12    $h$  := lista com raízes de  $h_1$  e  $h_2$  (em  $O(1)$ )
13   minroot( $h$ ) :=
14     if key(minroot( $h_1$ )) < key(minroot( $h_2$ ))  $h_1$  else  $h_2$ 
15
16 decreasekey( $h, p, c$ ) :=
17   key( $p$ ) :=  $c$ 
18   if  $c < \text{key}(\text{minRoot}(h))$ 
19     minRoot( $h$ ) :=  $p$ 
20   if not root( $p$ )
21     if key(parent( $p$ )) > key( $p$ )
22       corta  $p$  e adiciona na lista de raízes de  $h$ 
23       cut( $p$ ) := false
24       cascading-cut( $h, \text{parent}(p)$ )
25
26 cascading-cut( $h, p$ ) :=
27   {  $p$  perdeu um filho }
28   if root( $p$ )
29     return
30   if (not cut( $p$ )) then
31     cut( $p$ ) := true
32   else
33     corta  $p$  e adiciona na lista de raízes de  $h$ 
34     cut( $p$ ) := false
35     cascading-cut( $h, \text{parent}(p)$ )
36   end if
37
38 deletemin( $h$ ) :=
39   remover minroot( $h$ )
40   juntar as listas do resto de  $h$  e dos filhos de minroot( $h$ )
41   { reorganizar heap }
```



```

42  determina o posto máximo  $M = M(n)$  de  $h$ 
43   $r_i := \text{undefined}$  para  $0 \leq i \leq M$ 
44  for toda raiz  $r$  do
45      remove  $r$  da lista de raízes
46       $d := \text{degree}(r)$ 
47      while ( $r_d$  not undefined) do
48           $r := \text{link}(r, r_d)$ 
49           $r_d := \text{undefined}$ 
50           $d := d + 1$ 
51      end while
52       $r_d := r$ 
53  end for
54  definir a lista de raízes pelas entradas definidas  $r_i$ 
55  determinar o novo minroot
56
57   $\text{link}(h_1, h_2) :=$ 
58      if ( $\text{key}(h_1) < \text{key}(h_2)$ )
59           $h := \text{makechild}(h_1, h_2)$ 
60      else
61           $h := \text{makechild}(h_2, h_1)$ 
62       $\text{cut}(h_1) := \text{false}$ 
63       $\text{cut}(h_2) := \text{false}$ 
64      return  $h$ 

```

Para concluir que a implementação tem a complexidade desejada temos que provar que as árvores com no máximo um filho cortado não ficam esparsas demais e analisar o custo amortizado das operações.

Custo amortizado Para análise usaremos um potencial de $c_1t + c_2m$ sendo t o número de árvores, m o número de vértices marcados e c_1, c_2 constantes. As operações `makeheap`, `insert`, `getmin` e `meld` (preguiçoso) possuem complexidade (real) $O(1)$. Para `decreasekey` temos que considerar o caso em que o corte em cascata remove mais que uma subárvore. Supondo que cortamos n árvores, o número de raízes é $t + n$ após os cortes. Para todo corte em cascata, a árvore cortada é desmarcada, logo temos no máximo $m - (n - 1)$ marcas depois. Portanto custo amortizado é

$$O(n) - (c_1t + c_2m) + (c_1(t + n) + c_2(m - (n - 1))) = c_0n - (c_2 - c_1)n + c_2$$

e com $c_2 - c_1 \geq c_0$ temos custo amortizado constante $c_2 = O(1)$.

Com posto máximo M , a operação `deletemin` tem o custo real $O(M + t)$, com as seguintes contribuições

1. Algoritmos em grafos

- Linha 43: $O(M)$.
- Linhas 44–51: $O(M + t)$ com t o número inicial de árvores no heap. A lista de raízes contém no máximo as t árvores de h e mais M filhos da raiz removida. O laço total não pode executar mais que $M + t$ operações link, porque cada um reduz o número de raízes por um.
- Linhas 54–55: $O(M)$.

Seja m o número de marcas antes do deletemin e m' o número depois. Como deletemin marca nenhum vértice, temos $m' \leq m$. O número de árvores t' depois de deletemin satisfaz $t' \leq M$ porque deletemin garante que existe no máximo uma árvore de cada posto. Portanto, o potencial depois de deletemin é $\varphi' = c_1 t' + c_2 m' \leq c_1 M + c_2 m$, e o custo amortizado é

$$\begin{aligned} O(M + t) - (c_1 t + c_2 m) + \varphi' &\leq O(M + t) - (c_1 t + c_2 m) + (c_1 M + c_2 m) \\ &= (c_0 + c_1)M + (c_0 - c_1)t \end{aligned}$$

e com $c_1 \geq c_0$ temos custo amortizado $O(M)$.

Um limite para M Para provar que deletemin tem custo amortizado $\log n$, temos que provar que $M = M(n) = O(\log n)$. Esse fato segue da maneira “cautelosa” com que cortamos vértices das árvores.

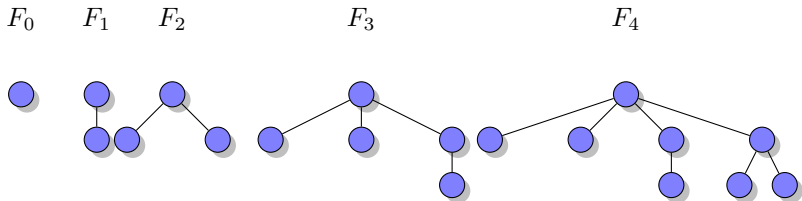
Lema 1.5

Seja p um vértice arbitrário de um heap Fibonacci. Considerando os filhos na ordem temporal em que eles foram introduzidos, filho i possui pelo menos $i - 2$ filhos.

Prova. No instante em que o filho i foi introduzido, p estava com pelo menos $i - 1$ filhos. Portanto i estava com pelo menos $i - 1$ filhos também. Depois filho i perdeu no máximo um filho, e portanto possui pelo menos $i - 2$ filhos.

■

Quais as menores árvores, que satisfazem esse critério?



Lema 1.6

Cada subárvore com uma raiz p com k filhos possui pelo menos F_{k+2} vértices.

Prova. Seja S_k o número mínimo de vértices para uma subárvore cuja raiz possui k filhos. Sabemos que $S_0 = 1$, $S_1 = 2$. Define $S_{-2} = S_{-1} = 1$. Com isso obtemos para $k \geq 1$

$$S_k = \sum_{0 \leq i \leq k} S_{i-2} = S_{k-2} + S_{k-3} + \cdots + S_{-2} = S_{k-2} + S_{k-1}.$$

Comparando S_k com os números Fibonacci

$$F_k = \begin{cases} k & \text{se } 0 \leq k \leq 1 \\ F_{k-2} + F_{k-1} & \text{se } k \geq 2 \end{cases}$$

e observando que $S_0 = F_2$ e $S_1 = F_3$ obtemos $S_k = F_{k+2}$. Usando que $F_n \in \Theta(\Phi^n)$ com $\Phi = (1 + \sqrt{5})/2$ (exercício!) conclui a prova. ■

Corolário 1.1

O posto máximo de um heap Fibonacci com n elementos é $O(\log n)$.

Sobre a implementação A implementação da árvore é a mesma que no caso de heaps binomiais. Uma vantagem do heap Fibonacci é que podemos usar os nós como ponteiros – lembre que a operação `decreasekey` precisa disso, porque os heaps não possuem uma operação de busca eficiente. Isso é possível, porque sem `heapify-up` e `heapify-down`, os ponteiros mantêm-se válidos.

1.5.4. Rank-pairing heaps

Haeupler et al. (2009) propõem um rank-pairing heap (um heap “emparelhando postos”) com as mesmas garantias de complexidade que um heap Fibonacci e uma implementação simplificada e mais eficiente na prática (ver observação 1.11).

Torneios Um *torneio* é uma representação alternativa de heaps. Começando com todos os elementos, vamos repetidamente comparar pares de elementos, e promover o vencedor para o próximo nível (Fig. 1.3(a)). Uma desvantagem de representar torneios explicitamente é o espaço para chaves redundantes. Por exemplo, o campeão (i.e. o menor elemento) ocorre $O(\log n)$ vezes. A figura 1.3(b) mostra uma representação sem chaves repetidas. Cada chave é representada somente na comparação mais alta que ela ganhou, as outras comparações ficam vazias. A figura 1.3(c) mostra uma representação compacta

1. Algoritmos em grafos

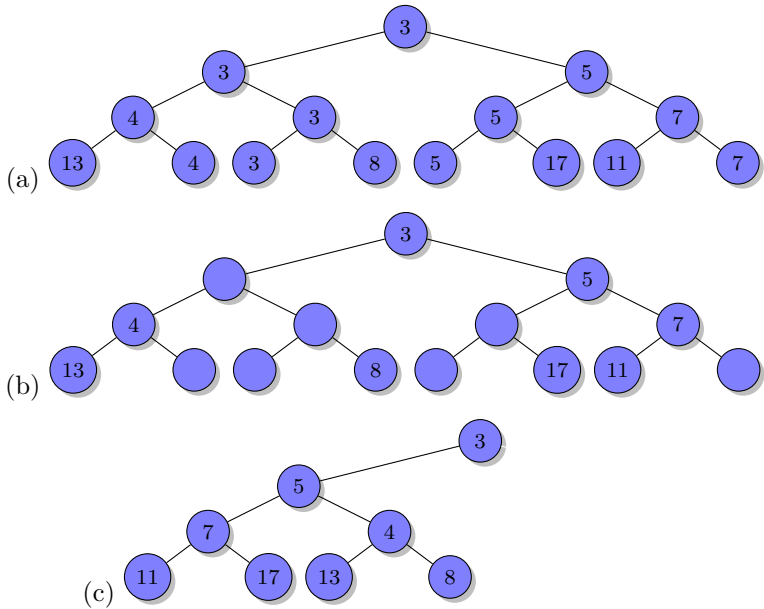


Figura 1.3.: Representações de heaps.

em forma de *semi-árvore*. Numa semi-árvore cada elemento possui um filho *ordenado* (na figura o filho da esquerda) e um filho *não-ordenado* (na figura o filho da direita). O filho ordenado é o perdedor da comparação direta com o elemento, enquanto o filho não-ordenado é o perdedor da comparação com o irmão vazio. A raiz possui somente um filho ordenado.

Cada elemento de um torneio possui um *posto*. Por definição, o posto de uma folha é 0. Uma comparação *justa* entre dois elementos do mesmo posto r resulta num elemento com posto $r + 1$ no próximo nível. Numa comparação *injusta* entre dois elementos com postos diferentes, o posto do vencedor é definido pelo maior dos dois postos dos participantes (uma alternativa é que o posto fica o mesmo). O posto de um elemento representa um limite inferior do número de elementos que perderam contra ele:

Lema 1.7

Um torneio com campeão de posto k possui pelo menos 2^k elementos.

Prova. Por indução. Caso um vencedor possui posto k temos duas possibilidades: (i) foi o resultado de uma comparação justa, com dois participantes

com posto $k - 1$ e pela hipótese da indução com pelo menos 2^{k-1} elementos, tal que o vencedor ganhou contra pelo menos 2^k elementos. (ii) foi resultado de uma comparação injusta. Neste caso um dos participantes possuiu posto k e o vencedor novamente ganhou contra pelo menos 2^k elementos. ■

Cada comparação injusta torna o limite inferior dado pelo posto menos preciso. Por isso uma regra na construção de torneios é fazer o maior número de comparações justas possíveis. A representação de um elemento de heap possui quatro campos para a chave (c), o posto (r), o filho ordenado (o) e o filho não-ordenado (u):

```
1 def Node(c,r,o,u)
```

Podemos implementar as operações de uma fila de prioridade (sem update ou decreasekey) como segue:

```
1 { compara duas árvores }
2 link( $t_1, t_2$ ) :=
3   if  $t_1.c < t_2.c$  then
4     return makechild( $t_1, t_2$ )
5   else
6     return makechild( $t_2, t_1$ )
7   end if
8
9 makechild(s,t) :=
10  t.u := s.o
11  s.o := t
12  setrank(t)
13  s.r := s.r + 1
14  return s
15
16 setrank(t) :=
17  if t.o.r = t.u.r
18    t.r = t.o.r + 1
19  else
20    t.r = max(t.o.r, t.u.r)
21  end if
22
23 { cria um heap com um único elemento com chave c }
24 make-heap(c) := return Node(c,0,undefined,undefined)
25
26 { insere chave c no heap }
27 insert(h,c) := link(h,make-heap(c))
```

1. Algoritmos em grafos

```
28
29 { união de dois heaps }
30 meld( $h_1, h_2$ ) := link( $h_1, h_2$ )
31
32 { elemento mínimo do heap }
33 getmin(h) := return h
34
35 { deleção do elemento mínimo do heap }
36 deletemin(h) :=
37   aloca array  $r_0 \dots r_{h.o.r+1}$ 
38   t = h.o
39   while t not undefined do
40     t' := t.u
41     t.u := undefined
42     register(t,r)
43     t := t'
44   end while
45   h' := undefined
46   for  $i = 0, \dots, h.o.r + 1$  do
47     if  $r_i$  not undefined
48       h' := link(h',  $r_i$ )
49     end if
50   end for
51   return h'
52 end
53
54 register(t,r) :=
55   if  $r_{t.o.r+1}$  is undefined then
56      $r_{t.o.r+1}$  := t
57   else
58     t := link(t,  $r_{t.o.r+1}$ )
59      $r_{t.o.r+1}$  := undefined
60     register(t,r)
61   end if
62 end
```

(A figura 1.4 visualiza a operação “link”).

Observação 1.9

Todas comparações de “register” são justas. As comparações injustas ocorrem na construção da árvore final nas linhas 35–39. $\diamond$

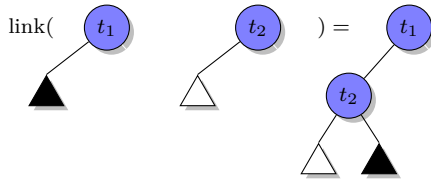


Figura 1.4.: A operação “link” para semi-árvores no caso $t_1.c < t_2.c$.

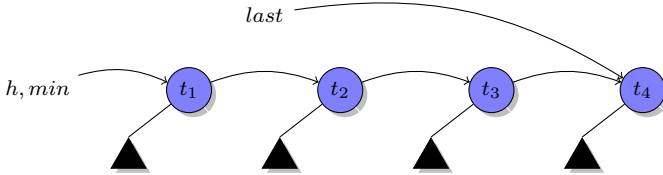


Figura 1.5.: Representação de um heap binomial.

Lema 1.8

Num torneio balanceado o custo amortizado de “make-heap”, “insert”, “meld” e “getmin” é $O(1)$, o custo amortizado de “deletemin” é $O(\log n)$.

Prova. Usaremos o número de comparações injustas no torneio como potencial. “make-heap” e “getmin” não alteram o potencial, “insert” e “meld” aumentam o potencial por no máximo um. Portanto a complexidade amortizada dessas operações é $O(1)$. Para analisar “deletemin” da raiz r do torneio vamos supor que houve k comparações injustas com r . Além dessas comparações injustas, r participou em no máximo $\log n$ comparações justas pelo lema 1.7. Em soma vamos liberar no máximo $k + \log n$ árvores, que reduz o potencial por k , e com no máximo $k + \log n$ comparações podemos produzir um novo torneio. Dessas $k + \log n$ comparações no máximo $\log n$ são comparações injustas. Portanto o custo amortizado é $k + \log n - k + \log n = 2 \log n = O(\log n)$. ■

Heaps binomiais com varredura única O custo de representar o heap numa árvore única é permitir comparações injustas. Uma alternativa é permitir somente comparações justas, que implica em manter uma coleção de $O(\log n)$ árvores. A estrutura de dados resultante é similar com os heaps binomiais: manteremos uma lista (simples) de raízes das árvores, junto com um ponteiro para a árvore com a raiz de menor valor. O heap é representado pela raiz de menor valor, ver Fig. 1.5.

1. Algoritmos em grafos

```
1  insert(h,c) :=
2    insere make-heap(c) na lista de raizes
3    atualize a árvore mínima
4
5  meld( $h_1, h_2$ ) :=
6    concatena as listas de  $h_1$  e  $h_2$ 
7    atualize a árvore mínima
8
9  Somente “deletemin” opera diferente agora:
10
11 deletemin(h) :=
12   aloca um array de listas  $r_0 \dots r_{\lceil \log n \rceil}$ 
13   remove a árvore mínima da lista de raizes
14   distribui as restantes árvores sobre  $r$ 
15
16    $t := h.o$ 
17   while  $t$  not undefined do
18      $t' := t.u$ 
19      $t.u := \text{undefined}$ 
20     insere  $t$  na lista  $r_{t.o.r+1}$ 
21      $t := t'$ 
22   end while
23
24   { executa o maior número possível }
25   { de comparações justas num único passo }
26
27    $h := \text{undefined}$  { lista final de raizes }
28   for  $i = 0, \dots, \lceil \log n \rceil$  do
29     while  $|r_i| \geq 2$ 
30        $t := \text{link}(r_i.\text{head}, r_i.\text{head}.\text{next})$ 
31       insere  $t$  na lista  $h$ 
32       remove  $r_i.\text{head}, r_i.\text{head}.\text{next}$  da lista  $r_i$ 
33     end if
34     if  $|r_i| = 1$  insere  $r_i.\text{head}$  na lista  $h$ 
35   end for
36   return  $h$ 
```

Observação 1.10

Continuando com comparações justas até sobrar somente uma árvore de cada posto, obteremos um heap binomial. $\diamond$

Lema 1.9

Num heap binomial com varredura única o custo amortizado de “make-heap”, “insert”, “meld”, “getmin” é $O(1)$, o custo amortizado de “deletemin” é $O(\log n)$.

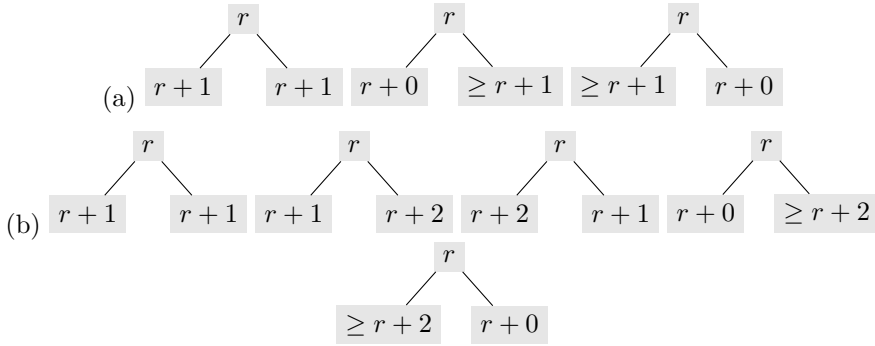


Figura 1.6.: Diferenças no posto de rp-heaps do tipo 1 (a) e tipo 2 (b).

Prova. Usaremos o dobro do número de árvores como potencial. “getmin” não altera o potencial. “make-heap”, “insert” e “meld” aumentam o potencial por no máximo dois (uma árvore), e portanto possuem custo amortizado $O(1)$. “deletemin” libera no máximo $\log n$ árvores, porque todas comparações foram justas. Com um número total de h árvores, o custo de deletemin é $O(h)$. Sem perda de generalidade vamos supor que o custo é h . A varredura final executa pelo menos $(h - \log n)/2 - 1$ comparações justas, reduzindo o potencial por pelo menos $h - \log n - 2$. Portanto o custo amortizado de “deletemin” é $h - (h - \log n - 2) = \log n + 2 = O(\log n)$. ■

rp-heaps O objetivo do rp-heap é adicionar ao heap binomial de varredura única uma operação “decreasekey” com custo amortizado $O(1)$. A ideia e os problemas são os mesmos do heap Fibonacci: (i) para tornar a operação eficiente, vamos cortar a sub-árvore do elemento cuja chave foi diminuída. (ii) o heap Fibonacci usava cortes em cascata para manter um número suficiente de elementos na árvore; no rp-heap ajustaremos os postos do heap que perde uma sub-árvore. Para poder cortar sub-árvores temos que permitir uma folga nos postos. Num heap binomial a diferença do posto de um elemento com o posto do seu pai (caso existe) sempre é um. Num rp-heap do tipo 1, exigimos somente que os dois filhos de um elemento possuem diferença do posto 1 e 1, ou 0 e ao menos 1. Num rp-heap do tipo 2, exigimos que os dois filhos de um elemento possuem diferença do posto 1 e 1, 1 e 2 ou 0 e pelo menos 2. (Figura 1.6.)

Com isso podemos implementar o “decreasekey” (para rp-heaps do tipo 2) como segue:

1. Algoritmos em grafos

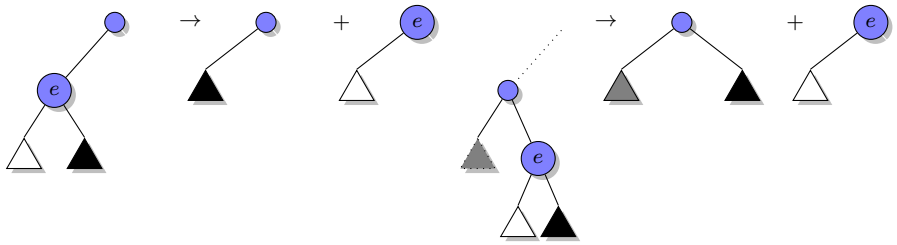


Figura 1.7.: A operação “decreasekey”.

```

1  decreasekey(h,e, $\Delta$ ) :=
2    e.c := e.c -  $\Delta$ 
3    if root(e)
4      return
5    if parent(e).o = e then
6      parent(e).o := e.u
7    else
8      parent(e).u := e.u
9    end if
10   parent(e).u := parent(e)
11   e.u := undefined
12   u := parent(e)
13   parent(e) := undefined
14   insere e na lista de raízes de h
15   decreaserank(u)
16
17  rank(e) :=
18    if e is undefined
19      return -1
20    else
21      return e.r
22
23  decreaserank(u) :=
24    if root(u)
25      return
26    if rank(u.o) > rank(u.u)+1 then
27      k := rank(u.o)
28    else if rank(u.u) > rank(u.o)+1 then
29      k := rank(u.u)
30    else

```

```

31     k = max(rank(u.o), rank(u.u)) + 1
32 end if
33 if u.r = k then
34     return
35 else
36     u.r := k
37     decreaserank(parent(u))
38
39 delete(h, e) :=
40     decreasekey(h, e, -∞)
41     deletemin(h)

```

Observação 1.11

Para implementar o rp-heap precisamos além dos ponteiros para o filho ordenado e não-ordenado um ponteiro para o pai do elemento. A (suposta) eficiência do rp-heap vem do fato que o `decreasekey` altera os postos do heap, e pouco da estrutura dele e do fato que ele usa somente três ponteiros por elemento, e não quatro como o heap Fibonacci. $\diamond$

Lema 1.10

Uma semi-árvore do tipo 2 com posto k contém pelo menos ϕ^k elementos, sendo $\phi = (1 + \sqrt{5})/2$ a razão áurea.

Prova. Por indução. Para folhas o lema é válido. Caso a raiz com posto k não é folha podemos obter duas semi-árvores: a primeira é o filho da raiz sem o seu filho não-ordenado, e a segunda é a raiz com o filho não ordenado do seu filho ordenado (ver Fig. 1.8). Pelas regras dos postos de árvores de tipo dois, essas duas árvores possuem postos $k - 1$ e $k - 1$, ou $k - 1$ e $k - 2$ ou k e no máximo $k - 2$. Portanto, o menor número de elementos n_k contido numa semi-árvore de posto k satisfaz a recorrência

$$n_k = n_{k-1} + n_{k-2}$$

que é a recorrência dos números Fibonacci. $\blacksquare$

Lema 1.11

As operações “`decreasekey`” e “`delete`” possuem custo amortizado $O(1)$ e $O(\log n)$

Prova. Ver Haeupler et al. (2009). $\blacksquare$

1.5.5. Heaps ociosos

Introdução

Objetivo: operações com a mesma complexidade amortizada que heaps de Fibonacci. Para um heap h , chave k e elemento e temos as operações:

1. Algoritmos em grafos

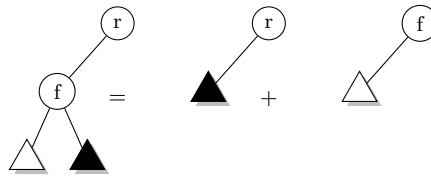


Figura 1.8.: Separar uma semi-árvore de posto k em duas.

- `make-heap()`: $O(1)$
- `find-min( $h$ )/getmin( $h$ )`: $O(1)$
- `meld( $h_1, h_2$ )`: $O(1)$
- `insert( $e, k, h$ )`: $O(1)$
- `decrease-key( $e, k, h$ )`: $O(1)$
- `delete( $e, h$ )`: $O(\log n)$
- `delete-min( $h$ )`: $O(\log n)$

Ideia principal: a operação `delete` esvazia nós, produzindo nós ocos (ingl. *hollow nodes*), a operação `decrease-key` é um `delete`, seguido por um `insert`. Teremos duas medidas:

n Número de elementos no heap

N Número de nós no heap = # de elementos + # de nós ocos = # operações `insert` + # operações `decrease-key`

Variantes de heaps ocos:

- Heaps ansiosos (ingl. “eager heaps”) com múltiplas raízes.
- Heaps ansiosos com uma única raíz.
- Heaps preguiçosos.

```
1 def Node =
2   item // elemento
3   key  // chave
4   fc   // ponteiro para primeiro filho
5   ns   // ponteiro para próximo irmão
```

```

6     rank // posto do nó
7
8     def Item =
9         no // nó correspondente
10        // mais dados satelites

```

Operação básica: link Um *link* gera um vencedor e um perdedor, que se torna filho do vencedor, e aumenta o posto do vencedor.

```

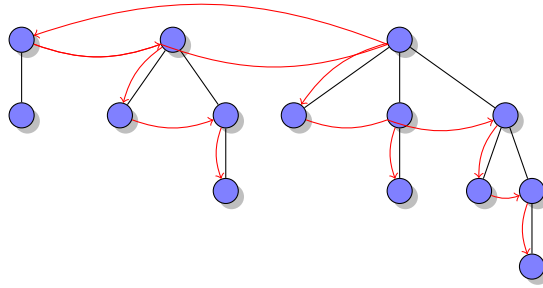
1     (ranked)link( $t_1, t_2$ ) :=
2         if  $t_1.key \leq t_2.key$ 
3             return makechild( $t_1, t_2$ )
4         else
5             return makechild( $t_2, t_1$ )
6
7     makechild( $w, l$ ) :=
8         l.ns := w.fc
9         w.fc := l
10        w.rank := w.rank+1
11        return w

```

Representação básica

- Lista simples circular de árvores com ordenação do heap, representada por um ponteiro à árvore cuja raiz contém a menor chave (chamada a *raiz mínima*).
- Cada *nó cheia* armazena um item. Podem existir *nós ocos* sem item.
- Nós ocos nunca mais ficam cheias, eles podem somente ser destruídos.
- Filhos ficam armazenados em listas simples, em ordem não-crescente de postos.

1. Algoritmos em grafos



```

1  make-heap() := return null
2
3  make-heap(e,k) := return Node(e,k,null,self,0)
4
5  getmin(h) := h
6
7  findmin(h) := return h is not null? h.item : null
8
9  meld(h1,h2) :=
10     if h1 is null return h2
11     if h2 is null return h1
12     swap(h1.ns,h2.ns) // cria uma lista circular simples
13     if h1.key ≤ h2.key return h1 else return h2
14
15  insert(e,k,h) := meld(make-heap(e,k),h)
16
17  decrease-key(e,k,h) :=
18     u = e.node
19     v = make-heap(e,k)
20     v.rank = max{0,u.rank-2}
21     // desloca os filhos de postos 0,...,rank-2 para v
22     if u.rank ≥ 2
23         v.fc := u.fc.ns.ns
24         u.fc.ns.ns := null
25     return meld(v,h)
26
27  delete(e,h) :=
28     e.node.item := null
29     if e.node = h
30         delete-min(h)

```

```

31
32 delete-min(h) :=
33     if h is null: return
34     h.node.item := null
35
36     aloca um array  $R_0, R_1, \dots, R_M$ 
37     // repetidamente remove raízes ociosas e une os heaps
38     r:=h
39     repeat
40         rn := r.ns
41         link-heap(r,R)
42         r:=rn
43     until r==h
44
45     // reconstrói o heap
46     h:=null
47     for i=0,...,M
48         if  $R_i$  is not null
49              $R_i.ns := R_i$ 
50             h := meld(h,  $R_i$ )
51     return h
52
53 link-heap(h,R) :=
54     if h is hollow
55         r:=h.fc
56         while r is not null
57             rn := r.ns
58             link-heap(r,R)
59             r := rn
60         destroy node h
61     else
62         i := h.rank
63         while  $R_i$  is not null
64             h := link(h,  $R_i$ )
65              $R_i := null$ 
66             i := i + 1
67     end
68      $R_i := h$ 

```

Invariantes

1. Ordenação do heap.
2. Invariante do posto: cada nó de posto r possui r filhos com postos $0, \dots, r-1$, exceto no caso $r \geq 2$ e o nó foi esvaziada por uma operação decrease-key. Neste caso o nó possui dois filhos de postos $r-1$ e $r-2$.

Corretude

Teorema 1.7

Heaps com nós ocos implementam corretamente todas operação e mantém as invariantes.

Prova. Por indução sobre o número de operações. ■

Lembrança: os números de Fibonacci são definidos por $F_0 = 0, F_1 = 1, F_{i+2} = F_i + F_{i+1}$, para $i \geq 0$ e temos $F_{i+2} \geq \Phi^i$, com a razão áurea $\Phi = (1 + \sqrt{5})/2$.

Teorema 1.8

Um nó de posto r possui pelo menos $F_{r+3} - 1$ descendentes (cheios ou ocos), incluindo o próprio nó, na árvore.

Prova. Por indução sobre r . Para $r = 0$, temos $F_3 - 1 = 1$, e para $r = 1$ temos $F_4 - 1 = 2$ e a afirmação está correta, porque para $r < 2$ um nó não perde filhos caso for esvaziado. Para $r \geq 2$ pela invariante do posto temos pelo menos dois filhos com postos $r-1$ e r_2 . Pela hipótese da indução eles tem pelo menos $F_{r+1} - 1$ e $F_{r+2} - 1$ descendentes e logo r possui pelo menos $F_{r+1} - 1 + F_{r+2} - 1 + 1 = F_{r+3} - 1$ descendentes. ■

Corolário 1.2

Depois uma operação delete-min o número de árvores é no máximo $\lceil \log_\Phi N \rceil = O(\log N)$ porque temos no máximo uma árvore por posto. Logo podemos escolher $M = \lceil \log_\Phi N \rceil$ na operação delete-min.

Teorema 1.9

O tempo amortizado por operação num heap oco é $O(1)$, exceto para as operações delete e delete-min, que tem complexidade $O(\log N)$ para um heap com N nós.

Prova. Todas operações exceto a deleção do elemento mínimo possuem tempo $O(1)$ no caso pessimista. O custo de uma deleção é $O(H+T)$ com H o número de nós ocos destruídos, e T o número de árvores antes das operações link. Depois das operações link temos no máximo $\log_\Phi N$ árvores, logo faremos pelo menos $T - \log_\Phi N$ operações link e no máximo $\log_\Phi N$ operações meld. Logo o custo total é $O(1)$ por destruição de um nó oco, e por link, mas $O(\log N)$.

Tabela 1.2.: Complexidade das operações de uma fila de prioridade. Complexidades em negrito são amortizados. (1): meld preguiçoso.

	insert	getmin	deletemin	update	decreasekey	delete
Vetor	$O(1)$	$O(1)$	$O(n)$	$O(1)$	(update)	$O(1)$
Lista ordenada	$O(n)$	$O(1)$	$O(1)$	$O(n)$	(update)	$O(1)$
Heap binário	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)$	(update)	$O(\log n)$
Heap binomial	$O(1)$	$O(1)$	$O(\log n)$	$O(\log n)$	(update)	$O(\log n)$
Heap binomial(1)	$O(1)$	$O(1)$	$O(\log n)$	$O(\log n)$	(update)	$O(\log n)$
Heap Fibonacci	$O(1)$	$O(1)$	$O(\log n)$	-	$O(1)$	$O(\log n)$
rp-heap	$O(1)$	$O(1)$	$O(\log n)$	-	$O(1)$	$O(\log n)$

Para contabilizar a destruição do um nó, aumentamos o custo de cada criação (insert, decrease-key) por 1.

Para contabilizar as operações link: define um potencial igual ao número de nós cheias, que não são filho de outro nó cheia (i.e. raízes e filhos de nós ociosos). Para todas operações diferente de delete-min e delete, o aumento do potencial é constante (no máximo 1 para insert, 3 para decrease-key, 0 para as demais). Para o delete que remove o elemento mínimo e delete-min, o custo amortizado de cada link é 0, porque um link combina duas raízes cheias, reduzindo o potencial por 1. Além disso, ao remover um elemento, o potencial aumenta por no máximo $\log_{\Phi} N$, um por cada filho do novo nó ocioso. Logo o custo amortizado de delete e delete-min é $O(\log N)$. ■

Re-otimizando o heap A análise acima é em função de N . Caso $\log N = O(\log n)$ temos um heap assintoticamente ótimo. Caso executamos muitas operações decrease-key, temos que reconstruir o heap periodicamente, para garantir $N = O(n)$. O método mais simples é: escolhe uma constante $c > 1$ e para $N > cn$ reconstrói o heap completamente, destruindo os nós ociosos, criando heaps de um único nó de todos nós cheios, e aplicando operações meld para unir todos heaps. O custo é $O(N)$ para percorrer todo nó uma vez e pode ser atribuído na análise amortizada para as operações insert e delete-min.

Resumo: Filas de prioridade A tabela 1.2 resume a complexidade das operações para diferentes implementações de uma fila de prioridade.

1.5.6. Árvores de van Emde Boas

Pela observação 1.5 é impossível implementar uma fila de prioridade baseada em comparação de chaves com todas as operações em $o(\log n)$. Porém existem algoritmos que ordenam n números em $o(n \log n)$, aproveitando o fato que as

1. Algoritmos em grafos

chaves são números com k bits, como por exemplo o radix sort que ordena em tempo $O(kn)$, ou aproveitando que as chaves possuem um domínio limitado, como por exemplo o counting sort que ordena n números em $[k]$ em tempo $O(n + k)$.

Uma *árvore de van Emde Boas* (árvore vEB) T realiza as operações

- $\text{member}(T, e)$: elemento e pertence a T ?
- $\text{insert}(T, e)$: insere e em T
- $\text{delete}(T, e)$: remove e de T
- $\text{min}(T)$ e $\text{max}(T)$: elemento mínimo e máximo de T , ou “undefined” caso não existe
- $\text{succ}(T, e)$ e $\text{pred}(T, e)$: successor e predecessor de e em T ; e não precisa pertencer a T

no universo de chaves $[0, u - 1]$ em tempo $O(\log \log u)$ e espaço $O(u)$.

Outras operações compostas podem ser implementadas, por exemplo

```
1  deletemin( $T$ ) :=  
2     $e := \text{min}(T)$ ; delete( $e$ ); return  $e$   
3  deletemax( $T$ ) :=  
4     $e := \text{max}(T)$ ; delete( $e$ ); return  $e$ 
```

Árvores binárias em ordem vEB Na discussão da implementação de árvores binárias na página 24 discutimos uma representação em ordem da busca em largura (BFS order). A ideia da ordem vEB é “cortar” a altura (número de níveis) h de uma árvore binária (que possui $n = 2^h - 1$ nodos e 2^{h-1} folhas) pela metade. Com isso obtemos

- uma árvore superior T_0 de altura $\lfloor h/2 \rfloor$
- e $b = 2^{\lfloor h/2 \rfloor} = \Theta(2^{h/2}) = \Theta(\sqrt{n})$ árvores inferiores $T_1, \dots, T_b$ de altura $\lceil h/2 \rceil$ e com $2^{\lceil h/2 \rceil} - 1 = \Theta(\sqrt{n})$ nodos.

Os nodos dessa árvore são armazenados em ordem $T_0, T_1, \dots, T_b$ e toda árvore T_i é ordenada recursivamente da mesma maneira, até chegar numa árvore de altura $h = 1$, como a Figura 1.9 mostra.

Armazenar uma árvore binária em ordem de vEB não altera a complexidade das operações. Uma busca, por exemplo, continua com complexidade $O(h)$. Porém, armazenado em ordem da busca por profundidade, uma busca pode gerar $\Theta(h)$ falhas no *cache*, no pior caso. Na ordem de vEB, a busca sempre atravessa $\Omega(\log_2 B)$ níveis, com B o tamanho de uma linha de cache,

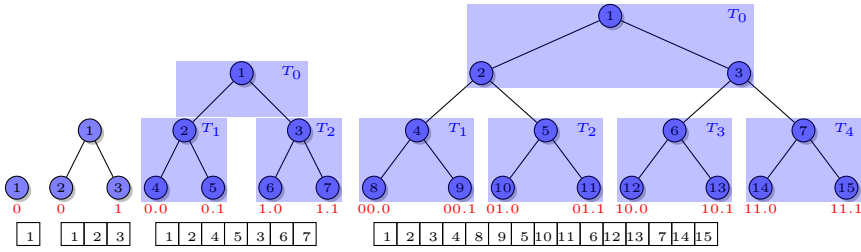


Figura 1.9.: Organização de árvores binárias em ordem de van Emde Boas para $h \in [4]$. As folhas são rotuladas por “cluster.subíndice”. Abaixo da árvore a ordem do armazenamento do vértices é dado. Os T_i correspondem com as subárvores do primeiro nível de recursão.

antes de gerar uma nova falha no *cache*. Logo uma busca gera somente $O(\log_2 n / \log_2 B) = O(\log_B n)$ falhas no *cache*. O layout se chama *cache oblivious* porque funciona sem conhecer o tamanho de uma linha de cache B .

Árvores vEB A estrutura básica de uma árvore de vEB é

1. Usar uma árvore binária de altura h para representar 2^{h-1} elementos nas folhas.
2. Cada folha armazena um bit, que é 1 caso o elemento correspondente pertença ao conjunto representado.
3. Os bits internos servem como *resumo* da sub-árvore: eles representam a conjunção dos bits dos filhos, i.e. um bit interno é um, caso na sua sub-árvore existe pelo menos uma folha que pertence ao conjunto representado.

Todas as operações da estrutura acima podem ser implementadas em tempo $O(h) = O(\log u)$. Para melhorar isso, vamos aplicar a mesma ideia da ordem de van Emde Boas: a árvore é separada em uma árvore superior, e uma série de árvores inferiores, cada uma com altura $\approx h/2$. As folhas da árvore superior contém o resumo das raízes das árvores inferiores: por isso a árvore superior possui altura $\lfloor h/2 \rfloor + 1$, uma a mais comparado com a ordem de vEB.

Fig. 1.10 mostra essa representação. A altura da árvore está armazenada no campo h . Além disso temos um ponteiro “top” para a árvore superior, e um vetor de ponteiros “bottom” de tamanho $b = 2^{\lfloor h/2 \rfloor}$ para as raízes das árvores inferiores. No caso base com $h = 2$, abusaremos os campos “top”

1. Algoritmos em grafos

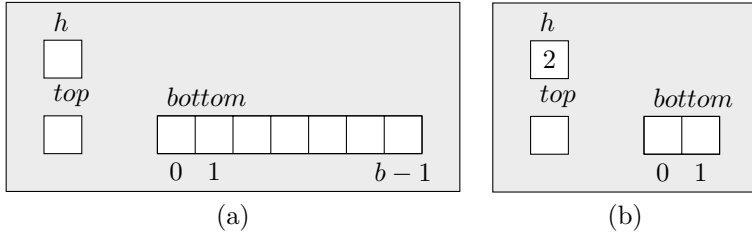


Figura 1.10.: Representação da primeira versão de uma árvore vEB. (a) Forma geral. (b) Caso base.

e “bottom” para armazenar os bits da raiz e dos dois filhos: um ponteiro arbitrário diferente de undefined representa um bit 1, o ponteiro undefined o bit 0. Para isso servem as funções auxiliares

```

1  set(p)  := p := 1
2  clear(p) := p := undefined
3  bit(p)   := return p ≠ undefined

```

Observe que as folhas $0, 1, \dots, 2^{h-1} - 1$ podem ser representadas com $h-1$ bits. Os primeiros $\lceil h/2 \rceil$ bits representam o número da sub-árvore que contém a folha, e os últimos $\lceil h/2 \rceil - 1$ bits o índice (relativo) da folha na sua sub-árvore. Isso explica a definição das funções auxiliares

```

1  subtree(e) := e ≫ ⌈h/2⌉ - 1
2  subindex(e) := e & (1 ≪ ⌈h/2⌉ - 1) - 1
3  element(s, i) := (s ≪ ⌈h/2⌉ - 1) | i

```

para extrair de um elemento o número da sub-árvore correspondente, ou o seu índice nesta sub-árvore, e para determinar o índice na árvore atual do i -ésimo elemento da sub-árvore s .

Com isso podemos implementar as operações como segue.

```

1  member(T, e) :=
2    if T.h = 2
3      return bit(T.bottom[e])
4    return member(T.bottom[subtree(e)], subindex(e))
5
6  min(T, e) :=
7    if T.h = 2
8      if bit(T.bottom[0])
9        return 0
10     if bit(T.bottom[1])

```

```

11         return 1
12     return undefined
13
14     c := min (T.top)
15     if c = undefined
16         return c
17     return element (c, min (T.bottom [c]))
18
19 succ (T, e) :=
20     if T.h = 2
21         if e = 0 and bit (T.bottom [1]) = 1
22             return 1
23         return 0
24
25     s := succ (T.bottom [subtree (e)], subindex (e))
26     if s ≠ undefined
27         return element (subtree (e), s)
28
29     c := succ (T.top, subtree (e))
30     if c = undefined
31         return c
32     return element (c, min (T.bottom [c]))
33
34 insert (T, e) :=
35     if T.h = 2
36         set (T.bottom [e])
37         set (T.top)
38     else
39         insert (T.bottom [subtree (e)], subindex (e))
40         insert (T.top, subtree (e))
41
42 delete (T, e) :=
43     if T.h = 2
44         clear (T.bottom [e])
45         if (bit (T.bottom [1 - e]) = 0
46             clear (T.top)
47     else
48         delete (T.bottom [subtree (e)], subindex (e))
49         s := min (T.bottom [subtree (e)])
50         if s = undefined

```

51 **delete** ($T.\text{top}, \text{subtree}(e)$)

As complexidades das operações implementadas no caso pessimista são (ver as chamadas recursivas acima em vermelho):

member $T(h) = T(\lceil h/2 \rceil) + O(1) = \Theta(\log h) = \Theta(\log \log u)$.

min $T(h) = T(\lfloor h/2 \rfloor + 1) + T(\lceil h/2 \rceil) + O(1) = 2T(h/2) + O(1) = \Theta(h) = \Theta(\log u)$.

insert $T(h) = T(\lceil h/2 \rceil) + T(\lfloor h/2 \rfloor + 1) + O(1) = \Theta(h) = \Theta(\log u)$.

succ/delete $T(h) = T(\lceil h/2 \rceil) + T(\lfloor h/2 \rfloor + 1) + O(h) = 2T(h/2) + O(h) = \Theta(h \log h) = \Theta(\log u \log \log u)$ (com um trabalho extra de $O(h)$ para chamar “min”).

Logo todas as operações com mais que uma chamada recursiva não possuem a complexidade desejada $O(\log \log u)$. A introdução de dois campos “min” e “max” que armazenam o elemento mínimo e máximo, junto com algumas modificações resolvem este problema.

1. Armazenar somente o mínimo, a operação “min” custa somente $O(1)$ e “insert”, “succ” e “delete” consequentemente somente $O(h)$.
2. Armazenado também o máximo, sabemos na operação “succ” se o sucessor está na árvore atual sem buscar, logo a operação “succ” pode ser implementada em $O(\log \log u)$.
3. A última modificação é *não armazenar* o elemento mínimo na sub-árvore correspondente. Com isso a primeira inserção somente modifica a árvore de resumo (top) e a segunda e as demais operações modificam somente a sub-árvore correspondente. A deleção funciona similarmente: ela remove ou um elemento na sub-árvore, ou o último elemento, modificando somente a árvore de resumo (top). Com isso todas operações podem ser implementadas em $O(\log \log u)$.

Na base armazenaremos os elementos somente nos campos “min” e “max”. Por convenção setamos “min” maior que “max” numa árvore vazia. As seguintes funções auxiliares permitem remover os elementos de uma árvore base e determinar se uma árvore possui nenhum, um ou mais elementos.

```

1 clear(T) :=
2   T.min:=1; T.max:=0; // convenção
3
4 empty(T) :=
```

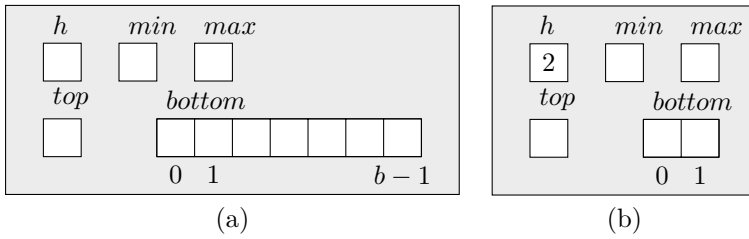


Figura 1.11.: Representação uma árvore vEB. (a) Forma geral. (b) Caso base.

```

5   return T.min>T.max
6
7   singleton(T) :=
8     return T.min=T.max
9
10  full(T) :=
11    return T.min<T.max

1  member(T,e) :=
2    if empty(T)
3      return false
4    if T.min = e or T.max = e
5      return true
6
7    { não é ``min'' nem ``max''? a base não contém o elemento }
8    if T.h = 2
9      return false
10
11    return member(T.bottom[subtree(e)],subindex(e))
12
13  min(T) :=
14    if empty(T)
15      return undefined
16    return T.min
17
18  max(T) :=
19    if empty(T)
20      return undefined
21    return T.max
22

```

1. Algoritmos em grafos

```
23 succ(T,e) :=
24   if T.h=2
25     if e = 0 and T.max = 1
26       return 1
27     return undefined
28
29   if not empty(T) and e < T.min
30     return T.min
31
32   { sucessor na árvore atual }
33   m:=max(T.bottom[subtree(e)])
34   if m ≠ undefined and subindex(e)<m
35     return element(subtree(e),
36                     succ(T.bottom[subtree(e)],subindex(e)))
37
38   { mínimo na árvore sucessora }
39   c:=succ(T.top,subtree(e))
40   if c = undefined
41     return c
42   return element(c,min(T.bottom[c]))
43
44 pred(T,e) :=
45   if T.h=2
46     if e = 1 and T.min=0
47       return 0
48     return undefined
49
50   if not empty(T) and T.max < e
51     return T.max
52
53   { predecessor na árvore atual }
54   m:=min(T.bottom[subtree(e)])
55   if m ≠ undefined and m < subindex(e)
56     return element(subtree(e),
57                     pred(T.bottom[subtree(e)],subindex(e)))
58
59   { máximo na árvore predecessora }
60   c:=pred(T.top,subtree(e))
61   if c = undefined
62     if not empty(T) and T.min<e
```



```

63         return T.min
64     else
65         return undefined
66
67     return element(c,max(T.bottom[c]))
68
69 insert(T,e) :=
70     if empty(T)
71         T.min := T.max := e
72         return
73
74     { novo mínimo: setar min, insere min anterior }
75     if e < T.min
76         swap(T.min,e)
77
78     { insere recursivamente }
79     if T.h > 2
80         if empty(T.bottom[subtree(e)])
81             insert(T.top,subtree(e))
82             insert(T.bottom[subtree(e)],subindex(e))
83
84     { novo máximo: atualiza }
85     if T.max < e
86         T.max := e
87
88 delete(T,e) :=
89     if empty(T)
90         return
91
92     if singleton(T)
93         if T.min = e
94             clear(T)
95         return
96
97     { novo mínimo? }
98     if e = T.min
99         T.min := element(min(T.top),min(T.bottom[min(T.top)]))
100         e := T.min
101
102     { remove e da árvore }

```

1. Algoritmos em grafos

```
103  delete (T.bottom[subtree(e)], subindex(e))
104
105  if empty(T.bottom[subtree(e)])
106      delete (T.top, subtree(e))
107      if e = T.max
108          c := max(T.top)
109          if c = undefined
110              T.max := T.min
111          else
112              T.max := element(c, max(T.bottom[c]))
113  else
114      T.max := element(subtree(e), max(T.bottom[subtree(e)]))
```

Com essas implementações cada função executa uma chamada recursiva e um trabalho constante a mais e logo precisa tempo $O(\log h)$. Em particular, na função “insert” caso a sub-árvore do elemento é vazia na linha 80 a segunda chamada “insert” na linha 82 precisa tempo constante. Similarmente, ou a deleção recursiva na linha 103 não remove o último elemento, e talvez custa $O(\log h)$, e logo a deleção da linha 106 não é executada, ou ela remove o último elemento e custa somente $O(1)$.

1.5.7. Exercícios

Exercício 1.1

Prove lema 1.4. Dica: Use indução sobre n .

Exercício 1.2

Prove que um heap binomial com n vértices possui $O(\log n)$ árvores. Dica: Por contradição.

Exercício 1.3 (Laboratório 1)

1. Implemente um heap binário. Escolha casos de teste adequados e verifique o desempenho experimentalmente.
2. Implemente o algoritmo de Prim usando o heap binário. Novamente verifique o desempenho experimentalmente.

Exercício 1.4 (Laboratório 2)

1. Implementa um heap binomial.
2. Verifica o desempenho dele experimentalmente.
3. Verifica o desempenho do algoritmo de Prim com um heap Fibonacci experimentalmente.

Exercício 1.5

A proposição 1.3 continua a ser correta para grafos com pesos negativos? Justifique.

1.6. Fluxos em redes

Seja $G = (V, A, c)$ um grafo direcionado e capacitado com capacidades $c : A \rightarrow \mathbb{R}$ nos arcos. Uma atribuição de fluxos aos arcos $f : A \rightarrow \mathbb{R}$ em G se chama *circulação*, se os fluxos respeitam os limites da capacidade ($f_a \leq c_a$) e satisfazem a conservação de fluxo $f(v) = 0$ com

$$f(v) := \sum_{a \in N^+(v)} f_a - \sum_{a \in N^-(v)} f_a \quad (1.7)$$

(ver Fig. 1.12).

Definição 1.2

Para $X, Y \subseteq V$ sejam $A(X, Y) := (X \times Y) \cap A$ os arcos passando de X para Y . O fluxo de X para Y é $f(X, Y) := \sum_{a \in A(X, Y)} f_a$. Ainda estendemos a notação do fluxo total de um vértice (1.7) para conjuntos: $f(X) := f(X, \bar{X}) - f(\bar{X}, X)$ é o fluxo neto saindo do conjunto X , onde $\bar{X} := V \setminus X$. Analogamente, escrevemos para as capacidades $c(X, Y) := \sum_{a \in A(X, Y)} c_a$.

Lema 1.12

Para qualquer conjunto de vértices $X \subseteq V$ temos $\sum_{v \in X} f(v) = f(X)$.

Prova.

$$\begin{aligned} \sum_{v \in X} f(v) &= \sum_{v \in X} \left(\sum_{a \in N^+(v)} f_a - \sum_{a \in N^-(v)} f_a \right) \\ &= \left(\sum_{a \in A(X, \bar{X})} f_a + \sum_{a \in A(X, X)} f_a \right) - \left(\sum_{a \in A(\bar{X}, X)} f_a + \sum_{a \in A(X, X)} f_a \right) \\ &= \sum_{a \in A(X, \bar{X})} f_a - \sum_{a \in A(\bar{X}, X)} f_a = f(X, \bar{X}) - f(\bar{X}, X) = f(X). \end{aligned}$$

■

Corolário 1.3

Qualquer atribuição de fluxos f satisfaz $\sum_{v \in V} f(v) = 0$.

Prova.

$$\sum_{v \in V} f(v) = f(V) = f(V, \bar{V}) - f(\bar{V}, V) = 0 - 0 = 0.$$

■

Uma circulação vira um *fluxo*, se o grafo possui alguns vértices que são fontes ou destinos (“sorvedouros”) de fluxo, e portanto não satisfazem a conservação de fluxo. Um fluxo s - t possui uma única fonte s e um único destino t . Um objetivo comum (transporte, etc.) é encontrar um fluxo s - t máximo.

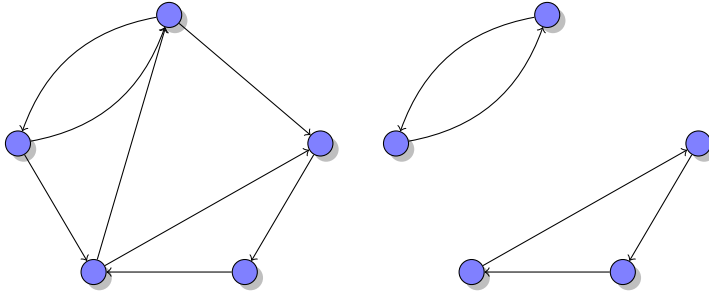


Figura 1.12.: Grafo (esquerda) com circulação (direita)

FLUXO s - t MÁXIMO

Instância Grafo direcionado $G = (V, A, c)$ com capacidades c nos arcos, um vértice origem $s \in V$ e um vértice destino $t \in V$.

Solução Um fluxo f , com $f(v) = 0, \forall v \in V \setminus \{s, t\}$.

Objetivo Maximizar o fluxo $f(s)$.

Lema 1.13

Um fluxo s - t satisfaz $f(s) + f(t) = 0$.

Prova. Temos

$$f(s) + f(t) = \sum_{v \in V} f(v) \stackrel{(1.3)}{=} 0,$$

onde a primeira igualdade vale pela conservação de fluxo nos vértices em $V \setminus \{s, t\}$. ■

Uma formulação como programa linear é

$$\begin{aligned} &\text{maximiza} && f(s) && (1.8) \\ &\text{sujeito a} && f(v) = 0, && \forall v \in V \setminus \{s, t\}, \\ &&& 0 \leq f_a \leq c_a, && \forall a \in A. \end{aligned}$$

Observação 1.12

O programa (1.8) possui uma solução, porque $f_a = 0$ é uma solução viável. O sistema não é ilimitado, porque todas variáveis são limitadas, e por isso possui

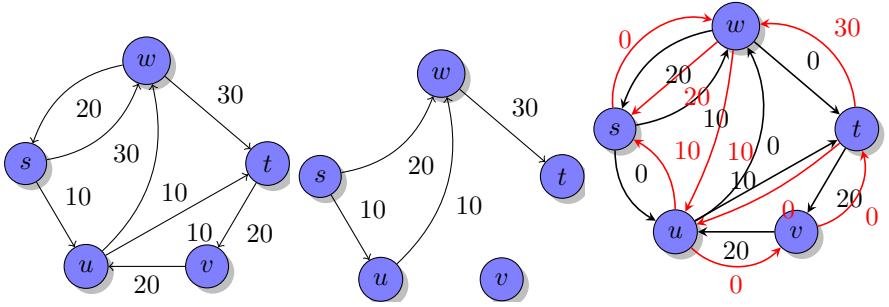


Figura 1.13.: Esquerda: Grafo com capacidades. Centro: Fluxo com valor 30. Direita: O grafo residual correspondente.

uma solução ótima. O problema de encontrar um fluxo s - t máximo pode ser resolvido em tempo polinomial via programação linear. $\diamond$

1.6.1. O algoritmo de Ford-Fulkerson

Nosso objetivo: Achar um algoritmo *combinatorial* mais eficiente. Ideia básica: Começar com um fluxo viável $f_a = 0$ e aumentar ele gradualmente. Observação: caso tenhamos um s - t -caminho $P = (v_0 = s, v_1, \dots, v_{n-1}, v_n = t)$, podemos aumentar o fluxo atual f um valor que corresponde ao “gargalo”

$$g(f, P) := \min_{\substack{a=(v_{i-1}, v_i) \\ i \in [n]}} c_a - f_a.$$

Observação 1.13

Repetidamente procurar um caminho de gargalo positivo e aumentar nem sempre produz um fluxo máximo. Na Fig. 1.13 o fluxo máximo possível é 40, obtido pelo aumentos de 10 no caminho $P_1 = (s, u, t)$ e 30 no caminho $P_2 = (s, w, t)$. Mas, se aumentamos 10 no caminho $P_1 = (s, u, w, t)$ e depois 20 no caminho $P_2 = (s, w, t)$ obtemos um fluxo de 30 e o grafo não possui mais caminho que aumenta o fluxo. $\diamond$

Problema no caso acima: para aumentar o fluxo e manter a conservação de fluxo num vértice interno v temos quatro possibilidades: (i) aumentar o fluxo num arco entrante e sainte, (ii) aumentar o fluxo num arco entrante, e diminuir num outro arco entrante, (iii) diminuir o fluxo num arco entrante e diminuir

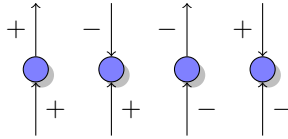


Figura 1.14.: Manter a conservação de fluxo.

num arco sainte e (iv) diminuir o fluxo num arco sainte e aumentar num outro arco sainte (ver Fig. 1.14).

Isso é o motivo para definir para um dado fluxo f o *grafo residual* G_f com

- Vértices V
- Arcos para frente (“forward”) A com capacidade $c_a - f_a$, caso $f_a < c_a$.
- Arcos para trás (“backward”) $A' = \{(v, u) \mid (u, v) \in A\}$ com capacidade $c_{(v,u)} = f_{(u,v)}$, caso $f_{(u,v)} > 0$.

Observe que na Fig. 1.13 o grafo residual possui um caminho $P = (s, w, u, t)$ que aumenta o fluxo por 10. O algoritmo de Ford-Fulkerson (Ford e Fulkerson, 1956) consiste em, repetidamente, aumentar o fluxo num caminho s - t no grafo residual.

Algoritmo 1.5 (Ford-Fulkerson)

Entrada Grafo $G = (V, A, c)$ com capacidades c_a nos arcos.

Saída Um fluxo f .

```

1  for all  $a \in A$ :  $f_a := 0$ 
2  while existe um caminho  $s$ - $t$  em  $G_f$  do
3    Seja  $P$  um caminho  $s$ - $t$  simples
4    Aumenta o fluxo  $f$  um valor  $g(f, P)$ 
5  end while
6  return  $f$ 
```

Análise de complexidade Na análise da complexidade, consideraremos somente capacidades em $\mathbb{N}$ (ou equivalente em $\mathbb{Q}$: todas capacidades podem ser multiplicadas pelo menor múltiplo em comum dos denominadores das capacidades.)

1. Algoritmos em grafos

Lema 1.14

Para capacidades inteiras, todo fluxo intermediário e as capacidades residuais são inteiros.

Prova. Por indução sobre o número de iterações. Inicialmente $f_a = 0$. Em cada iteração, o “gargalo” $g(f, P)$ é inteiro, porque as capacidades e fluxos são inteiros. Portanto, o fluxo e as capacidades residuais após o aumento são novamente inteiros. ■

Lema 1.15

Em cada iteração, o fluxo aumenta por pelo menos 1.

Prova. O caminho s - t possui por definição do grafo residual uma capacidade “gargalo” $g(f, P) > 0$. O fluxo $f(s)$ aumenta exatamente $g(f, P)$. ■

Lema 1.16

O algoritmo Ford-Fulkerson precisa no máximo $C = \sum_{a \in N^+(s)} c_a$ iterações. Portanto ele tem complexidade $O((n + m)C)$.

Prova. C é um limite superior do fluxo máximo. Como o fluxo inicialmente possui valor 0 e aumenta ao menos 1 por iteração, o algoritmo de Ford-Fulkerson termina em no máximo C iterações. Em cada iteração temos que achar um caminho s - t em G_f . Representando G por listas de adjacência, isso é possível em tempo $O(n + m)$ usando uma busca por profundidade. O aumento do fluxo precisa tempo $O(n)$ e a atualização do grafo residual é possível em $O(m)$, visitando todos arcos. ■

Corretude do algoritmo de Ford-Fulkerson

Definição 1.3

Uma partição $(X, \bar{X})$ de V é um *corte* s - t , se $s \in X$ e $t \in \bar{X}$. Um arco a é *saturado* para um fluxo f , caso $f_a = c_a$.

Lema 1.17

Para qualquer corte $(X, \bar{X})$ temos $f(X) = f(s)$.

Prova.

$$f(X) \stackrel{(1.12)}{=} f(s) + \sum_{v \in X \setminus \{s\}} f(v) = f(s).$$

(O último passo é correto, porque para todo $v \in X, v \neq s$, temos $f(v) = 0$ pela conservação de fluxo.) ■

Lema 1.18

O valor $c(X, \bar{X})$ de um corte s - t é um limite superior para um fluxo s - t .

Prova. Seja f um fluxo s - t . Temos

$$f(s) \stackrel{(1.17)}{=} f(X) = f(X, \bar{X}) - f(\bar{X}, X) \leq f(X, \bar{X}) \leq c(X, \bar{X}).$$

■

Consequência: O fluxo máximo é menor ou igual a o corte mínimo. De fato, a relação entre o fluxo máximo e o corte mínimo é mais forte:

Teorema 1.10 (Fluxo máximo – corte mínimo)

O valor do fluxo máximo entre dois vértices s e t é igual ao valor do corte mínimo.

Lema 1.19

Quando o algoritmo de Ford-Fulkerson termina, o valor do fluxo é máximo.

Prova. O algoritmo termina se não existe um caminho entre s e t em G_f . Podemos definir um corte $(X, \bar{X})$, tal que X é o conjunto de vértices alcançáveis em G_f a partir de s . Agora considere os arcos entre X e $\bar{X}$. Para um arco $a \in A(X, \bar{X})$ temos $f_a = c_a$, senão G_f terá um arco “forward” a , uma contradição. Para um arco $a = (u, v) \in A(\bar{X}, X)$ temos $f_a = 0$, senão G_f terá um arco “backward” $a' = (v, u)$, uma contradição. Logo

$$f(s) = f(X) = f(X, \bar{X}) - f(\bar{X}, X) = f(X, \bar{X}) = c(X, \bar{X}).$$

Pelo lema 1.18, o valor de um fluxo arbitrário é menor ou igual que $c(X, \bar{X})$, portanto f é um fluxo máximo. ■

Prova. (do teorema 1.10) Pela análise do algoritmo de Ford-Fulkerson. ■

Desvantagens do algoritmo de Ford-Fulkerson O algoritmo de Ford-Fulkerson tem duas desvantagens:

- (1) O número de iterações C pode ser alto, e existem grafos em que C iterações são necessárias (veja Fig. 1.15). Além disso, o algoritmo com complexidade $O((n + m)C)$ é somente pseudo-polinomial.
- (2) É possível que o algoritmo não termina para capacidades reais (veja Fig. 1.15). Usando uma busca por profundidade para achar caminhos s - t ele termina, mas é ineficiente (Dean et al., 2006).

1.6.2. O algoritmo de Edmonds-Karp

O algoritmo de Edmonds-Karp elimina esses problemas. O princípio dele é simples: Para achar um caminho s - t simples, usa busca por largura, i.e. seleccione o caminho mais curto entre s e t . Nós temos

1. Algoritmos em grafos

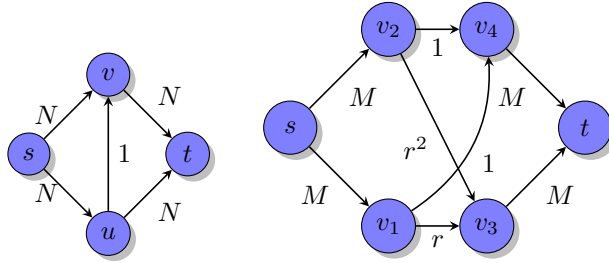


Figura 1.15.: Esquerda: Pior caso para o algoritmo de Ford-Fulkerson com pesos inteiros aumentando o fluxo por $2N$ vezes por 1 nos caminhos (s, u, v, t) e (s, v, u, t) . Direita: Menor grafo com pesos irracionais em que o algoritmo de Ford-Fulkerson falha (Zwick, 1995). $M \geq 3$, e $r = (1 + \sqrt{1 - 4\lambda})/2 \approx 0.682$ com $\lambda \approx 0.217$ a única raiz real de $1 - 5x + 2x^2 - x^3$. Aumentar (s, v_1, v_4, t) e depois repetidamente $(s, v_2, v_4, v_1, v_3, t)$, $(s, v_2, v_3, v_1, v_4, t)$, $(s, v_1, v_3, v_2, v_4, t)$, e $(s, v_1, v_4, v_2, v_3, t)$ converge para o fluxo máximo $2 + r + r^2$ sem terminar.

Teorema 1.11

O algoritmo de Edmonds-Karp precisa $O(nm)$ iterações, e portanto termina em tempo $O(nm^2)$.

Lema 1.20

Seja $\delta_f(v)$ a distância entre s e v em G_f . Durante a execução do algoritmo de Edmonds-Karp $\delta_f(v)$ cresce monotonicamente para todos vértices em V .

Prova. Para $v = s$ o lema é evidente. Suponha que uma iteração modificando o fluxo f para f' diminuirá o valor de um vértice $v \in V \setminus \{s\}$, i.e., $\delta_f(v) > \delta_{f'}(v)$ (o). Seja v o vértice de menor distância $\delta_{f'}(v)$ em $G_{f'}$ com essa característica, e $P = (s, \dots, u, v)$ um caminho mais curto de s para v em $G_{f'}$. Logo $\delta_{f'}(u) + 1 = \delta_{f'}(v)$ (Δ). Pela escolha de v , o valor de u não diminuiu nessa iteração, i.e., $\delta_f(u) \leq \delta_{f'}(u)$ (*).

Supondo $uv \in A(G_f)$, temos

$$\delta_f(v) \leq \delta_f(u) + 1 \stackrel{(*)}{\leq} \delta_{f'}(u) + 1 \stackrel{(\Delta)}{=} \delta_{f'}(v),$$

uma contradição com a hipótese (o) que a distância de v diminuiu. Logo o arco uv não existe in G_f , mas $uv \in A(G_{f'})$. Isso só é possível se o fluxo de v para u aumentou nessa iteração. Em particular, vu era parte de um caminho

mínimo de s para u e logo $\delta_f(v) + 1 = \delta_f(u)$ ($\dagger$). Para $v = t$ isso é uma contradição imediata. Caso $v \neq t$, temos

$$\delta_f(v) \stackrel{(\dagger)}{=} \delta_f(u) - 1 \stackrel{(*)}{\leq} \delta_{f'}(u) - 1 \stackrel{(\Delta)}{=} \delta_{f'}(v) - 2,$$

novamente uma contradição com a hipótese ($\circ$) que a distância de v diminuiu. Logo, o vértice v não existe. ■

Prova. (do teorema 1.11)

Chama um arco num caminho que aumenta o fluxo com capacidade igual ao gargalo *crítico*. Em cada iteração existe ao menos um arco crítico que desaparece do grafo residual. Provaremos que cada arco pode ser crítico no máximo $n/2 - 1$ vezes, e logo não temos mais que $m(n/2 - 1) = O(mn)$ iterações.

No grafo G_f em que um arco $uv \in A$ é crítico pela primeira vez temos $\delta_f(u) = \delta_f(v) - 1$. O arco só aparece novamente no grafo residual caso alguma iteração posterior diminui o fluxo em uv , i.e., aumenta o fluxo vu . Nessa iteração, com fluxo f' , $\delta_{f'}(v) = \delta_{f'}(u) - 1$. Juntamente com o fato de que a distância só aumenta (lema 1.20) obtemos

$$\delta_{f'}(u) = \delta_{f'}(v) + 1 \stackrel{\text{Lema 1.20}}{\geq} \delta_f(v) + 1 = \delta_f(u) + 2,$$

i.e., a distância do u entre dois instantes em que uv é crítico aumenta por pelo menos dois. Enquanto u é alcançável por s , a sua distância é no máximo $n - 2$, porque o caminho não contém s nem t , e por isso a aresta uv pode ser crítica por no máximo $(n - 2)/2 = n/2 - 1$ vezes. ■

Zadeh (1972) apresenta instâncias em que o algoritmo de Edmonds-Karp precisa $\Theta(n^3)$ iterações, logo o resultado do teorema 1.11 é o melhor possível para grafos densos.

1.6.3. O algoritmo “caminho mais gordo” (“fattest path”)

Idéia (Edmonds e Karp, 1972): usar o caminho de maior gargalo para aumentar o fluxo. (Exercício 1.6 pede provar que isso é possível com uma modificação do algoritmo de Dijkstra em tempo $O(n \log n + m)$.)

Teorema 1.12

O caminho de maior gargalo aumenta o fluxo atual f de valor v por pelo menos OPT/m , onde OPT é o fluxo máximo no grafo residual G_f .

Prova. Considere um arco crítico a no caminho de maior gargalo, com capacidade c_a no grafo residual G_f . Particiona $V = S \dot{\cup} T$, onde S contém s e

1. Algoritmos em grafos

todos vértices alcançáveis por arcos de capacidade maior que c_a . Por construção T contém pelo menos t . O corte (S, T) tem capacidade no máximo mc_a , logo pelo teorema 1.10 $v \leq OPT \leq mc_a$. Por isso o fluxo aumenta por pelo menos $c_a \geq OPT/m$. ■

Teorema 1.13

A complexidade do algoritmo de Ford-Fulkerson usando o caminho de maior gargalo é $O((n \log n + m)m \log C)$ para um limitante superior C do fluxo máximo.

Prova. Seja f_i o valor do caminho encontrado na i -ésima iteração, G_i o grafo residual após o aumento e OPT_i o fluxo máximo em G_i . Observe que G_0 é o grafo de entrada e $OPT_0 = OPT$ o fluxo máximo. Temos

$$OPT_{i+1} = OPT_i - f_i \leq OPT_i - OPT_i/(2m) = (1 - 1/(2m))OPT_i.$$

A desigualdade é válida pelo teorema 1.12, considerando que o grafo residual possui no máximo $2m$ arcos. Logo

$$OPT_i \leq (1 - 1/(2m))^i OPT \leq e^{-i/(2m)} OPT.$$

O algoritmo termina caso $OPT_i < 1$, por isso número de iterações é no máximo $2m \ln OPT + 1$. Cada iteração custa $O(m + n \log n)$. ■

Corolário 1.4

Caso U seja um limite superior da capacidade de um arco, o algoritmo termina em no máximo $O(m \log mU)$ passos.

1.6.4. O algoritmo push-relabel

O algoritmo push-relabel representa uma classe de algoritmos que não trabalha com um fluxo e caminhos aumentantes, mas mantém um *pré-fluxo* f que satisfaz

- os limites de capacidade ($0 \leq f_a \leq c_a$)
- e requer somente que o *excesso* $e(v) = -f(v)$ de um vértice $v \neq s$ é não-negativo.

Um vértice $v \neq t$ com $e(v) > 0$ é chamado *ativo*. A ideia do algoritmo é que vértices possuem uma “altura” e o fluxo passa para vértices de altura mais baixa (“operação push”) ou, caso isso não seja possível a altura de um

vértice ativo aumenta (“operação relabel”). Concretamente, manteremos um *potencial* (“altura”) p_v para cada $v \in V$, tal que,

$$\begin{aligned} p_s &= n; & p_t &= 0; \\ p_v &\geq p_u - 1 & (u, v) &\in A(G_f). \end{aligned} \quad (*)$$

Note que a segunda parte da condição tem que ser satisfeita somente para arcos no grafo residual.

Observação 1.14

Pela condição (*), para um caminho $v_0, v_1, \dots, v_k$ em G_f temos $p_{v_0} \leq p_{v_1} + 1 \leq p_{v_2} + 2 \leq \dots \leq p_{v_k} + k$. $\diamond$

Lema 1.21

Condição (*) pode ser satisfeita sse G_f não possui caminho s – t .

Prova. “ $\Rightarrow$ ”: Suponha que existe um caminho s – t simples $v_0 = s, v_1, \dots, v_k = t$. Pela observação (1.14)

$$p_s = p_{v_0} \leq p_{v_k} + k = p_t + k = k < n,$$

uma contradição. “ $\Leftarrow$ ”: Sejam X os vértices alcançáveis em G_f a partir de s (incluindo s). Como G_f não possui caminho s – t , $t \in \overline{X}$. Logo setando $p_v = n$ para $v \in X$ e $p_v = 0$ para $v \in \overline{X}$ satisfaz (*). $\blacksquare$

O lema mostra que enquanto algoritmos de caminho aumentante são algoritmos primais, mantendo uma solução factível, até encontrar o ótimo, algoritmos da classe push-relabel podem ser vistos como algoritmos duais: eles mantêm o critério de otimalidade (*), até encontrar uma solução factível.

Podemos realizar as operações “push” e “relabel” como segue. A operação “push(u, v)” num arco $(u, v) \in A(G_f)$ manda o fluxo $\min\{c_a, e(u)\}$ de u para v . A operação “relabel(v)” aumenta a altura p_v do vértice v por uma unidade.

```

1 push( $u, v$ ) :=
2   { pré-condição:  $u$  é ativo }
3   { pré-condição:  $p_v = p_u - 1$  }
4   { pré-condição:  $(u, v) \in A(G_f)$  }
5   aumenta o fluxo em  $(u, v)$  por  $\min\{c_{(u,v)}, e(u)\}$ 
6   { atualiza  $G_f$  de acordo }
7 end
8
9 relabel( $v$ ) :=
10  { pré-condição:  $v$  é ativo }
11  { pré-condição: não existe  $(u, v) \in A(G_f)$  com  $p_v = p_u - 1$  }
12   $p_v := p_v + 1$ 
13 end
```

1. Algoritmos em grafos

Observe que as duas operações mantêm a condição (*).

Algoritmo 1.6 (Push-relabel)

Entrada Grafo $G = (V, A, c)$ com capacidades c_a no arcos.

Saída Um fluxo f .

```
1   $p_s := n; p_v := 0, \quad \forall v \in V \setminus \{s\}$ 
2   $f_a := c_a, \quad \forall a \in N^+(s)$  senão  $f_a := 0$ 
3  while existe vértice ativo do
4      escolhe o vértice ativo  $u$  de maior  $p_u$ 
5      repete até  $u$  é inativo
6          if existe arco  $(u, v) \in G_f$  com  $p_v = p_u - 1$  then
7               $\text{push}(u, v)$ 
8          else
9               $\text{relabel}(u)$ 
10         end if
11     end
12 end while
13 return  $f$ 
```

Lema 1.22

O algoritmo push-relabel é parcialmente correto (i.e. correto caso termina).

Prova. Ao terminar não existe vértice ativo. Logo f é um fluxo. Pelo lema 1.21 não existe caminho s – t em G_f . Logo pelo teorema 1.10 o fluxo é ótimo. ■

A terminação é garantida por

Teorema 1.14

O algoritmo push-relabel executa $O(n^3)$ operações push e $O(n^2)$ operações relabel.

Prova. Um vértice ativo v tem excesso de fluxo, logo existe um caminho v – s em G_f . Por (1.14) $p_v \leq p_s + (n - 1) < 2n$, e logo o número de operações relabel é $O(n^2)$. Suponha que uma operação push satura um arco $a = (u, v)$ (i.e. manda fluxo c_a). Para mandar fluxo novamente, temos que mandar primeiramente fluxo de v para u ; isso só pode ser feito depois de pelo menos duas operações relabel em v . Logo o número de operações push saturantes é $O(mn)$. Para operações push não-saturantes, podemos observar que entre duas operações relabel temos no máximo n dessas operações, porque cada uma torna o vértice de maior p_v inativo (talvez ativando vértices de menor potencial), logo tem no máximo $O(n^3)$ deles. ■

Tabela 1.3.: Complexidade de diversos algoritmos de fluxo máximo (partes de Schrijver, 2003).

Ano	Referência	Complexidade	Obs
1951	Dantzig	$O(n^2 mC)$	Simplex
1955	Ford & Fulkerson	$O(mC) = O(mnU)$	Cam. aument.
1970	Dinitz	$O(nm^2)$	Cam. min. aument.
1972	Edmonds & Karp	$O(m^2 \log C)$	Escalonamento
1973	Dinitz	$O(nm \log C)$	Escalonamento
1974	Karzanov	$O(n^3)$	Preflow-Push
1977	Cherkassky	$O(n^2 m^{1/2})$	Preflow-Push
1986	Goldberg & Tarjan	$O(nm \log(n^2/m))$	Push-Relabel
1987	Ahuja & Orlin	$O(nm + n^2 \log C)$	Push-Relabel & Esc.
1990	Cheriyian et al.	$O(n^3 / \log n)$	
1990	Alon	$O(nm + n^{8/3} \log n)$	
1992	King et al.	$O(nm + n^{2+\epsilon})$	
1997	Goldberg & Rao	$O(m^{3/2} \log(n^2/m) \log C)$ $O(n^{2/3} m \log(n^2/m) \log C)$	
2012	Orlin	$O(nm)$	
2022	Chen et al.	$O(m^{1+o(1)})$	Pontos interiores

Para garantir uma complexidade de $O(n^3)$ temos que implementar um “push” em $O(1)$ e um “relabel” em $O(n)$. Para este fim, manteremos uma lista dos vértices em ordem do potencial. Para cada vértice manteremos uma lista de arcos candidatos para operações push, i.e. arcos para vizinhos com potencial um a menos com capacidade residual positiva.

Uma busca linear na lista de vértices encontra o vértice de maior potencial. Entre duas operações relabel a busca pode continuar no último ponto e precisa tempo $O(n)$ em total, logo a busca custa no máximo $O(n^3)$ sobre toda execução do algoritmo. Para a operação push podemos simplesmente consultar a lista de candidatos. Para um push saturante, o candidato será removido. Isso custa $O(1)$. Finalmente no caso de um relabel temos que encontrar em $O(n)$ a nova posição do vértice na lista, e reconstruir a lista de candidatos, que também precisa tempo $O(n)$. Logo todas operações relabel custam não mais que $O(n^3)$.

1.6.5. Variantes do problema

Fontes e destinos múltiplos Para $G = (V, A, c)$ define um conjunto de fontes $S \subseteq V$ e um conjunto de destinos $T \subseteq V$, com $S \cap T = \emptyset$, e considera

$$\begin{aligned} &\text{maximiza} && f(S) \\ &\text{sujeito a} && f(v) = 0, && \forall v \in V \setminus (S \cup T), \\ &&& f_a \leq c_a, && \forall a \in A. \end{aligned} \quad (1.9)$$

O problema (1.9) pode ser reduzido para um problema de fluxo máximo simples em $G' = (V', A', c')$ (veja Fig. 1.16(a)) com

$$\begin{aligned} V' &= V \cup \{s^*, t^*\} \\ A' &= A \cup \{s^*\} \times S \cup T \times \{t^*\} \\ c'_a &= \begin{cases} c_a, & a \in A, \\ c(S, \bar{S}), & a = (s^*, s), s \in S, \\ c(\bar{T}, T), & a = (t, t^*), t \in T. \end{cases} \end{aligned} \quad (1.10)$$

Lema 1.23

Se f' é uma solução máxima de (1.10), a restrição $f = f'|_A$ é uma solução máxima de (1.9) de mesmo valor. Por outro lado, se f é uma solução máxima de (1.9), a extensão

$$f'_a = \begin{cases} f_a, & a \in A, \\ f(s), & a = (s^*, s), s \in S, \\ -f(t), & a = (t, t^*), t \in T, \end{cases} \quad (1.11)$$

é uma solução máxima de (1.10) de mesmo valor.

Prova. Se f' é solução de (1.10), a restrição $f = f'|_A$ é uma solução de (1.9) de mesmo valor: f é viável porque $f(v) = f'(v) = 0$ para todo $v \in V \setminus S \setminus T$ e $f(S) = \sum_{s \in S} f(s) = f'(s^*)$. Similarmente, dado um fluxo válido f em G , a extensão f' (1.11) é um fluxo válido em G' de mesmo valor: f' é viável porque além de $f'(v) = f(v) = 0$ para todo $v \in V \setminus (S \cup T)$, também $f'(v) = 0$ para $v \in S \cup T$, e $f'(s^*) = \sum_{s \in S} f(s) = f(S)$. ■

Limites inferiores Para $G = (V, A, b, c)$ com limites inferiores $b : A \rightarrow \mathbb{R}$ considere o problema

$$\begin{aligned} &\text{maximiza} && f(s) \\ &\text{sujeito a} && f(v) = 0, && \forall v \in V \setminus \{s, t\}, \\ &&& b_a \leq f_a \leq c_a, && a \in A. \end{aligned} \quad (1.12)$$

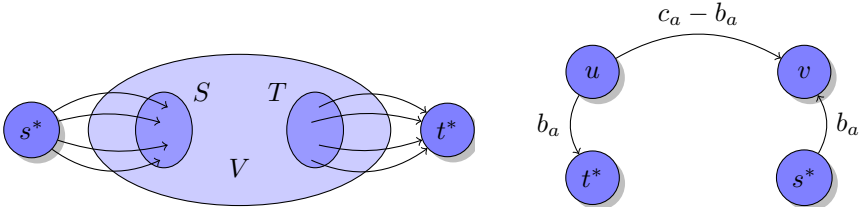


Figura 1.16.: Reduções entre variantes do problema do fluxo máximo. Esquerda: Fontes e destinos múltiplos. Direita: Limite inferior e superior para a capacidade de arcos.

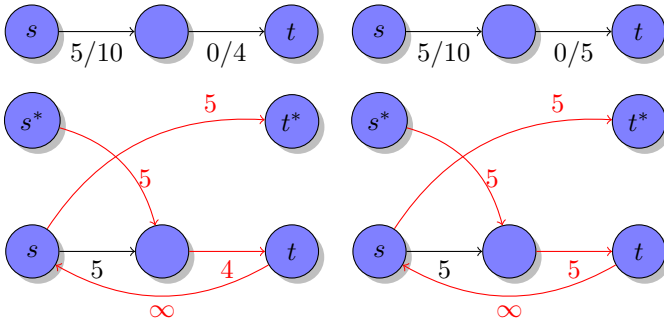


Figura 1.17.: Dois exemplos da transformação do lema 1.24. Esquerda: Grafo sem solução viável e grafo transformado com fluxo máximo 4. Direita: Grafo com solução viável e grafo transformado com fluxo máximo 5.

1. Algoritmos em grafos

O problema (1.12) pode ser reduzido para um problema de fluxo máximo simples em $G' = (V', A', c')$ (veja Fig. 1.16(b)) com

$$\begin{aligned} V' &= V \cup \{s^*, t^*\} \\ A' &= A \cup \{(u, t^*) \mid (u, v) \in A\} \cup \{(s^*, v) \mid (u, v) \in A\} \cup \{(t, s)\} \\ c'_a &= \begin{cases} c_a - b_a, & a \in A, \\ \sum_{v \in N^+(u)} b_{(u,v)}, & a = (u, t^*), \\ \sum_{u \in N^-(v)} b_{(u,v)}, & a = (s^*, v), \\ \infty, & a = (t, s). \end{cases} \end{aligned} \quad (1.13)$$

Chama um fluxo em 1.13 *saturado*, caso ele satura todos arcos auxiliares $\{(u, t^*) \mid (u, v) \in A\} \cup \{(s^*, v) \mid (u, v) \in A\}$.

Lema 1.24

Problema (1.12) possui um fluxo viável sse (1.13) possui um fluxo saturado.

Prova. Caso f é um fluxo viável em (1.12),

$$f'_a = \begin{cases} f_a - b_a, & a \in A, \\ \sum_{u \in N^+(v)} b_{(v,u)}, & a = (v, t^*), \\ \sum_{u \in N^-(v)} b_{(u,v)}, & a = (s^*, u), \\ f(s), & a = (t, s). \end{cases}$$

é um fluxo saturado de (1.13). Por outro lado, se f' é um fluxo saturado para (1.13), $f_a = f'_a + b_a$ é um fluxo viável em (1.12). ■

Como um fluxo saturado tem que ser máximo, ele pode ser obtido por um algoritmo de fluxo máximo aplicado a (1.13). Caso o fluxo máximo não satura, não tem solução viável, senão podemos extrair uma solução viável de (1.12) pela construção acima. Para obter um fluxo máximo de (1.12) podemos maximizar o fluxo a partir da solução viável obtida, com qualquer variante do algoritmo de Ford-Fulkerson. Na execução temos que garantir que um fluxo mínimo de b_a é mantido em cada arco $a = (u, v)$. Logo, o grafo residual de um fluxo f tem arcos “backward” $\bar{a} = (v, u)$ de capacidade reduzida $c_{\bar{a}} = f_a - b_a$.

Uma alternativa para obter um fluxo factível com limites inferiores nos arcos é primeiro mandar o limite inferior de cada arco, i.e. setar $f = b$, e depois considerar demandas $d_v = -f(v)$. Uma circulação factível com limites $0 \leq f \leq c - b$ corresponde com um fluxo factível $f + b$ no grafo original.

Existência de uma circulação com demandas nos vértices Para $G = (V, A, c)$ com demandas d_v , com $d_v > 0$ para destinos e $d_v < 0$ para fontes, considere

$$\begin{array}{ll} \text{existe} & f \\ \text{s.a} & f(v) = -d_v, \quad \forall v \in V, \\ & f_a \leq c_a, \quad a \in A. \end{array} \quad (1.14)$$

Evidentemente $\sum_{v \in V} d_v = 0$ é uma condição necessária (lema 1.3). O problema (1.14) pode ser reduzido para um problema de fluxo máximo em $G' = (V', A')$ com

$$\begin{aligned} V' &= V \cup \{s^*, t^*\} \\ A' &= A \cup \{(s^*, v) \mid v \in V, d_v < 0\} \cup \{(v, t^*) \mid v \in V, d_v > 0\} \\ c_a &= \begin{cases} c_a, & a \in A, \\ -d_v, & a = (s^*, v), \\ d_v, & a = (v, t^*). \end{cases} \end{aligned} \quad (1.15)$$

Lema 1.25

Problema (1.14) possui uma solução sse problema (1.15) possui uma solução com fluxo máximo $D = \sum_{v: d_v > 0} d_v$.

Prova. (Exercício.) ■

Circulações com limites inferiores Para $G = (V, A, b, c)$ com limites inferiores e superiores, considere

$$\begin{array}{ll} \text{existe} & f \\ \text{s.a} & f(v) = d_v, \quad \forall v \in V, \\ & b_a \leq f_a \leq c_a, \quad a \in A. \end{array} \quad (1.16)$$

O problema pode ser reduzido para a existência de uma circulação com somente limites superiores em $G' = (V', A', c', d')$ com

$$\begin{aligned} V' &= V \\ A' &= A \end{aligned} \quad (1.17)$$

$$\begin{aligned} c_a &= c_a - b_a \\ d'_v &= d_v - \sum_{a \in N^-(v)} b_a + \sum_{a \in N^+(v)} b_a \end{aligned} \quad (1.18)$$

Lema 1.26

O problema (1.16) possui solução sse problema (1.17) possui solução.

Prova. (Exercício.) ■

1.6.6. Aplicações

Projeto de pesquisa de opinião O objetivo é projetar uma pesquisa de opinião, com as restrições

- Cada cliente i recebe ao menos c_i perguntas (para obter informação suficiente) mas no máximo c'_i perguntas (para não cansá-lo). As perguntas podem ser feitas somente sobre produtos que o cliente já comprou.
- Para obter informações suficientes sobre um produto, entre p_i e p'_i clientes tem que ser interrogados sobre ele.

Um modelo é um grafo bi-partido entre clientes e produtos, com aresta (c_i, p_j) caso cliente i já comprou produto j . O fluxo de cada aresta possui limite inferior 0 e limite superior 1. Para representar os limites de perguntas por produto e por cliente, introduziremos ainda dois vértices s , e t , com arestas (s, c_i) com fluxo entre c_i e c'_i e arestas (p_j, t) com fluxo entre p_j e p'_j e uma aresta (t, s) .

Segmentação de imagens O objetivo é segmentar um imagem em duas partes, por exemplo “foreground” e “background”. Supondo que temos uma “probabilidade” a_i de pertencer ao “foreground” e outra “probabilidade” de pertencer ao “background” b_i para cada pixel i , uma abordagem direta é definir que pixels com $a_i > b_i$ são “foreground” e os outros “background”. Um exemplo pode ser visto na Fig. 1.19 (b). A desvantagem dessa abordagem é que a separação ignora o contexto de um pixel. Um pixel, “foreground” com todos os pixels adjacentes em “background” provavelmente pertence ao “background” também. Portanto obtemos um modelo melhor introduzindo penalidades p_{ij} para separar (atribuir à categorias diferentes) pixels adjacentes i e j . Um partição do conjunto de todos pixels I em $A \cup B$ tem um valor de

$$q(A, B) = \sum_{i \in A} a_i + \sum_{i \in B} b_i - \sum_{(i,j) \in A \times B} p_{ij}$$

nesse modelo, e o nosso objetivo é achar uma partição que maximiza $q(A, B)$. Isso é equivalente a minimizar

$$\begin{aligned} Q(A, B) &= \sum_{i \in I} a_i + b_i - \sum_{i \in A} a_i - \sum_{i \in B} b_i + \sum_{(i,j) \in A \times B} p_{ij} \\ &= \sum_{i \in B} a_i + \sum_{i \in A} b_i + \sum_{(i,j) \in A \times B} p_{ij}. \end{aligned}$$

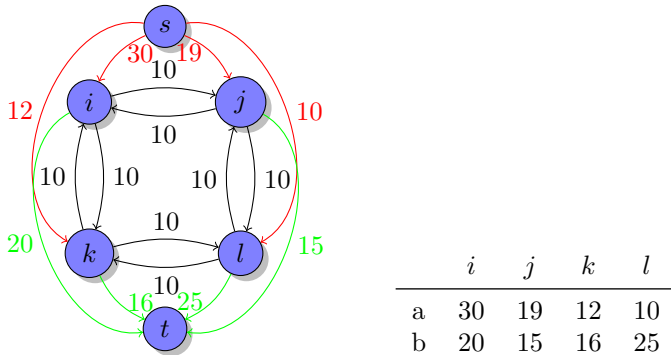


Figura 1.18.: Exemplo da construção para uma imagem 2×2 . Esquerda: Grafo com penalidade fixa $p_{ij} = 10$. Direita: Tabela com valores pele/não-pele.

A solução mínima de $Q(A, B)$ pode ser visto como corte mínimo num grafo. O grafo possui um vértice para cada pixel e uma aresta com capacidade p_{ij} entre dois pixels adjacentes i e j . Ele possui ainda dois vértices adicionais s e t , arestas (s, i) com capacidade a_i para cada pixel i e arestas (i, t) com capacidade b_i para cada pixel i (ver Fig. 1.18).

Sequenciamento O objetivo é programar um transporte com um número k de veículos disponíveis, dados pares de origem-destino com tempo de saída e chegada. Um exemplo é um conjunto de vôos é

1. Porto Alegre (POA), 6.00 – Florianópolis (FLN), 7.00
2. Florianópolis (FLN), 8.00 – Rio de Janeiro (GIG), 9.00
3. Fortaleza (FOR), 7.00 – João Pessoa (JPA), 8.00
4. São Paulo (GRU), 11.00 – Manaus (MAO), 14.00
5. Manaus (MAO), 14.15 – Belem (BEL), 15.15
6. Salvador (SSA), 17.00 – Recife (REC), 18.00

O mesmo avião pode ser usado para mais que um par de origem e destino, se o destino do primeiro é a origem do segundo, em tem tempo suficiente entre a chegada e saída (para manutenção, limpeza, etc.) ou tem tempo suficiente para deslocar o avião do destino para o origem.

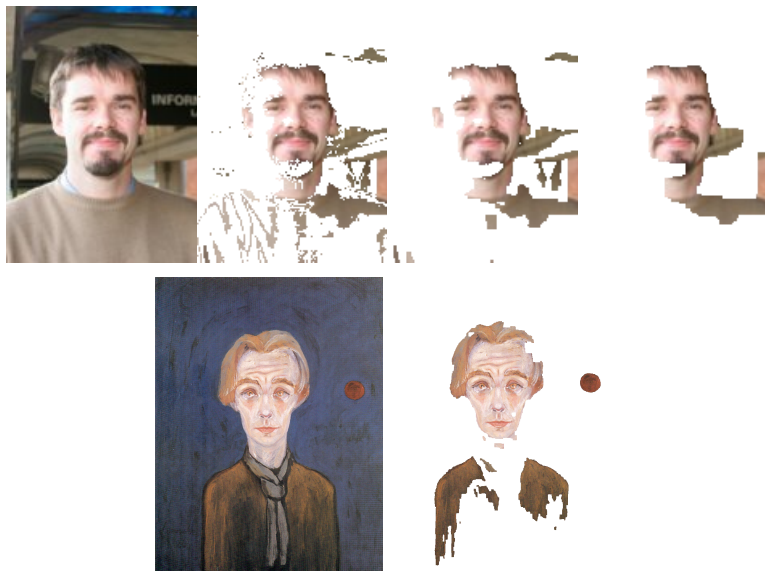


Figura 1.19.: Segmentação de imagens com diferentes penalidades p . Acima: (a) Imagem original (b) Segmentação somente com probabilidades ($p = 0$) (c) $p = 1000$ (d) $p = 10000$. Abaixo: (a) Walter Gramatté, Selbstbildnis mit rotem Mond, 1926 (b) Segmentação com $p = 10000$. A probabilidade de um pixel representar pele foi determinado conforme Jones e Rehg (1998).

Podemos representar o problema como grafo direcionado acíclico. Dado pares de origem destino, ainda adicionamos pares de destino-origem que são compatíveis com as regras acima. A ideia é representar aviões como fluxo: cada aresta origem-destino é obrigatório, e portanto recebe limites inferiores e superiores de 1, enquanto uma aresta destino-origem é facultativa e recebe limite inferior de 0 e superior de 1. Além disso, introduzimos dois vértices s e t , com arcos facultativos de s para qualquer origem e de qualquer destino para t , que representam os começos e finais da viagem completa de um avião. Para decidir se existe um solução com k aviões, finalmente colocamos um arco (t, s) com limite inferior de 0 e superior de k e decidir se existe uma circulação nesse grafo.

O problema $P \mid \text{pmtn}, r_i \mid L_{\max}$ Primeiramente resolveremos um problema mais simples: será que existe um sequenciamento tal que toda tarefa i executa dentro do seu intervalo $[r_i, d_i]$? Equivalentemente, será que existe uma solução com $L_{\max} = 0$?

Seja $\{t_1, t_2, \dots, t_k\} = \{r_1, r_2, \dots, r_n\} \cup \{d_1, d_2, \dots, d_n\}$, com $t_1 \leq t_2 \leq \dots \leq t_k$. (Observe que $k \leq 2n$, e $k < 2n$ no caso de tempos repetidos.) Podemos ver os t_i como eventos em que uma tarefa fica disponível ou tem que terminar o seu processamento. Os t_i definem $k-1$ intervalos $I_i = [t_i, t_{i+1}]$ para $i \in [k-1]$ com duração $S_i = t_{i+1} - t_i$ correspondente. Cada tarefa j pode ser executada no intervalo T_i caso $I_i \subseteq [r_j, d_j]$. Logo podemos modelar o problema via um grafo direcionado bipartido com vértices $T \dot{\cup} I$, sendo $T = [n]$ o conjunto de tarefas e $I = \{I_i \mid i \in [k-1]\}$ o conjunto de intervalos, e com arcos (j, i) caso tarefa j pode ser executada no intervalo i . Para completar o grafo adicionaremos um arco (s, j) de um vértice origem s para cada tarefa j , e um arco (i, t) de cada intervalo para um vértice destino t . Um fluxo nesse grafo representa tempo, e teremos capacidades p_j entre s e tarefa j , S_i entre tarefa j e intervalo i , e mS_i entre T_i e t , sendo mS_i o tempo total disponível durante o intervalo i . Figura 1.20 mostra a construção completa.

Logo $P \mid \text{pmtn}, r_i \mid L_{\max}$ pode ser resolvido em tempo $O(mn \log \bar{L})$.

Com essa abordagem podemos resolver o problema original por busca binária: para cada valor de $L_{\max}$ entre 0 e $\bar{L}$ testaremos se existe uma solução tal que cada tarefa executa no intervalo $[r_i, d_i + L_{\max}]$. Um limite superior simples é $\bar{L} = \max_i r_i + \sum_i p_i - \min_i d_i$ executando todas tarefas após a liberação da última numa única máquina em ordem arbitrária.

Agendamento de projetos Suponha que temos n projetos, cada um com lucro $p_i \in \mathbb{Z}$, $i \in [n]$, e um grafo de dependências $G = ([n], A)$ sobre os projetos. Caso $(i, j) \in A$, a execução do projeto i é pré-requisito para a

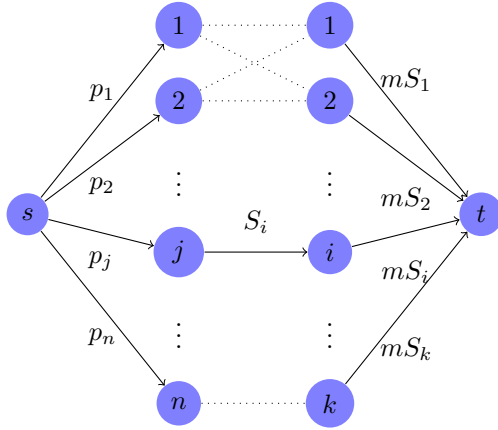


Figura 1.20.: Problema de fluxo para resolver a versão de decisão do problema $P \mid \text{pmtn}, r_i \mid L_{\max}$.

execução do projeto j . Um lucro pode ser negativo, e neste caso representa uma perda. Este problema pode ser reduzido para um problema de fluxo máximo s - t : crie um grafo G' com vértices $V = \{s, t\} \cup [n]$ é

- uma aresta (s, v) para todo $v \in [n]$ com $p_v > 0$, com capacidade p_v ,
- uma aresta (v, t) para todo $v \in [n]$ com $p_v < 0$, com capacidade $-p_v$, e
- uma aresta (u, v) para toda dependência $(v, u) \in A$, com capacidade ∞ .

(Note que projetos $v \in V$ com $p_v = 0$ não geram arcos (s, v) nem (v, t) .)

Lema 1.27

O valor de um corte $(X, \bar{X})$ em G' é mínimo, sse o lucro total dos projetos $S = X \setminus \{s\}$ é máximo. Além disso um corte mínimo em G' corresponde a uma seleção factível de projetos S .

Prova. Cada corte $(X, \bar{X})$ corresponde com uma seleção de projetos $S = X \setminus \{s\}$. Seja $\bar{S} = [n] \setminus S$. Uma seleção de projetos S é válida, caso para todo projeto $p \in S$, ela contém também todos projetos pré-requisitos de p . Logo, o corte correspondente não possui arcos com capacidade ∞ . Como o valor do corte $(s, V \setminus \{s\})$ é $\sum_{v \in [n] \mid p_v > 0} c_{sv}$ o corte mínimo é finito, e logo factível, porque não pode conter um arco entre um projeto selecionado e um projeto pré-requisito não selecionado.

O valor de um corte factível é

$$c(X, \bar{X}) = \sum_{a \in A(X, \bar{X})} c_a = \sum_{v \in \bar{S} | p_v > 0} p_v - \sum_{v \in S | p_v < 0} p_v$$

e nos temos

$$\begin{aligned} \sum_{v \in [n] | p_v > 0} p_v - c(X, \bar{X}) &= \sum_{v \in [n] | p_v > 0} p_v - \sum_{v \in \bar{S} | p_v > 0} p_v + \sum_{v \in S | p_v < 0} p_v \\ &= \sum_{v \in S | p_v > 0} p_v + \sum_{v \in S | p_v < 0} p_v \\ &= \sum_{v \in S} p_v, \end{aligned}$$

i.e. o lucro total da seleção S . Logo o lucro total é máximo sse o valor do corte é mínimo. ■

Vencendo um torneio. Suponha que temos um torneio de q equipes e que elas já ganharam $w_1, \dots, w_q$ vezes até agora. Para cada par de equipes, ainda temos g_{ij} jogos pela frente (g é simétrico). A equipe 1 ainda pode terminar em primeiro lugar, ou seja, ter o maior número de vitórias?

Para a equipe i , seja $r_i = \sum_j g_{ij}$ o número de jogos restantes. Precisamos que i) a equipe 1 vence todos os seus r_1 jogos restantes, portanto, tem $w_1 + r_1$ vitórias, e ii) todas as outras equipes $i \in T = [2, q]$ vencem no máximo m_i jogos, dado por $w_i + m_i < w_1 + r_1$ i.e. $m_i = w_1 + r_1 - w_i - 1$ (excluindo empates).

Caso algum $m_i < 0$ a equipe 1 já não pode ganhar mais. Caso contrário uma redução para um problema de fluxo é como segue. Crie um grafo com vértices s , $G = \binom{T}{2}$, T , e t e com os seguintes arcos:

- (s, g) para todo $g = (i, j) \in G$ de capacidade g_{ij} ,
- (g, i) e (g, j) para todo $g = (i, j) \in G$ de capacidade ∞ ,
- (i, t) para todo $i \in T$ de capacidade m_i .

Nos temos

Lema 1.28

Equipe 1 ainda pode vencer sse o grafo acima possui um fluxo st que satura (i.e. de valor $\sum_{(i,j) \in G} g_{ij}$).

1. Algoritmos em grafos

Prova. (Exercício. Nota que o que “flui” são jogos, e mandar fluxo em (g, i) ou (g, j) codifica que vence.) ■

A construção com q equipes tem $\Theta(q^2)$ vértices e $\Theta(q^2)$ arcos, logo usando um algoritmo para encontrar o fluxo de complexidade $O(nm)$ tem complexidade $O(q^4)$.

1.6.7. Outros problemas de fluxo

Obtemos um outro problema de fluxo em redes introduzindo *custos* de transporte por unidade de fluxo:

FLUXO DE MENOR CUSTO

Entrada Grafo direcionado $G = (V, A)$ com capacidades $c \in \mathbb{R}_+^{|E|}$ e custos $k \in \mathbb{R}_+^{|E|}$ nos arcos, um vértice origem $s \in V$, um vértice destino $t \in V$, e valor $v \in \mathbb{R}_+$.

Solução Um fluxo s - t f com valor v , respeitando as capacidades ($f \leq c$).

Objetivo Minimizar o *custo* $\sum_{a \in A} k_a f_a$ do fluxo.

Diferente do problema de fluxo máximo, o valor do fluxo é fixo.

1.6.8. Exercícios

Exercício 1.6

Mostra como podemos modificar o algoritmo de Dijkstra para encontrar o caminho mais curto entre dois vértices num grafo para encontrar o caminho com o maior gargalo entre dois vértices. (Dica: Enquanto o algoritmo de Dijkstra procura o caminho com a menor soma de distâncias, estamos procurando o caminho com o maior capacidade mínimo.)

1.7. Emparelhamentos

Dado um grafo não-direcionado $G = (V, A)$, um *emparelhamento* é uma seleção de arestas $M \subseteq A$ tal que todo vértice tem no máximo grau 1 em $G' = (V, M)$. (Notação: $M = \{u_1v_1, u_2v_2, \dots\}$.) O nosso interesse em emparelhamentos é maximizar o número de arestas selecionadas ou, no caso as arestas possuem pesos, maximizar o peso total das arestas selecionadas.

Para um grafo com pesos $c : A \rightarrow \mathbb{Q}$, seja $c(M) = \sum_{e \in M} c_e$ o *valor* do emparelhamento M .

EMPARELHAMENTO MÁXIMO (EM)

Entrada Um grafo não-direcionado $G = (V, A)$.

Solução Um emparelhamento $M \subseteq A$, i.e. um conjunto de arestas, tal que para todos vértices v temos $|N(v) \cap M| \leq 1$.

Objetivo Maximiza $|M|$.

EMPARELHAMENTO DE PESO MÁXIMO (EPM)

Entrada Um grafo não-direcionado $G = (V, A, c)$ com pesos $c : A \rightarrow \mathbb{Q}$ nas arestas.

Solução Um emparelhamento $M \subseteq A$.

Objetivo Maximiza o valor $c(M)$ de M .

Um emparelhamento se chama *perfeito* se todo vértice possui vizinho em M . Uma variação comum do problema é

EMPARELHAMENTO PERFEITO DE PESO MÍNIMO (EPPM)

Entrada Um grafo não-direcionado $G = (V, A, c)$ com pesos $c : A \rightarrow \mathbb{Q}$ nas arestas.

Solução Um emparelhamento perfeito $M \subseteq A$, i.e. um conjunto de arestas, tal que para todos vértices v temos $|N(v) \cap M| = 1$.

Objetivo Minimiza o valor $c(M)$ de M .

Observe que os pesos em todos problemas podem ser negativos. O problema de encontrar um emparelhamento de peso mínimo em $G = (V, A, c)$ é equivalente

1. Algoritmos em grafos

com EPM em $-G := (V, A, -c)$ (por quê?). Até EPPM pode ser reduzido para EPM.

Teorema 1.15

EPM e EPPM são problemas equivalentes.

Prova. Seja $G = (V, A, c)$ uma instância de EPM. Define um conjunto de vértices $V' = V \cup V^+$ que contém além de V mais $|V|$ vértices novos $V^+ = \{v^+ \mid v \in V\}$, e um grafo completo $G' = (V', V' \times V', c')$ com

$$c'_a = \begin{cases} -c_a, & \text{caso } a \in A, \\ 0, & \text{caso contrário.} \end{cases}$$

Um emparelhamento M em G de custo $c(M)$ corresponde com um emparelhamento M' em G' como segue. Dado M , define

$$M' = M \cup \{u'v' \mid uv \in M\} \cup \{vv' \mid v \text{ livre em } M\},$$

dado M' define $M = M' \cap V^2$. Ambas construções só adicionam ou removem arestas de custo 0 e o custo das demais arestas é invertido, logo $c'(M') = -c(M)$. Portanto, um EPPM em G' é um EPM em G .

Por outro lado, seja $G = (V, A, c)$ uma instância de EPPM. Define $C := 1 + \sum_{a \in A} |c_a|$, novos pesos $c'_e = C - c_e$ e um grafo $G' = (V, A, c')$. Para emparelhamentos M_1 e M_2 em G arbitrários temos

$$c(M_2) - c(M_1) \leq \sum_{\substack{a \in A \\ c_a > 0}} c_a - \sum_{\substack{a \in A \\ c_a < 0}} c_a = \sum_{a \in A} |c_a| < C. \quad (*)$$

Portanto, um emparelhamento de peso máximo em G' também é um emparelhamento de cardinalidade máxima: Para $|M_1| < |M_2|$ temos

$$c'(M_1) = C|M_1| - c(M_1) < C|M_1| + C - c(M_2) \leq C|M_2| - c(M_2) = c'(M_2),$$

onde a primeira desigualdade segue por (*). Se existe um emparelhamento perfeito no grafo original G , então o EPM em G' é perfeito e as arestas do EPM em G' definem um EPPM em G . ■

Formulações com programação inteira A formulação do problema do emparelhamento perfeito mínimo para $G = (V, A, c)$ é

$$\begin{aligned} \text{EPPM: } & \text{minimiza} && \sum_{a \in A} c_a x_a && (1.19) \\ & \text{sujeito a} && \sum_{u \in N(v)} x_{uv} = 1, && \forall v \in V, \\ & && x_a \in \mathbb{B}. \end{aligned}$$

A formulação do problema do emparelhamento máximo é

$$\begin{aligned} \text{EPM: maximiza} \quad & \sum_{a \in A} c_a x_a \\ \text{sujeito a} \quad & \sum_{u \in N(v)} x_{uv} \leq 1, \quad \forall v \in V, \\ & x_a \in \mathbb{B}. \end{aligned} \tag{1.20}$$

Observação 1.15

A matriz de coeficientes de (1.19) e (1.20) é totalmente unimodular no caso bi-partido (pelo teorema de Hoffman-Kruskal). Portanto: a solução da relaxação linear é inteira. (No caso geral isso não é verdadeiro, K_3 é um contra-exemplo, com solução ótima $3/2$). Observe que isso resolve o caso ponderado sem custo adicional. $\diamond$

Observação 1.16

O dual da relaxação linear de (1.19) é

$$\begin{aligned} \text{CIM: maximiza} \quad & \sum_{v \in V} y_v \\ \text{sujeito a} \quad & y_u + y_v \leq c_{uv}, \quad \forall uv \in A, \\ & y_v \in \mathbb{R}. \end{aligned} \tag{1.21}$$

e o dual da relaxação linear de (1.20)

$$\begin{aligned} \text{MVC: minimiza} \quad & \sum_{v \in V} y_v \\ \text{sujeito a} \quad & y_u + y_v \geq c_{uv}, \quad \forall uv \in A, \\ & y_v \in \mathbb{R}_+. \end{aligned} \tag{1.22}$$

Com pesos unitários $c_{uv} = 1$ e restringindo $y_v \in \mathbb{B}$ o primeiro dual é a formulação do conjunto independente máximo e o segundo da cobertura de vértices mínima. Portanto, a observação 1.15 rende no caso não-ponderado:

Teorema 1.16 (Berge, 1951)

Em grafos bi-partidos o tamanho da menor cobertura de vértices é igual ao tamanho do emparelhamento máximo.

Proposição 1.5

Um subconjunto de vértices $I \subseteq V$ de um grafo não-direcionado $G = (V, A)$ é um conjunto independente sse $V \setminus I$ é uma cobertura de vértices. Em particular um conjunto independente máximo I corresponde com uma cobertura de vértices mínima $V \setminus I$.

Prova. (Exercício 1.8.) $\blacksquare$ $\diamond$

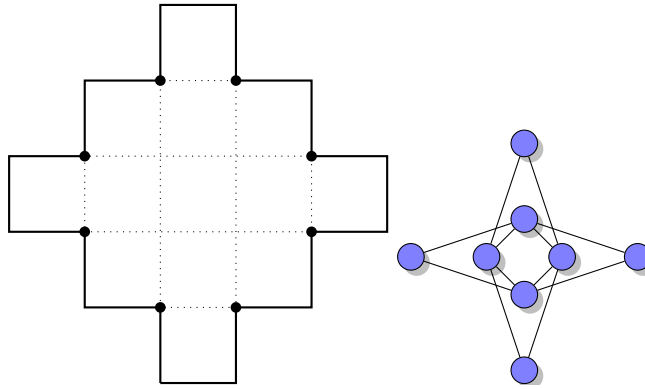


Figura 1.21.: Esquerda: Polígono ortogonal com $n = 8$ vértices de reflexo (pontos), $h = 0$ buracos. As cordas são pontilhadas. Direita: grafo de intersecção.

1.7.1. Aplicações

Alocação de tarefas Queremos alocar n tarefas a n trabalhadores, tal que cada tarefa é executada, e cada trabalhador executa uma tarefa. Os custos de execução dependem do trabalhador e da tarefa. Isso pode ser resolvido como problema de emparelhamento perfeito mínimo.

Particionamento de polígonos ortogonais

Teorema 1.17 (Sack e Urrutia (2000, cap. 11, Th. 1))

Um polígono ortogonal com n vértices de reflexo (ingl. reflex vertex, i.e., com ângulo interno maior que π), h buracos (ingl. holes) pode ser minimalmente particionado em $n - l - h + 1$ retângulos. A variável l é o número máximo de cordas (diagonais) horizontais ou verticais entre vértices de reflexo sem intersecção.

O número l é o tamanho do conjunto independente máximo no grafo de intersecção das cordas: cada corda é representada por um vértice, e uma aresta representa a duas cordas com intersecção. Pela proposição 1.7 podemos obter uma cobertura mínima via um emparelhamento máximo, que é o complemento de um conjunto independente máximo. Podemos achar o emparelhamento em tempo $O(n^{5/2})$ usando o algoritmo de Hopcroft-Karp, porque o grafo de intersecção é bi-partido (por quê?).

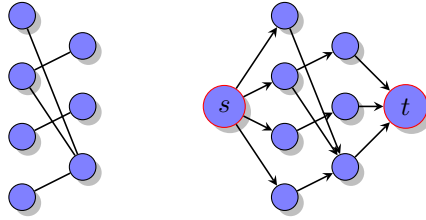


Figura 1.22.: Redução do problema de emparelhamento máximo para o problema do fluxo máximo

Problemas de agendamento O problema $1 \mid p_j = p \mid \sum w_j T_j$ é resolvido por um emparelhamento perfeito entre as tarefas e os intervalos de execução $[(i-1)p, ip]$, $i \in [n]$. Podemos resolver ainda $1 \mid p_j = 1, r_j \mid \sum w_j T_j$, observando que sempre existe uma solução com as tarefas executando nos intervalos $[t_i, t_i + 1]$, $i \in [n]$, definido por

$$t_0 = -\infty; \quad t_i = \max\{t_{i-1} + 1; r_i\}$$

e supondo que $r_1 \leq \dots \leq r_n$.

1.7.2. Grafos bi-partidos

Na formulação como programa inteiro a solução do caso bi-partido é mais fácil. Isso também é o caso para algoritmos combinatoriais, e portanto começamos estudar grafos bi-partidos.

Redução para o problema do fluxo máximo

Teorema 1.18

Um EM em grafos bi-partidos pode ser obtido em tempo $O(mn)$.

Prova. Introduz dois vértices s, t , liga s para todos vértices em V_1 , os vértices em V_1 com vértices em V_2 e os vértices em V_2 com t , com todos os pesos unitários. Aplica o algoritmo de Ford-Fulkerson para obter um fluxo máximo. O número de aumentos é limitado por n , cada busca tem complexidade $O(m)$, portanto o algoritmo de Ford-Fulkerson termina em tempo $O(mn)$. ■

Teorema 1.19

O valor do fluxo máximo é igual a cardinalidade de um emparelhamento máximo.

1. Algoritmos em grafos

Prova. Dado um emparelhamento máximo $M = \{v_{11}v_{21}, \dots, v_{1n}v_{2n}\}$, podemos construir um fluxo com arcos sv_{1i} , $v_{1i}v_{2i}$ e $v_{2i}t$ com valor $|M|$.

Dado um fluxo máximo, existe um fluxo integral equivalente (veja lema (1.14)). Na construção acima os arcos possuem fluxo 0 ou 1. Escolha todos os arcos entre V_1 e V_2 com fluxo 1. Não existe vértice com grau 2, pela conservação de fluxo. Portanto, os arcos formam um emparelhamento cuja cardinalidade é o valor do fluxo. ■

Solução não-ponderada combinatorial Um caminho $P = v_1v_2v_3 \dots v_k$ é *alternante* em relação a M (ou M -alternante) se $v_i v_{i+1} \in M$ sse $v_{i+1} v_{i+2} \notin M$ para todos $1 \leq i \leq k-2$. Um vértice $v \in V$ é *livre* em relação a M se ele tem grau 0 em M , e *emparelhado* caso contrário. Uma aresta $e \in E$ é *livre* em relação a M , se $e \notin M$, e *emparelhado* caso contrário. Escrevemos $|P| = k-1$ pelo *comprimento* do caminho P .

Observação 1.17

Caso tenhamos um caminho $P = v_1v_2v_3 \dots v_{2k}$ que é M -alternante com v_1 é v_{2k} livre, podemos obter um emparelhamento $M \setminus (P \cap M) \cup (P \setminus M)$ de tamanho $|M| + k - (k-1) = |M| + 1$. Notação: Diferença simétrica $M \oplus P = (M \setminus P) \cup (P \setminus M)$. A operação $M \oplus P$ é um *aumento* do emparelhamento M . ◇

Teorema 1.20 (Hopcroft e Karp (1973))

Seja M^* um emparelhamento máximo e M um emparelhamento arbitrário. O conjunto $M \oplus M^*$ contém pelo menos $k = |M^*| - |M|$ caminhos M -aumentantes disjuntos (de vértices). Um deles possui comprimento no máximo $|V|/k - 1$.

Prova. Considere os componentes de G em relação às arestas $M \oplus M^*$. Cada vértice possui no máximo grau 2. Portanto, os componentes são vértices livres, caminhos simples ou ciclos, todos disjuntos de vértices, por construção. Os caminhos e ciclos possuem alternadamente arestas de M e M^* , logo os ciclos têm comprimento par. Os caminhos de comprimento ímpar são M -aumentantes, porque para a solução ótima M^* não existem caminhos aumentantes. Ainda temos

$$|M^* \setminus M| = |M^*| - |M^* \cap M| = |M| - |M^* \cap M| + k = |M \setminus M^*| + k$$

e portanto $M \oplus M^*$ contém k arestas mais de M^* que de M . Isso mostra que existem pelo menos $|M^*| - |M|$ caminhos M -aumentantes, porque somente os caminhos de comprimento ímpar possuem exatamente uma aresta mais de M^* . Pelo menos um desses caminhos tem que ter um comprimento (em arestas) menor ou igual que $|V|/k - 1$, senão cada um possui pelo menos $|V|/k + 1$ vértices, i.e. eles contém em total mais que $|V|$ vértices. ■

Corolário 1.5 (Berge (1957))

Um emparelhamento é máximo sse não existe um caminho M -aumentante.

Rascunho de um algoritmo:

Algoritmo 1.7 (Emparelhamento máximo)

Entrada Grafo não-direcionado $G = (V, A)$.

Saída Um emparelhamento máximo M .

```

1   $M = \emptyset$ 
2  while (existe um caminho  $M$ -aumentante  $P$ ) do
3     $M := M \oplus P$ 
4  end while
5  return  $M$ 

```

Problema: como encontrar caminhos M -aumentantes eficientemente?

Observação 1.18

Um caminho M -aumentante começa num vértice livre em V_1 e termina num vértice livre em V_2 . Ideia: comece uma busca por largura com todos vértices livres em V_1 . Segue alternadamente arcos livres em M para encontrar vizinhos em V_2 e arcos em M , para encontrar vizinhos em V_1 . A busca pára ao encontrar um vértice livre em V_2 ou após visitar todos os vértices. Ela tem complexidade $O(m + n)$. $\diamond$

Teorema 1.21

O problema do emparelhamento máximo não-ponderado em grafos bi-partidos pode ser resolvido em tempo $O(mn)$.

Prova. Última observação e o fato que o emparelhamento máximo tem tamanho $O(n)$. $\blacksquare$

Observação 1.19

O último teorema é o mesmo que teorema 1.18. $\diamond$

Observação 1.20

Pelo teorema 1.20 sabemos que existem vários caminhos M -alternantes disjuntos (de vértices) e nos podemos aumentar M com todos eles em paralelo. Portanto, estruturamos o algoritmo em fases: cada fase procura um conjunto de caminhos aumentantes disjuntos e aplicá-los para obter um novo emparelhamento. Observe que pelo teorema 1.20 um aumento com o maior conjunto de caminhos M -alternantes disjuntos resolve o problema imediatamente, mas

1. Algoritmos em grafos

não sabemos como encontrar esse conjunto de forma eficiente. Portanto, procuramos somente um conjunto maximal de caminhos M -alternantes disjuntos de menor comprimento.

Podemos encontrar um tal conjunto após uma busca em profundidade usando o DAG (grafo direcionado acíclico) definido pela busca por profundidade. (i) Escolhe um vértice livre em V_2 . (ii) Segue os predecessores para encontrar um caminho aumentante. (iii) Coloca todos vértices em uma fila de deleção. (iv) Processa a fila de deleção: Até que a fila esteja vazia, remove um vértice dela. Remove todos arcos adjacentes no DAG. Caso um vértice sucessor após de remoção de um arco possui grau de entrada 0, coloca ele na fila. (v) Repete o procedimento no DAG restante, para encontrar outro caminho, até não existirem mais vértices livres em V_2 . A nova busca ainda possui complexidade $O(m)$. $\diamond$

O que ganhamos com essa nova busca? Os seguintes dois lemas dão a resposta:

Lema 1.29

Em cada fase o comprimento de um caminho aumentante mínimo aumenta por pelo menos dois.

Lema 1.30

O algoritmo termina em no máximo $\sqrt{n}$ fases.

Teorema 1.22

O problema do emparelhamento máximo não-ponderado em grafos bi-partidos pode ser resolvido em tempo $O(m\sqrt{n})$.

Prova. Pelas lemas 1.29 e 1.30 e a observação que toda fase pode ser completada em $O(m)$. $\blacksquare$

Usaremos outro lema para provar os dois lemas acima.

Lema 1.31

Seja M um emparelhamento, P um caminho M -aumentante mínimo, e Q um caminho $M \oplus P$ -aumentante. Então $|Q| \geq |P| + 2|P \cap Q|$. ($P \cap Q$ denota as arestas em comum entre P e Q .)

Prova. Caso P e Q não possuam vértices em comum, Q é M -aumentante, $P \cap Q = \emptyset$ e a desigualdade é consequência da minimalidade de P . Caso contrário, P e Q possuem um vértice em comum, e logo também uma aresta, senão $M \oplus P \oplus Q$ possui um vértice de grau dois. $P \oplus Q$ consiste em dois caminhos, e eventualmente um coleção de ciclos. Os dois caminhos são M -aumentantes, pelas seguintes observações:

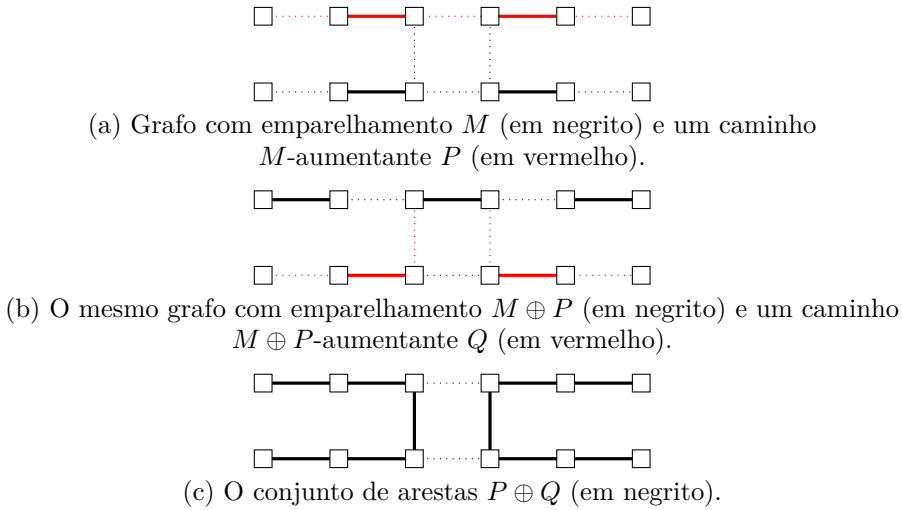


Figura 1.23.: Ilustração do lema 1.31.

1. Cada caminho inicia numa ponta de Q e termina numa ponta de P . Além disso, em M as pontas de P são livres, porque P é M -aumentante; as pontas de Q também são livres em M : são livres $M \oplus P$, e logo não pertencem a P . (Nenhum outro vértice de $P \oplus Q$ é livre em relação a M : P só contém dois vértices livres e Q só contém dois vértices livres em $Q \setminus P$.)
2. Os dois caminhos são M -alternantes. Começando com um vértice livre em Q , a parte do caminho Q em $Q \setminus P$ é M -alternante, porque as arestas livres em $M \oplus P$ são exatamente as arestas livres em M . O caminho entra em P com uma aresta livre, porque todo vértice em P já está emparelhado em $M \oplus P$. A parte de P em $P \oplus Q$ tem que continuar com aresta livre em $M \oplus P$, e logo aresta emparelhada em M . Logo, temos um caminho M -alternante.

Os dois caminhos M -aumentantes em $P \oplus Q$ tem que ser maiores que $|P|$. Com isso temos $|P \oplus Q| \geq 2|P|$ e

$$|Q| = |P \oplus Q| + 2|P \cap Q| - |P| \geq |P| + 2|P \cap Q|.$$

■

Prova. (do lema 1.29). Seja S o conjunto de caminhos M -aumentantes da fase anterior, e P um caminho aumentante. Caso P é disjunto de todos

1. Algoritmos em grafos

caminhos em S , ele deve ser mais comprido, porque S é um conjunto máximo de caminhos aumentantes. Caso P possui um vértice em comum com algum caminho em S , ele possui também um arco em comum (por quê?) e podemos aplicar lema 1.31. ■

Prova. (do lema 1.30). Seja M^* um emparelhamento máximo e M o emparelhamento obtido após $\sqrt{n}/2$ fases. O comprimento de qualquer caminho M -aumentante é no mínimo $\sqrt{n}$, pelo lema 1.29. Pelo teorema 1.20 existem pelo menos $|M^*| - |M|$ caminhos M -aumentantes disjuntos de vértices. Mas então $|M^*| - |M| \leq \sqrt{n}$, porque no caso contrário eles possuem mais que n vértices em total. Como o emparelhamento cresce pelo menos um em cada fase, o algoritmo executa no máximo mais $\sqrt{n}$ fases. Portanto, o número total de fases é no máximo $3/2\sqrt{n} = O(\sqrt{n})$. ■

O algoritmo de Hopcroft-Karp é o melhor algoritmo conhecido para encontrar emparelhamentos máximos em grafos bipartidos não-ponderados esparsos⁵. Para subclasses de grafos bipartidos existem algoritmos melhores. Por exemplo, existe um algoritmo randomizado para grafos bipartidos regulares com complexidade de tempo esperado $O(n \log n)$ (Goel et al., 2010).

Sobre a implementação A seguir supomos que o conjunto de vértices é $V = [1, n]$ e um grafo $G = (V, A)$ bi-partido com partição $V_1 \dot{\cup} V_2$. Podemos representar um emparelhamento usando um vetor `mate`, que contém, para cada vértice emparelhado, o índice do vértice vizinho, e 0 caso o vértice é livre.

O núcleo de uma implementação do algoritmo de Hopcroft e Karp é descrito na observação 1.20: ele consiste numa busca por largura até encontrar um ou mais caminhos M -alternantes mínimos e depois uma fase que extrai do DAG definido pela busca um conjunto máximo de caminhos disjuntos (de vértices). A busca por largura começa com todos vértices livres em V_1 . Usamos um vetor H para marcar os arcos que fazem parte do DAG definido pela busca por largura⁶ e um vetor m para marcar os vértices visitados.

```
1  search_paths( $M$ ) :=
2    for all  $v \in V$  do  $m_v := \text{false}$ 
3
4     $U_1 := \{v \in V_1 \mid v \text{ livre}\}$ 
5    for all  $u \in U_1$  do  $d_u := 0$ 
6
7    do
```

⁵Feder e Motwani (1991) e Feder e Motwani (1995) propuseram um algoritmo em $O(\sqrt{nm}(2 - \log_n m))$ que é melhor em grafos densos.

⁶ H , porque o DAG se chama *árvore Húngara* na literatura.

```

8      { determina vizinhos em  $U_2$  via arestas livres }
9       $U_2 := \emptyset$ 
10     for all  $u \in U_1$  do
11          $m_u := \text{true}$ 
12         for all  $uv \in A$ ,  $uv \notin M$  do
13             if not  $m_v$  then
14                  $d_v := d_u + 1$ 
15                  $U_2 := U_2 \cup v$ 
16             end if
17         end for
18     end for
19
20     { determina vizinhos em  $U_1$  via arestas emparelhadas }
21     found := false           { pelo menos um caminho encontrado? }
22      $U_1 := \emptyset$ 
23     for all  $u \in U_2$  do
24          $m_u := \text{true}$ 
25         if ( $u$  livre) then
26             found := true
27         else
28              $v := \text{mate}[u]$ 
29             if not  $m_v$  then
30                  $d_v := d_u + 1$ 
31                  $U_1 := U_1 \cup v$ 
32             end if
33         end for
34     end for
35     while (not found)
36 end

```

Após a busca, podemos extrair um conjunto máximo de caminhos M -alternantes mínimos disjuntos. Enquanto existe um vértice livre em V_2 , nos extraímos um caminho alternante que termina em v como segue:

```

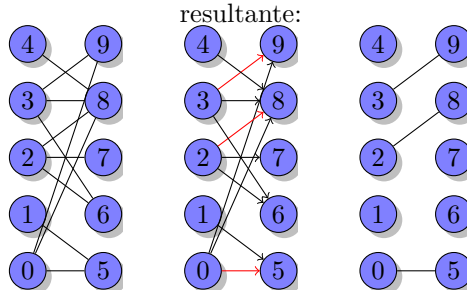
1  extract_paths() :=
2      while existe vértice  $v$  livre em  $V_2$  do
3          aplica um busca em profundidade a partir de  $v$  em  $H$ 
4              (procurando um vértice livre em  $V_1$ )
5          remove todos vértices visitados durante a busca
6          caso um caminho alternante  $P$  foi encontrado:  $M := M \oplus P$ 
7      end while
8  end

```

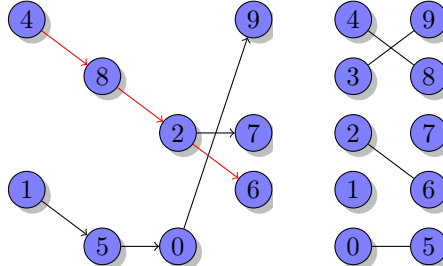
Exemplo 1.7

Segue um exemplo da aplicação do algoritmo de Hopcroft-Karp.

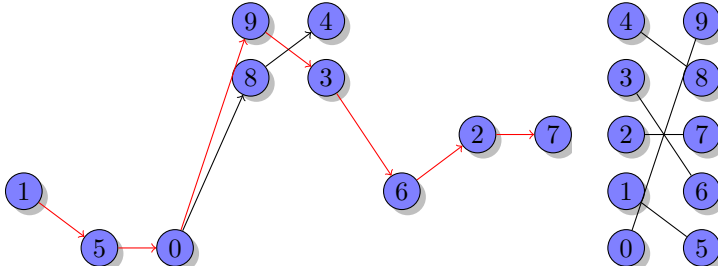
Grafo original, árvore Húngara primeira iteração e emparelhamento



Árvore Húngara segunda iteração e emparelhamento resultante:



Árvore Húngara terceira iteração e emparelhamento resultante:



◇

Emparelhamentos, coberturas e conjuntos independentes

Proposição 1.6

Seja $G = (S \cup T, A)$ um grafo bipartido e $M \subseteq A$ um emparelhamento em G . Seja R o conjunto de todos vértices livres em S e todos vértices alcançáveis por uma busca na árvore Húngara (i.e. via arestas livres de S para T e arestas

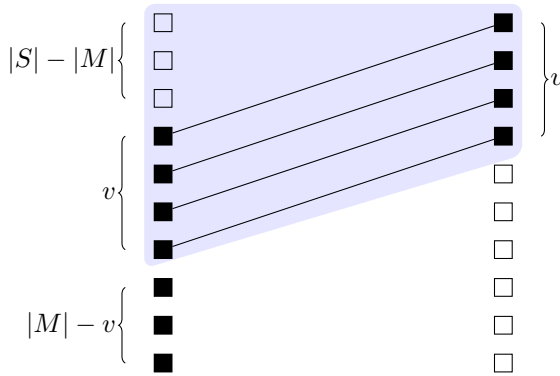


Figura 1.24.: Ilustração da prova da proposição 1.7.

emparelhadas de T para S). Então $(S \setminus R) \cup (T \cap R)$ é uma cobertura de vértices em G .

Prova. Seja $uv \in A$ uma aresta não coberta. Logo $u \in S \setminus (S \setminus R) = S \cap R$ e $v \in T \setminus (T \cap R) = T \setminus R$. Caso $uv \notin M$, uv é parte da árvore Húngara e com $u \in R$ temos $v \in R$, uma contradição. Mas caso $uv \in M$, vu é parte da árvore Húngara, o único predecessor possível de $u \in R$ é v , logo $v \in R$, novamente uma contradição. ■

A próxima proposição mostra que no caso de um emparelhamento máximo obtemos uma cobertura mínima.

Proposição 1.7

Seja $G = (S \dot{\cup} T, A)$. Caso M seja um emparelhamento máximo o conjunto $(S \setminus R) \cup (T \cap R)$ é uma cobertura mínima.

Prova. O tamanho de qualquer emparelhamento M é um limite inferior para o tamanho de qualquer cobertura, porque uma cobertura tem que conter pelo menos um vértice da cada aresta emparelhada. Logo é suficiente demonstrar que $|(S \setminus R) \cup (T \cap R)| = |M|$.

Temos $|(S \setminus R) \cup (T \cap R)| = |S \setminus R| + |T \cap R|$ porque S e T são disjuntos. Vamos demonstrar que $|T \cap R| = v$ implica $|S \setminus R| = |M| - v$.

Suponha $|T \cap R| = v$. Como M é máximo não existe caminho M -aumentante, e logo $T \cap R$ contém somente vértices emparelhados. Por isso o número de vértices emparelhados em $S \cap R$ também é v . Além disso $S \cap R$ contém todos $|S| - |M|$ vértices livres em S . Logo $|S \setminus R| = |S| - (|S| - |M|) - v = |M| - v$. ■

1. Algoritmos em grafos

Observação 1.21

O complemento $V \setminus C$ de uma cobertura C é um conjunto independente (por quê?). Logo um emparelhamento M que define um conjunto R de acordo com a proposição 1.6 corresponde com um conjunto independente $(S \cap R) \cup (T \setminus R)$, e caso M é máximo, o conjunto independente também. $\diamond$

Solução ponderada em grafos bi-partidos Dado um grafo $G = (S \dot{\cup} T, A)$ bipartido com pesos $c : A \rightarrow \mathbb{Q}_+$ queremos achar um emparelhamento de maior peso. Escrevemos $V = S \cup T$ para o conjunto de todos vértices em G .

Observação 1.22

O caso ponderado pode ser restrito para emparelhamentos perfeitos: caso S e T possuem cardinalidade diferente, podemos adicionar vértices, e depois completar todo grafo com arestas de custo 0. O problema de encontrar um emparelhamento perfeito máximo (ou mínimo) em grafos ponderados é conhecido pelo nome “problema de alocação” (ingl. assignment problem). $\diamond$

Observação 1.23

A redução do teorema 1.18 para um problema de fluxo máximo não se aplica no caso ponderado. Mas, com a simplificação da observação 1.22, podemos reduzir o problema no caso ponderado para um problema de fluxo de menor custo: a capacidade de todas arestas é 1, e o custo de transportação são os pesos das arestas. Como o emparelhamento é perfeito, procuramos um fluxo de valor $|V|/2$, de menor custo. $\diamond$

O dual do problema 1.22 é a motivação para

Definição 1.4

Um *rotulamento* é uma atribuição $y : V \rightarrow \mathbb{R}_+$. Ele é *viável* caso $y_u + y_v \geq c_a$ para todas arestas $a = \{u, v\}$. (Um rotulamento viável é uma *c-cobertura de vértices*.) Uma aresta é *apertada* (ingl. tight) caso $y_u + y_v = c_a$. O subgrafo de arestas apertadas é $G_y = (V, A', c)$ com $A' = \{a \in A \mid a \text{ apertada em } y\}$.

Pelo teorema forte de dualidade e o fato que a relaxação linear dos sistemas acima possui uma solução integral (ver observação 1.15) temos

Teorema 1.23 (Egerváry (1931))

Para um grafo bi-partido $G = (S \dot{\cup} T, A, c)$ com pesos não-negativos $c : A \rightarrow \mathbb{Q}_+$ nas arestas, o maior peso de um emparelhamento perfeito é igual ao peso da menor c -cobertura de vértices.

O método húngaro Aplicando um caminho M -aumentante $P = (v_1 v_2 \dots v_{2n+1})$ produz um emparelhamento de peso $c(M) + \sum_{i \text{ ímpar}} c_{v_i v_{i+1}} - \sum_{i \text{ par}} c_{v_i v_{i+1}}$. Isso motiva a definição de uma árvore Húngara ponderada. Para um emparelhamento M , seja H_M o grafo direcionado com as arestas $e \in M$ orientadas de T para S para obter um arco a com peso $l_a := w_e$, e com as restantes arestas $e \in A \setminus M$ orientadas de S para T para obter um arco a com peso $l_a := -w_a$. Com isso a aplicação do caminho M -aumentante P produz um emparelhamento de peso $c(M) - l(P)$ em que $l(P) = \sum_{1 \leq i \leq 2n} l_{v_i v_{i+1}}$ é o comprimento do caminho P .

Com isso podemos modificar o algoritmo para emparelhamentos máximos para

Algoritmo 1.8 (Emparelhamento de peso máximo)

Entrada Um grafo não-direcionado ponderado $G = (V, E, c)$.

Saída Um emparelhamento de maior peso $c(M)$.

```

1   $M = \emptyset$ 
2  while (existe um caminho  $M$ -aumentante  $P$ ) do
3    encontra o caminho  $M$ -aumentante mínimo  $P$  em  $H_M$ 
4    caso  $l(P) \geq 0$ : return  $M$ ;
5     $M := M \oplus P$ 
6  end while
7  return  $M$ 

```

Chamaremos um emparelhamento M *extremo* caso ele possua o maior peso entre todos emparelhamentos de tamanho $|M|$.

Observação 1.24

O grafo H_M de um emparelhamento extremo M não possui ciclo (par) negativo. Isso seria uma contradição com a maximalidade de M . Portanto podemos encontrar o caminho mínimo no passo 3 do algoritmo usando o algoritmo de Bellman-Ford em tempo $O(mn)$. Com isso a complexidade do algoritmo é $O(mn^2)$. $\diamond$

Observação 1.25

Lembrando Bellman-Ford: Seja $d_k(t)$ a distância mínima entre s e t com um caminho usando no máximo k arcos ou ∞ caso tal caminho não existe. Temos

$$d_{k+1}(t) = \min\{d_k(t), \min_{(u,t) \in A} d_k(u) + l(u,t)\} \forall t \in V,$$

com $d_0(t) = 0$ caso t é um vértice livre em S e $d_0(t) = \infty$ caso contrário. (O algoritmo se aplica igualmente para as distâncias de um conjunto de vértices,

1. Algoritmos em grafos

como o conjunto de vértices livres em S .) A atualização de k para $k + 1$ é possível em $O(m)$ e como $k < n$ o algoritmo possui complexidade $O(nm)$. $\diamond$

Teorema 1.24

Cada emparelhamento encontrado no Algoritmo 1.8 é extremo.

Prova. Por indução sobre $|M|$. Para $M = \emptyset$ o teorema é correto. Seja M um emparelhamento extremo, P o caminho aumentante encontrado pelo algoritmo 1.8 e N um emparelhamento de tamanho $|M| + 1$ arbitrário. Como $|N| > |M|$, pelo teorema 1.20 $M \oplus N$ contém um caminho M -aumentante Q . Sabemos $l(Q) \geq l(P)$ pela minimalidade de P . $N \oplus Q$ é um emparelhamento de cardinalidade $|M|$ (Q é um caminho com arestas em N e M com uma aresta em N a mais), logo $c(N \oplus Q) \leq c(M)$. Com isso temos

$$c(N) = c(N \oplus Q) - l(Q) \leq c(M) - l(P) = c(M \oplus P)$$

(observe que o comprimento $l(Q)$ é definido no emparelhamento M). $\blacksquare$

Proposição 1.8

Caso não exista caminho M -aumentante com comprimento negativo no Algoritmo 1.8, M é máximo.

Prova. Suponha que existe um emparelhamento N com $c(N) > c(M)$. Logo $|N| > |M|$ porque M possui o maior peso entre todos emparelhamentos de cardinalidade no máximo $|M|$. Pelo teorema de Hopcroft-Karp, existem $|N| - |M|$ caminhos M -aumentantes disjuntos de vértices em $N \oplus M$. Nenhum deles tem comprimento negativo, pelo critério de parada do algoritmo. Portanto $c(N) \leq c(M)$, uma contradição. $\blacksquare$

Fato 1.1

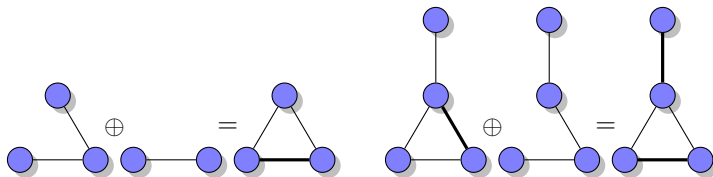
É possível encontrar o caminho mínimo no passo 3 em tempo $O(m + n \log n)$ usando uma transformação para distâncias positivas e aplicando o algoritmo de Dijkstra. Com isso um algoritmo em tempo $O(n(m + n \log n))$ é possível.

1.7.3. Emparelhamentos em grafos não-bipartidos

O teorema de Berge 1.16 (ou o de Hopcroft & Karp 1.20) vale em qualquer grafo.

Exemplo 1.8 (Caminhos M -aumentantes em grafos não-bipartidos)

Consequência: dado um caminho M -aumentante, a sua aplicação produz emparelhamentos maiores.



◇

Portanto, o problema central em grafos gerais ainda é

Problema 1.1 (Encontra um caminho M -aumentante)

Dado um emparelhamento M , retorne um caminho M -aumentante, caso existir.

Dado uma solução em tempo $T(n)$, o algoritmo canônico (inicia com $M = \emptyset$; repetidamente resolve Problema 1.1; caso tem caminho M -aumentante P , $M := M \oplus P$ e repete; senão: para) termina em no máximo $\lfloor n/2 \rfloor = O(n)$ iterações em tempo $O(nT(n))$.

O caso não-ponderado

Primeiramente vamos entender porque a abordagem utilizada em grafos bipartidos $G = (S \dot{\cup} T, E)$ falha. Sejam X os vértices livres em G . Em grafos bipartidos encontramos um caminho M -aumentante por uma busca em largura:

Algoritmo 1.9 (Busca caminho M -aumentante)

Inicia em $S_0 = S \cap X$. Dado S_i sejam T_i os vértices ainda não explorados alcançáveis por S_i via arestas livres. Caso T_i contém um vértice livre, termina, senão sejam S_{i+1} os vértices ainda não explorados alcançáveis por T_i via arestas emparelhadas. Repete.

Proposição 1.9

Algoritmo 1.9 sempre encontra pelo menos um caminho mais curto M -aumentante em grafos bipartidos.

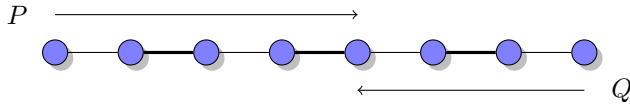
Prova. Para todo caminho M -aumentante mais curto $P = (v_0, v_1, \dots, v_t)$, vértice v_i é encontrado na iteração i . Pela existência do caminho P , é claro que vértice v_i é descoberto em no máximo i iterações. Agora assume v_i é o vértice de menor índice descoberto numa iteração $j < i$, por um caminho alternante $Q = (u_0, u_1, \dots, u_j = v_i)$ iniciando em u_0 livre. Temos os seguintes casos:

1. Algoritmos em grafos

- a) j é par, e i é par. Logo $u_{j-1}v_i \in M$, e $v_{i-1}v_i \in M$, e por isso $u_{j-1} = v_{i-1}$ em contradição com a minimalidade de i .
- b) j é ímpar, i é ímpar. Logo $u_{j-1}v_i \notin M$, $v_iv_{i+1} \in M$ e Q junto com o caminho $(v_i, v_{i+1}, \dots, v_t)$ é um caminho M -aumentante de comprimento $j + (t - i) < t$, em contradição com a minimalidade de P .
- c) j é par, e i é ímpar. Logo $v_i \in S_{j/2}$ e $v_i \in T_{\lfloor i/2 \rfloor}$, em contradição com G sendo bipartido.
- d) j é ímpar, i é par. Similar ao caso c) temos $v_i \in T_{\lfloor j/2 \rfloor}$ e $v_i \in S_{i/2}$, uma contradição.

■

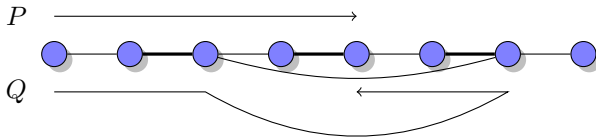
Num grafo geral não temos a partição em S e T . Uma possível alternativa é iniciar a busca em $R_0 = X$ e aplicar a mesma busca alternante para descobrir uma sequência de conjuntos R_i . Mas mesmo em grafos bipartidos, Algoritmo 1.9 então falha: em



os caminhos alternantes P e Q se encontram. Note que isso corresponde com o caso d) da Proposição 1.9, mas não é mais uma contradição, porque os conjuntos R_i contém vértices de S e T .

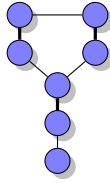
Esse problema pode ser resolvido por i) modificar o Algoritmo 1.9 para combinar caminhos encontrados em buscas iniciados em vértices livres diferentes, ou, mais simples, mas menos eficiente, ii) buscar a partir de cada vértice $x \in X$ separadamente.

Por ser mais simples considera a solução ii): mesmo procurando a partir de um único vértice $x \in X$ falha em grafos gerais. Por exemplo:



Note que isso corresponde com o caso c) da Proposição 1.9 e não é mais uma contradição, porque em grafos gerais podemos ter laços ímpares.

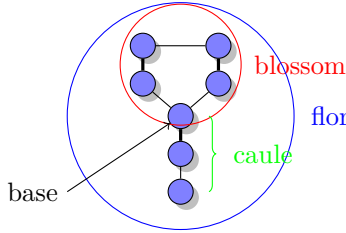
O exemplo acima sugere que ciclos ímpares formam o núcleo do Problema 1.1. A árvore de busca do exemplo anterior pode ser visualizado como



Isso é o motivo para:

Definição 1.5 (Flor)

Seja $P = (v_0, v_1, \dots, v_t)$ uma caminhada M -alternante. Caso (i) $v_0 \in X$, (ii) todos vértices $v_0, \dots, v_{t-1}$ são distintos, (iii) t é ímpar, e (iv) existe um $i < t$, i par, tal que $v_i = v_t$, P é chamado uma *flor*, com *caule* $(v_0, \dots, v_i)$, *base* v_i , e *blossom* $B = (v_i, v_{i+1}, \dots, v_t)$.



Caminhadas M -alternantes. Como encontrar *caminhos* M -alternantes falha, uma outra ideia, que vamos discutir agora, é buscar *caminhadas* M -alternantes. Para conseguir isso, vamos introduzir um grafo direcionado auxiliar $D = (V, A)$, onde $A = \{uv \mid ux \in E, xv \in M \text{ para um } x \in V\}$. A ideia é substituir $\textcircled{u} - \textcircled{x} - \textcircled{v}$ por $\textcircled{u} \rightarrow \textcircled{v}$.

Essa construção tem a seguinte característica

Proposição 1.10

Sejam $N(X)$ todos vértices vizinhos de vértices livres X . Então D possui um caminho $X-N(X)$ sse G possui uma caminhada $X-X$.

Prova. “ $\Rightarrow$ ”: é suficiente expandir os arcos e adicionar uma aresta final para um vértice livre.

“ $\Leftarrow$ ”: dado $W = (v_0, \dots, v_t)$ remove o vértice livre v_t para obter uma caminhada terminando em $N(X)$. Podemos assumir que v_{t-1} é o único vértice em $N(X)$, senão um prefixo de W serve. Contraí arestas $v_{2i}v_{2i+1}$ para arcos e remove eventuais ciclos para obter um caminho. Como o vértice inicial é livre e o vértice final v_{t-1} não tem sucessor, ambos não fazem parte de um ciclo. Logo o caminho resultante é $X-N(X)$. ■

1. Algoritmos em grafos

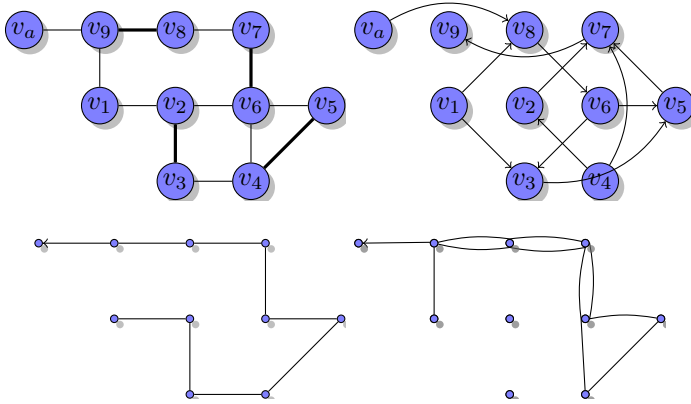


Figura 1.25.: Grafo com emparelhamento, grafo auxiliar e duas caminhadas M -alternantes.

Isso nos permite, em tempo $O(m)$ usar uma busca por profundidade em D iniciando em X e terminando em algum vértice em $N(X)$ para encontrar uma caminhada M -alternante X - X . Porém o seguinte exemplo mostra que as flores ainda são uma fonte de problemas para caminhadas M -alternantes.

Exemplo 1.9

Considere o exemplo da Figure 1.25. O caminho $v_1 v_3 v_5 v_7 v_9$ corresponde com o caminho M -aumentante $v_1 v_2 v_3 v_4 v_5 v_6 v_7 v_8 v_9 v_a$. Mas caminho $v_1 v_8 v_6 v_5 v_7 v_9$ que corresponde com o caminhada $v_1 v_9 v_8 v_7 v_6 v_4 v_5 v_6 v_7 v_8 v_9 v_a$ que não é M -aumentante, mesmo sendo M -alternante entre dois vértices livres. O problema novamente é o laço ímpar $v_6 v_4 v_5 v_6$. $\diamond$

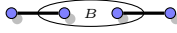
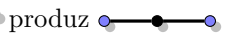
Note que no Exemplo 1.9 o prefixo $v_1 v_9 v_8 v_7 v_6 v_4 v_5 v_6$ da segunda caminhada é uma flor. Isso de fato sempre é o caso:

Proposição 1.11

Seja $P = (v_0, v_1, \dots, v_t)$ uma caminhada M -alternante X - X mais curta. Então ou (i) P é um caminho M -aumentante, ou (ii) o prefixo $(v_0, v_1, \dots, v_j)$ para algum $j \leq t$ é uma flor.

Prova. Assume que P não é um caminho. Selecciona $i < j$ tal que $v_i = v_j$ e j mínimo. Então todos vértices $v_0, \dots, v_{j-1}$ são distintos. A diferença $j - i$ não pode ser par, senão podemos remover $(v_i, \dots, v_j)$ para obter a caminhada $(v_0, \dots, v_i)$ M -alternante X - X mais curta que P , em contradição com a minimalidade de P . Ainda, caso i é ímpar e j é par, temos $v_i v_{i+1} \in M$, e

$v_{j-1}v_j \in M$ e como $v_i = v_j$ também $v_{i+1} = v_{j-1}$, em contradição com a minimalidade de j . Logo i é par, j é ímpar, e $(v_0, \dots, v_j)$ satisfaz todos os critérios da Definição 1.5 e por isso é uma flor. ■

Lidar com flores. O problema central então é como lidar com flores. Esse problema tem uma solução simples: ao encontrar uma flor, contrai a sua blossom B . Vamos escrever G/B para o grafo resultante, e assumir que ele tem vértices $G \setminus B \cup \{B\}$ (ou seja o vértice B representa a blossom contraída). Ao contrair, vamos descartar laços. Ainda dado um emparelhamento M , M/B é o emparelhamento após a contração. (Nota que somente caso $|M \cap \delta(B)| \leq 1$ onde $\delta(B) = \{uv \mid u \in B, v \notin B\}$, M/B é um emparelhamento; por exemplo  produz  que não é.)

O seguinte teorema nos garante a corretude dessa estratégia.

Teorema 1.25

M é um emparelhamento máximo em G sse M/B é um emparelhamento máximo em G/B .

Prova. Seja $B = (v_i, v_{i+1}, \dots, v_t)$.

“ $\Rightarrow$ ”: Assume M/B não é máximo, e seja P um caminho M/B -aumentante. Vamos mostrar que então existe um caminho M -aumentante, logo M também não é máximo. Caso $B \notin P$, P já é M -aumentante. Caso contrário, seja uB a aresta em P que entra em B . Podemos assumir que uB é livre em M/B (senão inverte P). Seja uv_j , $i \leq j \leq t$, a aresta correspondente em G . Caso j é ímpar, podemos expandir B para $v_j, v_{j+1}, \dots, v_t$ para obter um caminho M -aumentante (nota que $v_j v_{j+1} \in M$) em G . Similarmente, caso j é par, podemos expandir B para $v_j, v_{j-1}, \dots, v_i$ para obter um caminho M -aumentante.

“ $\Leftarrow$ ”: Assume M não é máximo. Para caule Q , $M \oplus Q$ é um emparelhamento da mesma cardinalidade porque v_i tem índice par, pela Definição 1.5. Então podemos supor que $i = 0$; nota que isso torna v_i livre em M , e logo B é livre em M/B . Dado um caminho M -aumentante $P = (u_0, \dots, u_s)$, então vamos construir um caminho M/B -aumentante em G/B , mostrando que M/B não é máximo. Caso $P \cap B = \emptyset$, P já é um caminho M/B -aumentante. Caso contrário, podemos assumir $u_0 \notin B$ (senão inverte P). Seja u_j , $j > 0$ o primeiro vértice em P em B . O caminho $(u_0, \dots, u_j)$ é M/B -alternante, e como B é livre em G/B , também aumentante. ■

Combinando as peças. Com isso podemos resolver o Problema 1.1, como segue.

Tabela 1.4.: Resumo emparelhamentos. Aqui $C = \max_{a \in A} |c_a|$.

	Cardinalidade	Ponderado
Bi-partido	$O(n\sqrt{mn/\log n})$ (Alt et al., 1991) $O(m\sqrt{n \frac{\log(n^2/m)}{\log n}})$ (Feder e Motwani, 1995)	$O(nm + n^2 \log n)$ (Kuhn, 1955; Munkres, 1957)
Geral	$O(m\sqrt{n})$ (Micali e Vazirani, 1980) $O(m\sqrt{n \frac{\log(n^2/m)}{\log n}})$ (Goldberg e Karzanov, 2004; Fremuth-Paeger e Jungnickel, 2003)	$O(n^3)$ (Edmonds, 1965) $O(mn + n^2 \log n)$ (Gabow, 1990) $O(m\sqrt{n} \log nC)$ (Duan et al., 2018)

Algoritmo 1.10 (Busca caminho M -aumentante)

- 1) Encontra um caminho P M -alternante $X-X$ mais curto. Caso não tenha: para, não existe caminho M -aumentante. (Proposição 1.10).
- 2) Pela Proposição 1.11 ou
 - a) P é um caminho M -aumentante: retorna P ; ou
 - b) um prefixo de P é uma flor com blossom B : recursivamente encontra um caminho P' M/B -aumentante em G/B . Depois expande P' para um caminho M -aumentante P'' em G , de acordo com Teorema 1.25. retorna P'' .

A corretude do algoritmo segue das proposições e teoremas mencionadas. A complexidade de encontrar o caminho P no passo 1, bem como a complexidade da contração para G/B no passo 2c é $O(m)$. Por isso, todas chamadas recursivas não custam mais que $O(nm)$, porque em cada recursão temos pelo menos um vértice a menos. Logo, o algoritmo canônico termina em tempo $O(n^2m)$.

1.7.4. Notas

Duan et al. (2011) apresentam técnicas de aproximação para emparelhamentos.

1.7.5. Exercícios

Exercício 1.7

É possível somar uma constante $c \in \mathbb{R}$ para todos custos de uma instância do EPM ou EPPM, mantendo a otimalidade da solução?

Exercício 1.8

Prove a proposição 1.5.

2. Tabelas hash

Em *hashing* nosso interesse é uma estrutura de dados H para gerenciar um conjunto de chaves sobre um universo U e que oferece as operações de um *dicionário*:

- Inserção de uma chave $c \in U$: $\text{insert}(c, H)$
- Deleção de uma chave $c \in U$: $\text{delete}(c, H)$
- Teste da pertinência: Chave $c \in H$? $\text{lookup}(c, H)$

Uma característica do problema é que tamanho $|U|$ do universo de chaves possíveis pode ser grande, por exemplo o conjunto de todos strings ou todos números inteiros. Portanto usar a chave como índice de um vetor de booleano não é uma opção. Uma tabela hash é uma alternativa para outras estruturas de dados de dicionários, p.ex. árvores. O princípio de tabelas hash: aloca uma tabela de tamanho m e usa uma *função hash* $h : U \rightarrow [m]$ para calcular a posição de uma chave na tabela.

Como o tamanho da tabela hash é menor que o número de chaves possíveis, existem chaves c_1, c_2 com $h(c_1) = h(c_2)$, que geram *colisões*. Logo uma tabela hash precisa definir um método de *resolução de colisões*. Uma solução é *Hashing perfeito*: escolhe uma função hash, que para um dado conjunto de chaves não tem colisões. Isso é possível se o conjunto de chaves é conhecido e estático.

2.1. Hashing com listas encadeadas

Seja $h : U \rightarrow [m]$ uma função hash. Mantemos uma coleção de m listas $l_0, \dots, l_{m-1}$ tal que a lista l_i contém as chaves c com *valor hash* $h(c) = i$. Supondo que a avaliação de h é possível em $O(1)$, a inserção custa $O(1)$, e o teste é proporcional ao tamanho da lista.

Para obter uma distribuição razoável das chaves nas listas, supomos que h é uma função hash *simples* e *uniforme*:

$$\Pr(h(c) = i) = 1/m. \quad (2.1)$$

Seja $n_i := |l_i|$ o tamanho da lista i e c_{ji} a variável aleatória que indica se chave j pertence a lista i . Temos $\Pr(c_{ji} = 1) = \Pr(h(j) = i)$. Ainda $n_i =$

2. Tabelas hash

$\sum_{1 \leq j \leq n} c_{ji}$ e com isso

$$E[n_i] = E\left[\sum_{1 \leq j \leq n} c_{ji}\right] = \sum_{1 \leq j \leq n} E[c_{ji}] = \sum_{1 \leq j \leq n} \Pr(h(c_j) = i) = n/m.$$

O valor $\alpha := n/m$ é o *fator de ocupação* da tabela hash.

```

1  insert(c, H) :=
2      insert(c, lh(c))
3
4  lookup(c, H) :=
5      lookup(c, lh(c))
6
7  delete(c, H) :=
8      delete(c, lh(c))

```

Teorema 2.1

Uma busca sem sucesso precisa tempo esperado $\Theta(1 + \alpha)$.

Prova. A chave c tem a probabilidade $1/m$ de ter um valor hash i . O tamanho esperado da lista i é α . Uma busca sem sucesso nessa lista precisa tempo $\Theta(\alpha)$. Junto com a avaliação da função hash em $\Theta(1)$, obtemos tempo esperado total $\Theta(1 + \alpha)$. ■

Teorema 2.2

Uma busca com sucesso precisa tempo esperado $\Theta(1 + \alpha)$.

Prova. Supomos que a chave c é uma das chaves na tabela com probabilidade uniforme. Então, a probabilidade de pertencer a lista i (ter valor hash i) é n_i/n . Uma busca com sucesso toma tempo $\Theta(1)$ para avaliação da função hash, e mais um número de operações proporcional à posição p da chave na sua lista. Com isso obtemos tempo esperado $\Theta(1 + E[p])$.

Para determinar a posição esperada na lista, $E[p]$, seja $c_1, \dots, c_n$ a sequência na qual as chaves foram inseridas. Supondo que inserimos as chaves no início da lista, $E[p]$ é um mais que o número de chaves inseridas depois de c na mesma lista.

Seja X_{ij} um variável aleatória que indica se chaves c_i e c_j tem o mesmo valor hash. $E[X_{ij}] = \Pr(h(c_i) = h(c_j)) = \sum_{1 \leq k \leq m} \Pr(h(c_i) = k) \Pr(h(c_j) = k) = 1/m$. Seja p_i a posição da chave c_i na sua lista. Temos

$$E[p_i] = E\left[1 + \sum_{j: j > i} X_{ij}\right] = 1 + \sum_{j: j > i} E[X_{ij}] = 1 + (n - i)/m$$

e para uma chave aleatória c

$$\begin{aligned} E[p] &= \sum_{1 \leq i \leq n} 1/n E[p_i] = \sum_{1 \leq i \leq n} 1/n(1 + (n - i)/m) \\ &= 1 + n/m - (n + 1)/(2m) = 1 + \alpha/2 - \alpha/(2n). \end{aligned}$$

Portanto, o tempo esperado de uma busca com sucesso é

$$\Theta(1 + E[p]) = \Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha).$$

■

Seleção de uma função hash Para implementar uma tabela hash, temos que escolher uma função hash, que satisfaz (2.1). Para facilitar isso, supomos que o universo de chaves é um conjunto $U = [u]$ de números inteiros. (Para tratar outros tipos de chaves, costuma-se convertê-los para números inteiros.) Se cada chave ocorre com a mesma probabilidade, $h(i) = i \bmod m$ é uma função hash simples e uniforme. Essa abordagem é conhecida como *método de divisão*. O problema com essa função na prática é que não conhecemos a distribuição de chaves, e ela provavelmente não é uniforme. Por exemplo, se m é par, o valor hash de chaves pares é par, e de chaves ímpares é ímpar, e se $m = 2^k$ o valor hash consiste nos primeiros k bits. Uma escolha que funciona na prática é um número primo “suficientemente” distante de uma potência de 2.

O *método de multiplicação* define

$$h(c) = \lfloor m \{Ac\} \rfloor.$$

O método funciona para qualquer valor de m , mas depende de uma escolha adequada de $A \in \mathbb{R}$. Knuth propôs $A \approx (\sqrt{5} - 1)/2$.

Hashing universal Outra ideia: Para qualquer função hash h fixa, sempre existe um conjunto de chaves, tal que essa função hash gera muitas colisões. (Em particular, um “adversário” que conhece a função hash pode escolher chaves $c \in h^{-1}(i)$ para qualquer posição $i \in [m]$, tal que $h(c) = i$ é constante.) Para evitar isso podemos escolher uma função hash aleatória de uma família de funções hash.

Uma família $\mathcal{H}$ de funções hash $U \rightarrow [m]$ é *universal* se

$$|\{h \in \mathcal{H} \mid h(c_1) = h(c_2)\}| = |\mathcal{H}|/m$$

ou equivalente

$$\Pr(h(c_1) = h(c_2)) = 1/m$$

para qualquer par de chaves c_1, c_2 .

2. Tabelas hash

Teorema 2.3

Se escolhemos uma função hash $h \in \mathcal{H}$ uniformemente, para uma chave arbitrária c o tamanho esperado de $l_{h(c)}$ é

- α , caso $c \notin H$, e
- $1 + \alpha$, caso $c \in H$.

Prova. Para chaves c_1, c_2 seja $X_{ij} = [h(c_1) = h(c_2)]$ e temos

$$E[X_{ij}] = \Pr(X_{ij} = 1) = \Pr(h(c_1) = h(c_2)) = 1/m$$

pela universalidade de $\mathcal{H}$. Para uma chave fixa c seja Y_c o número de colisões.

$$E[Y_c] = E\left[\sum_{\substack{c' \in H \\ c' \neq c}} X_{cc'}\right] = \sum_{\substack{c' \in H \\ c' \neq c}} E[X_{cc'}] \leq \sum_{\substack{c' \in H \\ c' \neq c}} 1/m.$$

Para uma chave $c \notin H$, o tamanho da lista é Y_c , e portanto de tamanho esperado $E[Y_c] \leq n/m = \alpha$. Caso $c \in H$, o tamanho da lista é $1 + Y_c$ e com $E[Y_c] = (n - 1)/m$ esperadamente

$$1 + (n - 1)/m = 1 + \alpha - 1/m < 1 + \alpha.$$

■

Um exemplo de um conjunto de funções hash universais: Seja $c = (c_0, \dots, c_r)_m$ uma chave na base m , escolha $a = (a_0, \dots, a_r)_m$ aleatoriamente e define

$$h_a = \sum_{0 \leq i \leq r} c_i a_i \mod m.$$

Hashing perfeito Hashing é *perfeito* sem colisões. Isso podemos garantir somente caso conheçamos as chaves a serem inseridos na tabela. Para uma função aleatória de uma família universal de funções hash para uma tabela hash de tamanho m , o número esperado de colisões é $E[\sum_{i \neq j} X_{ij}] = \sum_{i \neq j} E[X_{ij}] \leq n^2/m$. Portanto, caso escolhamos uma tabela de tamanho $m > n^2$ o número esperado de colisões é menos que um. Em particular, para $m > cn^2$ com $c > 1$ a probabilidade de uma colisão é $\Pr(\sum_{i \neq j} X_{ij} \geq 1) \leq E[\sum_{i \neq j} X_{ij}] \leq n^2/m < 1/c$ onde a primeira desigualdade segue da desigualdade de Markov.

2.2. Hashing com endereçamento aberto

Uma abordagem para resolução de colisões, chamada *endereçamento aberto*, é escolher uma outra posição para armazenar uma chave, caso $h(c)$ é ocupada. Uma estratégia para conseguir isso é procurar uma posição livre numa permutação de todos índices restantes. Assim garantimos que um insert tem sucesso enquanto ainda existe uma posição livre na tabela. Uma função hash $h(c, i)$ com dois argumentos, tal que $h(c, 1), \dots, h(c, m)$ é uma permutação de $[m]$, representa essa estratégia.

```

1 insert(c, H) :=
2   for i in [m]
3     if H[h(c, i)] = free
4       H[h(c, i)] = c
5   return
6
7 lookup(c, H) :=
8   for i in [m]
9     if H[h(c, i)] = free
10      return false
11     if H[h(c, i)] = c
12      return true
13  return false

```

A função $h(c, i)$ é *uniforme*, se a probabilidade de uma chave randômica ter associada uma dada permutação é $1/m!$. A seguir supomos que h é uniforme.

Teorema 2.4

As funções lookup e insert precisam no máximo $1/(1 - \alpha)$ testes caso a chave não está na tabela.

Prova. Seja X o número de testes até encontrar uma posição livre. Temos

$$E[X] = \sum_{i \geq 1} i \Pr(X = i) = \sum_{i \geq 1} \sum_{j \geq i} \Pr(X = j) = \sum_{i \geq 1} \Pr(X \geq i).$$

Com T_i o evento que o teste i ocorre e a posição i é ocupada, podemos escrever

$$\Pr(X \geq i) = \Pr(T_1 \cap \dots \cap T_{i-1}) = \Pr(T_1) \Pr(T_2|T_1) \Pr(T_3|T_1, T_2) \dots \Pr(T_{i-1}|T_1, \dots, T_{i-2}).$$

Agora $\Pr(T_1) = n/m$, e como h é uniforme $\Pr(T_2|T_1) = (n-1)/(m-1)$ e em geral

$$\Pr(T_k|T_1, \dots, T_{k-1}) = (n-k+1)/(m-k+1) \leq n/m = \alpha.$$

2. Tabelas hash

Portanto $\Pr(X \geq i) \leq \alpha^{i-1}$ e

$$E[X] = \sum_{i \geq 1} \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = \sum_{i \geq 0} \alpha^i = 1/(1 - \alpha).$$

■

Lema 2.1

Para $i < j$, temos $H_j - H_i \leq \ln j - \ln i$.

Prova.

$$H_j - H_i \leq \int_i^j \frac{1}{x} dx = \ln j - \ln i.$$

■

Teorema 2.5

Caso $\alpha < 1$ a função lookup precisa esperadamente $1/\alpha \ln 1/(1 - \alpha)$ testes caso a chave esteja na tabela, e cada chave tem a mesma probabilidade de ser procurada.

Prova. Seja c a i -ésima chave inserida. No momento de inserção temos $\alpha = (i - 1)/m$ e o número esperado de testes T até encontrar a posição livre foi $1/(1 - (i - 1)/m) = m/(m - (i - 1))$, e portanto o número esperado de testes até encontrar uma chave arbitrária é

$$E[T] = 1/n \sum_{1 \leq i \leq n} m/(m - (i - 1)) = 1/\alpha \sum_{0 \leq i < n} 1/(m - i) = 1/\alpha (H_m - H_{m-n})$$

e com $H_m - H_{m-n} \leq \ln(m) - \ln(m - n)$ temos

$$E[T] = 1/\alpha (H_m - H_{m-n}) < 1/\alpha (\ln(m) - \ln(m - n)) = 1/\alpha \ln(1/(1 - \alpha)).$$

■

Remover elementos de uma tabela hash com endereçamento aberto é mais difícil, porque a busca para um elemento termina ao encontrar uma posição livre. Para garantir a corretude de lookup, temos que marcar posições como “removidas” e continuar a busca nessas posições. Infelizmente, nesse caso, as garantias da complexidade não mantêm-se – após uma série de deleções e inserções toda posição livre será marcada como “removida” tal que delete e lookup precisam n passos. Portanto o endereçamento aberto é favorável somente se temos poucas deleções.

Funções hash para endereçamento aberto

- Linear: $h(c, i) = h(c) + i \mod m$
- Quadrática: $h(c, i) = h(c) + c_1 i + c_2 i^2 \mod m$
- Hashing duplo: $h(c, i) = h_1(c) + i h_2(c) \mod m$

Nenhuma das funções é uniforme, mas o hashing duplo mostra um bom desempenho na prática.

2.3. Cuco hashing

Cuco hashing é outra abordagem que procura posições alternativas na tabela em caso de colisões, com o objetivo de garantir um tempo de acesso constante no pior caso. Para conseguir isso, usamos duas funções hash h_1 e h_2 , e inserimos uma chave em uma das duas posições $h_1(c)$ ou $h_2(c)$. Desta forma a busca e a deleção possuem complexidade constante $O(1)$:

```

1 lookup(c, H) :=
2   if H[h1(c)] = c or H[h2(c)] = c
3     return true
4   return false
5
6 delete(c, H) :=
7   if H[h1(c)] = c
8     H[h1(c)] := free
9   if H[h2(c)] = c
10    H[h2(c)] := free

```

Inserir uma chave é simples, caso uma das posições alternativas é livre. No caso contrário, a solução do cuco hashing é comportar-se como um cuco com ovos de outras aves que jogá-los fora do seu “ninho”: “insert” ocupa a posição de uma das duas chaves. A chave “jogada fora” será inserida novamente na tabela. Caso a posição alternativa dessa chave é livre, a inserção termina. Caso contrário, o processo se repete. Esse procedimento termina após uma série de reinserções ou entra num laço infinito. Nesse último caso temos que realocar todas chaves com novas funções hash.

```

1 insert(c, H) :=
2   if H[h1(c)] = c or H[h2(c)] = c
3     return
4   p := h1(c)
5   do n vezes

```

2. Tabelas hash

```

6      if  $H[p] = \text{free}$ 
7           $H[p] := c$ 
8          return
9      swap( $c, H[p]$ )
10     { escolhe a outra posição da chave atual }
11     if  $p = h_1(c)$ 
12          $p := h_2(c)$ 
13     else
14          $p := h_1(c)$ 
15     rehash( $H$ )
16     insert( $c, H$ )

```

Uma maneira de visualizar uma tabela hash com cuco hashing, é usar o *grafo cuco*: caso foram inseridas as chaves $c_1, \dots, c_n$ na tabela nas posições $p_1, \dots, p_n$, o grafo é $G = (V, A)$, com $V = [m]$ e $(p_i, h_2(c_i)) \in A$ caso $h_1(c_i) = p_i$ e $(p_i, h_1(c_i)) \in A$ caso $h_2(c_i) = p_i$, i.e., os arcos apontam para a posição alternativa. O grafo cuco é um grafo direcionado e eventualmente possui ciclos. Uma característica do grafo cuco é que uma posição p é eventualmente analisada na inserção de uma chave c somente se existe um caminho de $h_1(c)$ ou $h_2(c)$ para p . Para a análise é suficiente considerar o grafo cuco não-direcionado.

Exemplo 2.1

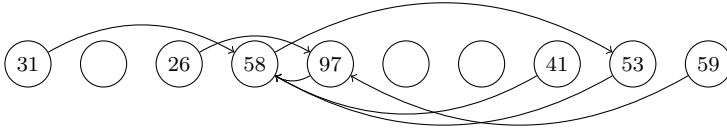
Para chaves de dois dígitos c_1c_2 seja $h_1(c) = 3c_1 + c_2 \pmod m$ e $h_2(c) = 4c_1 + c_2$. Para $m = 10$ obtemos para uma sequência aleatória de chaves

c	31	41	59	26	53	58	97
$h_1(c)$	0	3	4	2	8	3	4
$h_2(c)$	3	7	9	4	3	8	3

e a seguinte sequência de tabelas hash

0	1	2	3	4	5	6	7	8	9	
										Inicial
31										Inserção 31
31			41							Inserção 41
31			41	59						Inserção 59
31		26	41	59						Inserção 26
31		26	41	59				53		Inserção 53
31		26	58	59			41	53		Inserção 58
31		26	58	97			41	53	59	Inserção 59

O grafo cuco correspondente é



◇

Lema 2.2

Para posições i e j e um $c > 1$ tal que $m \geq 2cn$, a probabilidade de existir um caminho mínimo de i para j de comprimento $d \geq 1$ é no máximo c^{-d}/m .

Prova. Observe que a probabilidade de um item c ter posições i e j como alternativas é no máximo $\Pr(h_1(c) = i, h_2(c) = j) + \Pr(h_1(c) = j, h_2(c) = i) = 2/m^2$. Portanto a probabilidade de pelo menos uma das n chaves ter posições alternativas i e j é no máximo $2n/m^2 = c^{-1}/m$.

A prova do lema é por indução sobre d . Para $d = 1$ a afirmação está correta pela observação acima. Para $d > 1$ existe um caminho mínimo de comprimento $d - 1$ de i para um k . A probabilidade disso é no máximo $c^{-(d-1)}/m$ e a probabilidade de existir um elemento com posições alternativas k e j no máximo c^{-1}/m . Portanto, para um k fixo, a probabilidade de existir um caminho de comprimento d é no máximo c^{-d}/m^2 e considerando todas posições k possíveis no máximo c^{-d}/m . ■

Com isso a probabilidade de existir um caminho entre duas chaves i e j , é igual a probabilidade de existir um caminho começando em $h_1(i)$ ou $h_2(i)$ e terminando em $h_1(j)$ ou $h_2(j)$, que é no máximo $4 \sum_{i \geq 1} c^{-i}/m \leq 4/m(c - 1) = O(1/m)$. Logo o número esperado de itens visitados numa inserção é $4n/m(c - 1) = O(1)$, caso não seja necessário reconstruir a tabela hash.

2.4. Filtros de Bloom

Um filtro de Bloom armazena um conjunto de n chaves, com as seguintes restrições:

- Não é mais possível remover elementos.
- É possível que o teste de pertinência tem sucesso, sem o elemento fazer parte do conjunto (“false positive”).

Um filtro de Bloom consiste em m bits B_i , $1 \leq i \leq m$, e usa k funções hash $h_1, \dots, h_k$.

2. Tabelas hash

```
1  insert(c, B) :=
2    for i in 1...k
3      bhi(c) := 1
4    end for
5
6  lookup(c, B) :=
7    for i in 1...k
8      if bhi(c) = 0
9        return false
10   return true
```

Após inserir n chaves, um dado bit é ainda 0 com probabilidade

$$p' = \left(1 - \frac{1}{m}\right)^{kn} = \left(1 - \frac{kn/m}{kn}\right)^{kn} \approx e^{-kn/m}$$

que é igual ao valor esperado da fração de bits não setados¹. Sendo ρ a fração de bits não setados realmente, a probabilidade de erradamente classificar um elemento como membro do conjunto é

$$(1 - \rho)^k \approx (1 - p')^k \approx \left(1 - e^{-kn/m}\right)^k$$

porque ρ é com alta probabilidade perto do seu valor esperado (Broder e Mitzenmacher, 2003). Broder e Mitzenmacher (2003) também mostram que o número ótimo k de funções hash para dados valores de n, m é $m/n \ln 2$ e com isso temos um erro de classificação $\approx (1/2)^k$.

Aplicações:

1. Hifenação: Manter uma tabela de palavras com hifenação excepcional (que não pode ser determinado pelas regras).
2. Comunicação efetiva de conjuntos, p.ex. seleção em bancos de dados distribuídos. Para calcular um join de dois bancos de dados A, B , primeiramente A filtra os elementos, manda um filtro de Bloom S_A para B e depois B executa o join baseado em S_A . Para eliminação de eventuais elementos classificados erradamente, B manda os resultados para A e A filtra os elementos errados.

¹Lembrando que $e^x \geq (1 + x/n)^n$ para $n > 0$.

Tabela 2.1.: Complexidade das operações em tabelas hash. Complexidades em negrito são amortizados.

	insert	lookup	delete
Listas encadeadas	$\Theta(1)$	$\Theta(1 + \alpha)$	$\Theta(1 + \alpha)$
Endereçamento aberto	$O(1/(1 - \alpha))$	$O(1/(1 - \alpha))$	-
(com/sem sucesso)	$O(1/\alpha \ln 1/(1 - \alpha))$	$O(1/\alpha \ln 1/(1 - \alpha))$	-
Cuco	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

3. Algoritmos de aproximação

Para vários problemas não conhecemos um algoritmo eficiente. Para problemas NP-completos, em particular, uma solução eficiente é pouco provável. Um *algoritmo de aproximação* calcula uma solução aproximada para um problema de otimização. Diferente de uma heurística, o algoritmo *garante* a qualidade da aproximação no pior caso. Dado um problema e um algoritmo de aproximação A , escrevemos $A(x) = y$ para a solução aproximada da instância x , $\varphi(x, y)$ para o valor dessa solução, y^* para a solução ótima e $\text{OPT}(x) = \varphi(x, y^*)$ para o valor da solução ótima.

3.1. Problemas, classes e reduções

Definição 3.1

Um *problema de otimização* $\Pi = (\mathcal{P}, \varphi, \text{opt})$ é uma relação binária $\mathcal{P} \subseteq I \times S$ com instâncias $x \in I$ e soluções $y \in S$, junto com

- uma função de otimização (função de objetivo) $\varphi : \mathcal{P} \rightarrow \mathbb{N}$ (ou $\mathbb{Q}$).
- um objetivo: Encontrar mínimo ou máximo

$$\text{OPT}(x) = \text{opt}\{\varphi(x, y) \mid (x, y) \in \mathcal{P}\}$$

junto com uma solução y^* tal que $\varphi(x, y^*) = \text{OPT}(x)$.

O par $(x, y) \in \mathcal{P}$ caso y seja uma solução para x .

Uma instância x de um problema de otimização possui soluções $S(x) = \{y \mid (x, y) \in \mathcal{P}\}$.

Convenção 3.1

Escrevemos um problema de otimização na forma

NOME

Instância x

Solução y

3. Algoritmos de aproximação

Objetivo Minimiza ou maximiza $\varphi(x, y)$.

Com um dado problema de otimização correspondem três problemas:

- Construção: Dado x , encontra a solução ótima y^* e seu valor $\text{OPT}(x)$.
- Avaliação: Dado x , encontra valor ótimo $\text{OPT}(x)$.
- Decisão: Dado x e k , decide se $\text{OPT}(x) \geq k$ (maximização) ou $\text{OPT}(x) \leq k$ (minimização).

Definição 3.2

Uma relação binária R é *polinomialmente limitada* se

$$\exists p \in \text{poly} : \forall (x, y) \in R : |y| \leq p(|x|).$$

Definição 3.3 (Classes de complexidade)

A classe PO consiste dos problemas de otimização tal que existe um algoritmo polinomial A com $\varphi(x, A(x)) = \text{OPT}(x)$ para $x \in I$.

A classe NPO consiste dos problemas de otimização tal que

- (i) As instâncias $x \in I$ são reconhecíveis em tempo polinomial.
- (ii) A relação $\mathcal{P}$ é polinomialmente limitada.
- (iii) Para y arbitrário, polinomialmente limitado: $(x, y) \in \mathcal{P}$ é decidível em tempo polinomial.
- (iv) φ é computável em tempo polinomial.

Definição 3.4

Uma *redução preservando a aproximação* entre dois problemas de minimização Π_1 e Π_2 consiste num par de funções f e g (computáveis em tempo polinomial) tal que para instância x_1 de Π_1 , $x_2 := f(x_1)$ é instância de Π_2 com

$$\text{OPT}_{\Pi_2}(x_2) \leq \text{OPT}_{\Pi_1}(x_1) \quad (3.1)$$

e para uma solução y_2 de Π_2 temos uma solução $y_1 := g(x_1, y_2)$ de Π_1 com

$$\varphi_{\Pi_1}(x_1, y_1) \leq \varphi_{\Pi_2}(x_2, y_2) \quad (3.2)$$

Uma redução preservando a aproximação fornece uma α -aproximação para Π_1 dada uma α -aproximação para Π_2 , porque

$$\varphi_{\Pi_1}(x_1, y_1) \leq \varphi_{\Pi_2}(x_2, y_2) \leq \alpha \text{OPT}_{\Pi_2}(x_2) \leq \alpha \text{OPT}_{\Pi_1}(x_1).$$

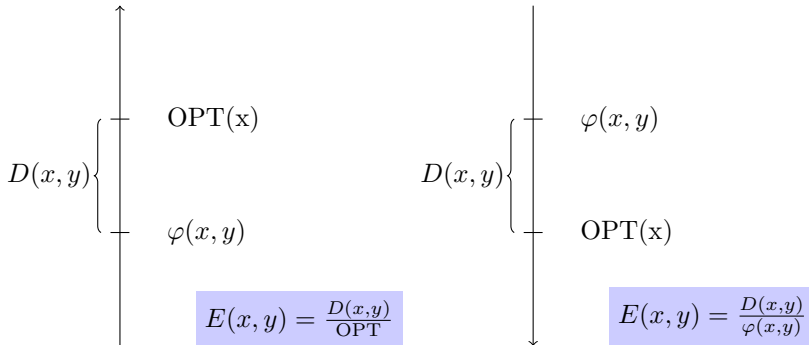
Observe que essa definição vale somente para problemas de minimização. A definição no caso de maximização é semelhante.

3.2. Medidas de qualidade

Uma *aproximação absoluta* garante que $D(x, y) = |\text{OPT}(x) - \varphi(x, y)| \leq D$ para uma constante D e todo x , enquanto uma *aproximação relativa* garante que o *erro relativo* $E(x, y) = D(x, y) / \max\{\text{OPT}(x), \varphi(x, y)\} \leq \epsilon \leq 1$ todos x . Um algoritmo que consegue um aproximação com constante ϵ também se chama ϵ -aproximativo. Tais algoritmos fornecem uma solução que difere no máximo um fator constante da solução ótima. A classe de problemas de otimização que permitem uma ϵ -aproximação em tempo polinomial para uma constante ϵ se chama APX.

Uma definição alternativa é a *taxa de aproximação* $R(x, y) = 1/(1 - E(x, y)) \geq 1$. Um algoritmo com taxa de aproximação r se chama r -aproximativo. (Não tem perigo de confusão com o erro relativo, porque $r \geq 1$.)

Aproximação relativa



Exemplo 3.1

Coloração de grafos planares e o problema de determinar a árvore geradora e a árvore Steiner de grau mínimo (Fürier e Raghavachari, 1994) permitem uma aproximação absoluta, mas não o problema da mochila.

Os problemas da mochila e do caixeiro viajante métrico permitem uma aproximação absoluta constante, mas não o problema do caixeiro viajante. $\diamond$

3.3. Técnicas de aproximação

3.3.1. Algoritmos gulosos

Cobertura de vértices

Algoritmo 3.1 (Cobertura de vértices)

Entrada Grafo não-direcionado $G = (V, E)$.

3. Algoritmos de aproximação

Saída Cobertura de vértices $C \subseteq V$.

```

1  VC-GV( $G$ ) :=
2    ( $C, G$ ) := Reduz( $G$ )
3    if  $V = \emptyset$  then
4      return  $C$ 
5    else
6      escolhe  $v \in V : \deg(v) = \Delta(G)$  { grau máximo }
7      return  $C \cup \{v\} \cup \text{VC-GV}(G - v)$ 
8    end if

```

Proposição 3.1

O algoritmo VC-GV é uma $O(\log |V|)$ -aproximação.

Prova. Seja G_i o grafo após iteração i e C^* uma cobertura ótima, i.e., $c := |C^*| = \text{OPT}(G)$.

A cobertura ótima C^* cobre todos G_i . Logo, a soma dos graus dos vértices em C^* (contando somente arestas em G_i) é pelo menos o número de arestas em G_i

$$\sum_{v \in C^*} \delta_{G_i}(v) \geq \|G_i\|,$$

e o grau médio dos vértices C^* em G_i satisfaz

$$\sum_{v \in C^*} \delta_{G_i}(v)/c \geq \|G_i\|/c.$$

Como o grau máximo do grafo é pelo menos o grau médio em C^* temos

$$\Delta(G_i) \geq \|G_i\|/c,$$

o que permite estimar

$$\sum_{0 \leq i < c} \Delta(G_i) \geq \sum_{0 \leq i < c} \|G_i\|/c \geq \sum_{0 \leq i < c} \|G_c\|/c = \|G_c\| = \|G\| - \sum_{0 \leq i < c} \Delta(G_i)$$

e logo

$$\sum_{0 \leq i < c} \Delta(G_i) \geq \|G\|/2,$$

i.e. o algoritmo remove em c iterações pelo menos a metade das arestas. Essa estimativa continua a ser válida, logo após

$$c \lceil \lg \|G\| \rceil \leq c \lceil 2 \log |G| \rceil = O(c \log |G|)$$

iterações não tem mais arestas. Como em cada iteração foi escolhido um vértice, a taxa de aproximação é $\log |G|$. ■

Algoritmo 3.2 (Cobertura de vértices)

Entrada Grafo não-direcionado $G = (V, E)$.

Saída Uma cobertura de vértices $C \subseteq V$.

```

1  VC-GE( $G$ ) :=
2    ( $C, G$ ) := Reduz( $G$ )
3    if  $E = \emptyset$  then
4      return  $C$ 
5    else
6      escolhe  $e = \{u, v\} \in E$ 
7      return  $C \cup \{u, v\} \cup \text{VC-GE}(G - \{u, v\})$ 
8    end if

```

Proposição 3.2

Algoritmo VC-GE é uma 2-aproximação para VC.

Prova. Cada cobertura C contém pelo menos um dos dois vértices escolhidos, logo temos $\phi_{\text{VC-GE}}(G) \leq 2|C|$, e no caso particular da solução ótima também $\phi_{\text{VC-GE}}(G) \leq 2\text{OPT}(G)$. ■

Algoritmo 3.3 (Cobertura de vértices)

Entrada Grafo não-direcionado $G = (V, E)$.

Saída Cobertura de vértices $C \subseteq V$.

```

1  VC-B( $G$ ) :=
2    ( $C, G$ ) := Reduz( $G$ )
3    if  $V = \emptyset$  then
4      return  $C$ 
5    else
6      escolhe  $v \in V : \deg(v) = \Delta(G)$  { grau máximo }
7       $C_1 := C \cup \{v\} \cup \text{VC-B}(G - v)$ 
8       $C_2 := C \cup N(v) \cup \text{VC-B}(G - v - N(v))$ 
9      if  $|C_1| < |C_2|$  then
10       return  $C_1$ 
11     else
12       return  $C_2$ 
13     end if
14   end if

```

Problema da mochila

KNAPSACK

Instância Um número n de itens com valores $v_i \in \mathbb{N}$ e tamanhos $t_i \in \mathbb{N}$, para $i \in [n]$, um limite M , tal que $t_i \leq M$ (todo item cabe na mochila).

Solução Uma seleção $S \subseteq [n]$ tal que $\sum_{i \in S} t_i \leq M$.

Objetivo Maximizar o valor total $\sum_{i \in S} v_i$.

Observação: O problema da mochila é NP-completo.

Como aproximar?

- Ideia: Ordene por v_i/t_i (“valor médio”) em ordem decrescente e enche o mochila o mais possível nessa ordem.

[[fragile=singleslide]Abordagem

```

1 K-G( $v_i, t_i$ ) :=
2   ordene os itens tal que  $v_i/t_i \geq v_j/t_j, \forall i < j$ .
3   for  $i \in X$  do
4     if  $t_i < M$  then
5        $S := S \cup \{i\}$ 
6        $M := M - t_i$ 
7     end if
8   end for
9   return  $S$ 

```

Aproximação boa?

- Considere

$$v_1 = 1, \dots, v_{n-1} = 1, v_n = M - 1$$

$$t_1 = 1, \dots, t_{n-1} = 1, t_n = M = kn \quad k \in \mathbb{N} \text{ arbitrário}$$

- Então:

$$v_1/t_1 = 1, \dots, v_{n-1}/t_{n-1} = 1, v_n/t_n = (M - 1)/M < 1$$

- K-G acha uma solução com valor $\varphi(x) = n - 1$, mas o ótimo é $\text{OPT}(x) = M - 1$.
- Taxa de aproximação:

$$\text{OPT}(x)/\varphi(x) = \frac{M - 1}{n - 1} = \frac{kn - 1}{n - 1} \geq \frac{kn - k}{n - 1} = k$$

- K-G não possui taxa de aproximação fixa!
- Problema: Não escolhemos o item com o maior valor.

[[fragile=singleslide]]Tentativa 2: Modificação

```

1 K-G'(v_i, t_i) :=
2   S_1 := K-G(v_i, t_i) // solução gulosa
3   v_1 := ∑_{i ∈ S_1} v_i
4   S_2 := {argmax_i v_i} // maior item
5   v_2 := ∑_{i ∈ S_2} v_i
6   retorna a maior das duas soluções

```

Aproximação boa?

- O algoritmo melhorou?
- Surpresa

Proposição 3.3

K-G' é uma 2-aproximação, i.e. $\text{OPT}(x) < 2\varphi_{\text{K-G}'}(x)$.

Prova. Seja j o primeiro item que K-G não coloca na mochila. Nesse ponto temos valor e tamanho

$$\bar{v}_j = \sum_{1 \leq i < j} v_i \leq \varphi_{\text{K-G}}(x) \quad (3.3)$$

$$\bar{t}_j = \sum_{1 \leq i < j} t_i \leq M \quad (3.4)$$

Afirmção: $\text{OPT}(x) < \bar{v}_j + v_j$. Nesse caso

(a) Seja $v_j \leq \bar{v}_j$.

$$\text{OPT}(x) < \bar{v}_j + v_j \leq 2\bar{v}_j \leq 2\varphi_{\text{K-G}}(x) \leq 2\varphi_{\text{K-G}'}$$

(b) Seja $v_j > \bar{v}_j$

$$\text{OPT}(x) < \bar{v}_j + v_j < 2v_j \leq 2v_{\max} \leq 2\varphi_{\text{K-G}'}$$

Prova da afirmação: No momento em que item j não cabe, temos espaço $M - \bar{t}_j < t_j$ sobrando. Como os itens são ordenados em ordem de densidade decrescente, obtemos um limite superior para a solução ótima preenchendo esse espaço com a densidade v_j/t_j :

$$\text{OPT}(x) \leq \bar{v}_j + (M - \bar{t}_j) \frac{v_j}{t_j} < \bar{v}_j + v_j.$$

■

3.3.2. Aproximações com randomização

Randomização

- Ideia: Permite escolhas randômicas (“joga uma moeda”)
- Objetivo: Algoritmos que decidem correta com probabilidade alta.
- Objetivo: Aproximações com *valor esperado* garantido.
- Minimização: $E[\varphi_A(x)] \leq 2\text{OPT}(x)$
- Maximização: $2E[\varphi_A(x)] \geq \text{OPT}(x)$

Randomização: Exemplo

SATISFATIBILIDADE MÁXIMA, MAXIMUM SAT

Instância Uma fórmula $\varphi \in \mathcal{L}(V)$ sobre variáveis $V = \{v_1, \dots, v_m\}$, $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_n$ em FNC.

Solução Uma atribuição de valores de verdade $a : V \rightarrow \mathbb{B}$.

Objetivo Maximiza o número de cláusulas satisfeitas

$$|\{C_i \mid \llbracket C_i \rrbracket_a = 1\}|.$$

[[fragile=singleslide]Nossa solução

```

1 SAT-R( $\varphi$ ) :=
2   seja  $\varphi = \varphi(v_1, \dots, v_k)$ 
3   for all  $i \in [1, k]$  do
4     escolhe  $v_i = 1$  com probabilidade  $1/2$ 
5   end for
```

Observação 3.1

A quantidade $\llbracket C \rrbracket_a$ é o valor da cláusula C na atribuição a .

◇

Aproximação?

- Surpresa: Algoritmo SAT-R é 2-aproximação.

Prova. O valor esperado de uma cláusula C com l variáveis é $E[\llbracket C \rrbracket] = \Pr(\llbracket C \rrbracket = 1) = 1 - 2^{-l} \geq 1/2$. Logo o valor esperado do número total $T = \sum_{i \in [n]} \llbracket C_i \rrbracket$ de cláusulas satisfeitas é

$$E[T] = E\left[\sum_{i \in [n]} \llbracket C_i \rrbracket\right] = \sum_{i \in [n]} E[\llbracket C_i \rrbracket] \geq n/2 \geq \text{OPT}/2$$

pela linearidade do valor esperado. ■

Outro exemplo Cobertura de vértices guloso e randomizado.

```

1 VC-RG( $G$ ) :=
2   seja  $\bar{w} := \sum_{v \in V} \deg(v)$ 
3    $C := \emptyset$ 
4   while  $E \neq \emptyset$  do
5     escolhe  $v \in V$  com probabilidade  $\deg(v)/\bar{w}$ 
6      $C := C \cup \{v\}$ 
7      $G := G - v$ 
8   end while
9   return  $C \cup V$ 

```

Resultado: $E[\phi_{\text{VC-RG}}(x)] \leq 2\text{OPT}(x)$.

3.3.3. Programação linear

Técnicas de programação linear são frequentemente usadas em algoritmos de aproximação. Entre eles são o *arredondamento randomizado* e *algoritmos primais-duais*.

Exemplo 3.2 (Arredondamento para cobertura por conjuntos)

Considere o problema de cobertura por conjuntos

$$\begin{aligned}
 &\text{minimiza} && \sum_{i \in [n]} w_i x_i, && (3.5) \\
 &\text{sujeito a} && \sum_{i \in [n] \mid u \in C_i} x_i \geq 1, && \forall u \in U, \\
 &&& x_i \in \{0, 1\}, && \forall i \in [n].
 \end{aligned}$$

Seja f_e a frequência de um elemento e , i.e. o número de conjuntos que contém e e f a maior frequência. Um algoritmo de arredondamento simples é dado por

Teorema 3.1

A seleção dos conjuntos com $x_i \geq 1/f$ na relaxação linear de (3.5) é uma f -aproximação do problema de cobertura de conjuntos.

Prova. Como $|\{i \in [n] \mid u \in C_i\}| \leq f$, temos $x_i \geq 1/f$ em média sobre esse conjunto. Logo existe, para cada $u \in U$ um conjunto com $x_i \geq 1/f$ que cobre u e a seleção é uma solução válida. O arredondamento aumenta o custo por no máximo um fator f , logo temos uma f -aproximação. ■ ◇

3.4. Esquemas de aproximação

Novas considerações

- Frequentemente uma r -aproximação não é suficiente. $r = 2$: 100% de erro!
- Existem aproximações melhores? p.ex. para SAT? problema da mochila?
- Desejável: Esquema de aproximação em tempo polinomial (EATP); polynomial time approximation scheme (PTAS)
 - Para cada entrada e taxa de aproximação r :
 - Retorne r -aproximação em tempo polinomial.

Um exemplo: Mochila máxima (Knapsack)

- Problema da mochila (veja página 122):
- Algoritmo MM-PD com programação dinâmica (pág. 187): tempo $O(n \sum_{i \in [n]} v_i)$.
- Desvantagem: Pseudo-polinomial.

Denotamos uma instância do problema da mochila com $I = (\{v_i\}, \{t_i\})$. Seja $r > 1$ uma qualidade de aproximação desejada.

```

1  MM-PTAS( $I, r$ ) :=
2     $v_{\max} := \max_i \{v_i\}$ 
3     $t := \lfloor \log_2 \frac{r-1}{r} v_{\max}/n \rfloor$ 
4     $v'_i := \lfloor v_i/2^t \rfloor$  para  $i = 1, \dots, n$ 
5    Define a nova instância  $I' = (\{v'_i\}, \{t_i\})$ 
6    return MM-PD( $I'$ )
    
```

Teorema 3.2

MM-PTAS é uma r -aproximação em tempo $O(rn^3/(r-1))$.

Prova. A complexidade da preparação nas linhas 1–3 é $O(n)$. A chamada para MM-PD custa

$$\begin{aligned}
 O\left(n \sum_{i \in [n]} v'_i\right) &= O\left(n \sum_{i \in [n]} \frac{v_i}{((r-1)/r)(v_{\max}/n)}\right) \\
 &= O\left(\frac{r}{r-1} n^2 \sum_{i \in [n]} v_i/v_{\max}\right) = O\left(\frac{r}{r-1} n^3\right).
 \end{aligned}$$

Seja $S = \text{MM-PTAS}(I)$ a solução obtida pelo algoritmo e S^* uma solução ótima.

$$\begin{aligned}
 \varphi_{\text{MM-PTAS}}(I, S) &= \sum_{i \in S} v_i \geq \sum_{i \in S} 2^t \lfloor v_i / 2^t \rfloor && \text{definição de } \lfloor \cdot \rfloor \\
 &\geq \sum_{i \in S^*} 2^t \lfloor v_i / 2^t \rfloor && \text{otimalidade de MM-PD sobre } v'_i \\
 &\geq \sum_{i \in S^*} v_i - 2^t && (\text{A.2}) \\
 &= \left(\sum_{i \in S^*} v_i \right) - 2^t |S^*| \\
 &\geq \text{OPT}(I) - 2^t n
 \end{aligned}$$

Portanto

$$\begin{aligned}
 \text{OPT}(I) &\leq \varphi_{\text{MM-PTAS}}(I, S) + 2^t n \leq \varphi_{\text{MM-PTAS}}(I, S) + \frac{\text{OPT}(I)}{v_{\max}} 2^t n \\
 \iff \text{OPT}(I) \left(1 - \frac{2^t n}{v_{\max}} \right) &\leq \varphi_{\text{MM-PTAS}}(I, S)
 \end{aligned}$$

e com $2^t n / v_{\max} \leq (r - 1) / r$

$$\iff \text{OPT}(I) \leq r \varphi_{\text{MM-PTAS}}(I, S).$$

■

Um EATP frequentemente não é suficiente para resolver um problema adequadamente. Por exemplo temos um EATP para

- o problema do caixeiro viajante euclidiano com complexidade $O(n^{3000/\epsilon})$ (Arora, 1996);
- o problema do mochila múltiplo com complexidade $O(n^{12(\log 1/\epsilon)/\epsilon^8})$ (Chekuri, Kanna, 2000);
- o problema do conjunto independente máximo em grafos com complexidade $O(n^{(4/\pi)(1/\epsilon^2+1)^2(1/\epsilon^2+2)^2})$ (Erlebach, 2001).

Para obter uma aproximação com 20% de erro, i.e. $\epsilon = 0.2$ obtemos algoritmos com complexidade $O(n^{15000})$, $O(n^{375000})$ e $O(n^{523804})$, respectivamente!

3. Algoritmos de aproximação

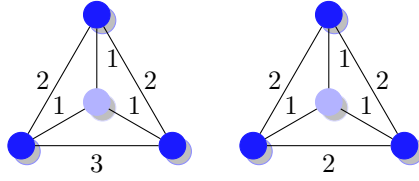


Figura 3.1.: Grafo com fecho métrico.

3.5. Aproximando o problema da árvore de Steiner mínima

Seja $G = (V, A)$ um grafo completo, não-direcionado com custos $c_a \geq 0$ nos arcos. O problema da árvore Steiner mínima (ASM) consiste em achar o subgrafo conexo mínimo que inclui um dado conjunto de *vértices necessários* ou *terminais* $R \subseteq V$. Esse subgrafo sempre é uma árvore (ex. 3.1). O conjunto $V \setminus R$ forma os *vértices Steiner*. Para um conjunto de arcos A , define o custo $c(A) = \sum_{a \in A} c_a$.

Observação 3.2

ASM é NP-completo. Para um conjunto fixo de vértices Steiner $V' \subseteq V \setminus R$, a melhor solução é a árvore geradora mínima sobre $R \cup V'$. Portanto a dificuldade é a seleção dos vértices Steiner da solução ótima. $\diamond$

Definição 3.5

Os custos são *métricos* se eles satisfazem a desigualdade triangular, i.e.

$$c_{ij} \leq c_{ik} + c_{kj}$$

para qualquer tripla de vértices i, j, k .

Teorema 3.3

Existe uma redução preservando a aproximação de ASM para a versão métrica do problema.

Prova. O fecho métrico de $G = (V, A)$ é um grafo G' completo sobre vértices e com custos $c'_{ij} := d_{ij}$, sendo d_{ij} o comprimento do menor caminho entre i e j em G . Evidentemente $c'_{ij} \leq c_{ij}$ e portanto (3.1) é satisfeita. Para ver que (3.2) é satisfeita, seja T' uma solução de ASM em G' . Define T como união de todos caminhos definidos pelos arcos em T' , menos um conjunto de arcos para remover eventuais ciclos. O custo de T é no máximo $c(T')$ porque o custo de todo caminho é no máximo o custo da aresta correspondente em T' . ■

Consequência: Para o problema do ASM é suficiente considerar o caso métrico.

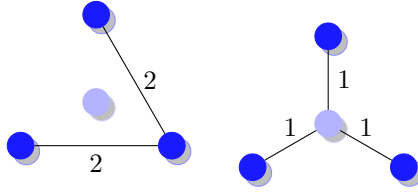


Figura 3.2.: AGM sobre R e melhor solução. ●: vértice em R , ●: vértice Steiner.

Teorema 3.4

O AGM sobre R é uma 2-aproximação para o problema do ASM.

Prova. Considere a solução ótima S^* de ASM. Duplica todas as arestas¹ tal que todo vértice possui grau par. Encontra um ciclo Euleriano nesse grafo. Remove vértices duplicados nesse caminho. O custo do caminho C obtido dessa forma não é mais que o dobro do custo original: o grafo com todas arestas custa $2c(S^*)$ e a remoção de vértices duplicados não aumenta esse custo, pela metricidade. Como esse caminho é uma árvore geradora, temos $c(A) \leq c(C) \leq 2c(S^*)$ para AGM A . ■

3.6. Aproximando o PCV

Teorema 3.5

Para qualquer função $\alpha(n)$ computável em tempo polinomial o PCV não possui $\alpha(n)$ -aproximação em tempo polinomial, caso $P \neq NP$.

Prova. Via redução de HC para PCV. Para uma instância $G = (V, A)$ de HC define um grafo completo G' com

$$c_a = \begin{cases} 1, & a \in A, \\ \alpha(n)n, & \text{caso contrário.} \end{cases}$$

Se G possui um ciclo Hamiltoniano, então o custo da menor rota é n . Caso contrário qualquer rota usa ao menos uma aresta de custo $\alpha(n)n$ e portanto o custo total é $\geq \alpha(n)n$. Portanto, dada uma $\alpha(n)$ -aproximação de PCV podemos decidir HC em tempo polinomial. ■

Caso métrico No caso métrico podemos obter uma aproximação melhor. Determina uma rota como segue:

¹Isso transforma G num multigrafo.

3. Algoritmos de aproximação

1. Determina uma AGM A de G .
2. Duplica todas as arestas de A .
3. Acha um ciclo Euleriano nesse grafo.
4. Remove vértices duplicados.

Teorema 3.6

O algoritmo acima define uma 2-aproximação.

Prova. A melhor solução do PCV menos uma aresta é uma árvore geradora de G . Portanto $c(A) \leq \text{OPT}$. A solução S obtida pelo algoritmo acima satisfaz $c(S) \leq 2c(A)$ e portanto $c(S) \leq 2\text{OPT}$, pelo mesmo argumento da prova do teorema 3.4. ■

O fator 2 dessa aproximação é resultado do passo 2 que duplica todas as arestas para garantir a existência de um ciclo Euleriano. Isso pode ser garantido mais barato: A AGM A possui um número par de vértices com grau ímpar (ver exercício 3.2), e portanto podemos calcular um emparelhamento perfeito mínimo E entre esses vértices. O grafo com arestas $A \cup E$ possui somente vértices com grau par e portanto podemos aplicar os restantes passos nesse grafo.

Teorema 3.7 (Cristofides)

O algoritmo usando um emparelhamento perfeito mínimo no passo 2 é uma 3/2-aproximação.

Prova. O valor do emparelhamento E não é mais que $\text{OPT}/2$: remove vértices não emparelhados em E da solução ótima do PCV. O ciclo obtido dessa forma é a união de dois emparelhamentos perfeitos E_1 e E_2 formados pelas arestas pares ou ímpares no ciclo. Com E_1 o emparelhamento de menor custo, temos

$$c(E) \leq c(E_1) \leq (c(E_1) + c(E_2))/2 = \text{OPT}/2$$

e portanto

$$c(S) = c(A) + c(E) \leq \text{OPT} + \text{OPT}/2 = 3/2\text{OPT}.$$

■

3.7. Aproximando problemas de cortes

Seja $G = (V, A, c)$ um grafo conectado com pesos c nas arestas. Lembramos que um corte C é um conjunto de arestas que separa o grafo em duas partes

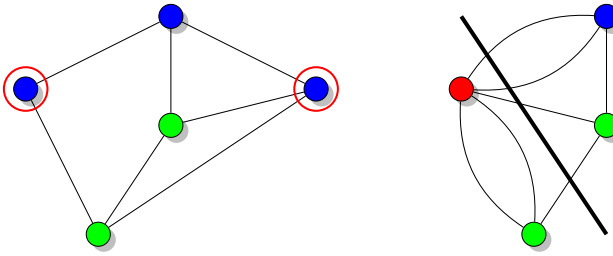


Figura 3.3.: Identificação de dois terminais e um corte no grafo reduzido. Vértices em verde, terminais em azul. O grafo reduzido possui múltiplas arestas entre vértices.

$S \cup V \setminus S$. Dado dois vértices $s, t \in V$, o problema de achar um corte mínimo que separa s e t pode ser resolvido via fluxo máximo em tempo polinomial. Generalizações desse problema são:

- Corte múltiplo mínimo (CMM): Dado terminais $s_1, \dots, s_k$ determine o menor corte C que separa todos.
- k -corte mínimo (k -CM): Mesmo problema, sem terminais definidos. (Observe que todos k componentes devem ser não vazios).

Fato 3.1

CMM é NP-difícil para qualquer $k \geq 3$. k -CM possui uma solução polinomial em tempo $O(n^{k^2})$ para qualquer k , mas é NP-difícil, caso k faz parte da entrada (Goldschmidt e Hochbaum, 1988).

Solução de CMM Chamamos um corte que separa um vértice dos outros um *corte isolante*. Ideia: A união de cortes isolantes para todo s_i é um corte múltiplo. Para calcular o corte isolante para um dado terminal s_i , identificamos os restantes terminais em um único vértice S e calculamos um corte mínimo entre s_i e S . (Na identificação de vértices temos que remover self-loops, e somar os pesos de múltiplas arestas.)

Isso leva ao algoritmo

Algoritmo 3.4 (CI)

Entrada Grafo $G = (V, A, c)$ e terminais $s_1, \dots, s_k$.

Saída Um corte múltiplo que separa os s_i .

1 Para cada $i \in [1, k]$: Calcula o corte isolante C_i de

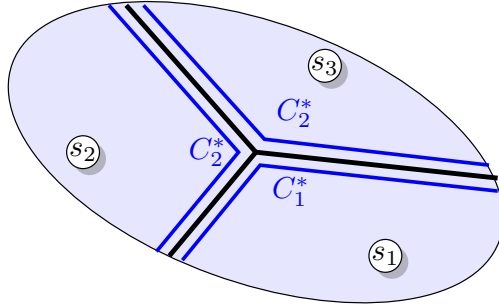


Figura 3.4.: Corte múltiplo e decomposição em cortes isolantes.

s_i .

2 Remove o maior desses cortes e retorne a união dos restantes.

Teorema 3.8

Algoritmo 3.4 é uma $2 - 2/k$ -aproximação.

Prova. Considere o corte mínimo C^* . De acordo com a Fig. 3.4 ele pode ser representado pela união de k cortes que separam os k componentes individualmente:

$$C^* = \bigcup_{i \in [k]} C_i^*.$$

Cada aresta de C^* faz parte dos cortes das duas componentes adjacentes, e portanto

$$\sum_{i \in [k]} w(C_i^*) = 2w(C^*)$$

e ainda $w(C_i) \leq w(C_i^*)$ para os cortes C_i do algoritmo 3.4, porque usamos o corte isolante mínimo de cada componente. Logo, para o corte C retornado pelo algoritmo temos

$$w(C) \leq (1 - 1/k) \sum_{i \in [k]} w(C_i) \leq (1 - 1/k) \sum_{i \in [k]} w(C_i^*) \leq 2(1 - 1/k)w(C^*).$$

■

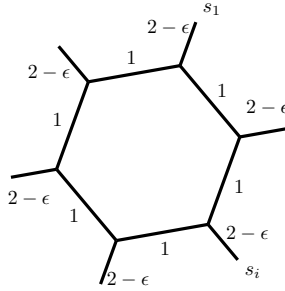


Figura 3.5.: Exemplo de um grafo em que o algoritmo 3.4 retorna uma $2 - 2/k$ -aproximação.

A análise do algoritmo é ótimo, como o exemplo da Fig. 3.5 mostra. O menor corte que separa s_i tem peso $2 - \epsilon$, portanto o algoritmo retorna um corte de peso $(2 - \epsilon)k - (2 - \epsilon) = (k - 1)(2 - \epsilon)$, enquanto o menor corte que separa todos terminais é o ciclo interno de peso k .

Solução de k -CM Problema: Como saber onde cortar?

Fato 3.2

Existem somente $n - 1$ cortes diferentes num grafo. Eles podem ser organizados numa árvore de *Gomory-Hu* (AGH) $T = (V, T)$. Cada aresta dessa árvore define um corte associado em G pelos dois componentes após a sua remoção.

1. Para cada $u, v \in V$ o menor corte $u-v$ em G é igual ao menor corte $u-v$ em T (i.e. a aresta de menor peso no caminho único entre u e v em T).
2. Para cada aresta $a \in T$, $w'(a)$ é igual a valor do corte associado.

Por consequência, a AGH codifica o valor de todos cortes em G .

Ele pode ser calculado determinando $n - 1$ cortes $s-t$ mínimos:

1. Define um grafo com um único vértice que representa todos vértices do grafo original. Chama um vértice que representa mais que um vértice do grafo original *gordo*.
2. Enquanto existem vértices gordos:
 - a) Escolhe um vértice gordo e dois vértices do grafo original que ele representa.
 - b) Calcula um corte mínimo entre esses vértices.

3. Algoritmos de aproximação

c) Separa o vértice gordo de acordo com o corte mínimo encontrado.

Observação: A união dos cortes definidos por $k - 1$ arestas na AGH separa G em pelo menos k componentes. Isso leva ao seguinte algoritmo.

Algoritmo 3.5 (KCM)

Entrada Grafo $G = (V, A, c)$.

Saída Um k -corte.

- 1 Calcula uma AGH T em G .
- 2 Forma a união dos $k - 1$ cortes mais leves definidos por $k - 1$ arestas em T .

Teorema 3.9

Algoritmo 3.5 é uma $2 - 2/k$ -aproximação.

Prova. Seja $C^* = \bigcup_{i \in [k]} C_i^*$ um corte mínimo, decomposto igual à prova anterior. O nosso objetivo é demonstrar que existem $k - 1$ cortes definidos por uma aresta em T que são mais leves que os C_i^* .

Removendo C^* de G gera componentes $V_1, \dots, V_k$: Defina um grafo sobre esses componentes contraindo os vértices de uma componente, com arcos da AGH T entre os componentes, e eventualmente removendo arcos até obter uma nova árvore T' . Seja C_k^* o corte de maior peso, e defina V_k como raiz da árvore. Desta forma, cada componente $V_1, \dots, V_{k-1}$ possui uma aresta associada na direção da raiz. Para cada dessas arestas (u, v) temos

$$w(C_i^*) \geq w'(u, v)$$

porque C_i^* isola o componente V_i do resto do grafo (particularmente separa u e v), e $w'(u, v)$ é o peso do menor corte que separa u e v . Logo

$$w(C) \leq \sum_{a \in T'} w'(a) \leq \sum_{1 \leq i < k} w(C_i^*) \leq (1 - 1/k) \sum_{i \in [k]} w(C_i^*) = 2(1 - 1/k)w(C^*).$$

■

3.8. Aproximando empacotamento unidimensional

Dado n itens com tamanhos $s_i \in \mathbb{Z}_+$, $i \in [n]$ e contêineres de capacidade $S \in \mathbb{Z}_+$ o problema do *empacotamento unidimensional* é encontrar o menor número de contêineres em que os itens podem ser empacotados.

EMPACOTAMENTO UNIDIMENSIONAL (MIN-EU) (BIN PACKING)

Entrada Um conjunto de n itens com tamanhos $s_i \in \mathbb{Z}_+$, $i \in [n]$ e o tamanho de um contêiner S .

Solução Uma partição de $[n] = C_1 \cup \dots \cup C_m$ tal que $\sum_{i \in C_k} s_i \leq S$ para $k \in [m]$.

Objetivo Minimiza o número de partes (“contêineres”) m .

A versão de decisão do empacotamento unidimensional (EU) pede decidir se os itens cabem em m contêineres.

Fato 3.3

EU é fortemente NP-completo.

Proposição 3.4

Para um tamanho S fixo EU pode ser resolvido em tempo $O(n^{S^S})$.

Prova. Podemos supor, sem perda de generalidade, que os itens possuem tamanhos $1, 2, \dots, S-1$. Um padrão de alocação de um contêiner pode ser descrito por uma tupla $(t_1, \dots, t_{S-1})$ sendo t_i o número de itens de tamanho i . Seja T o conjunto de todos padrões que cabem num contêiner. Como $0 \leq t_i \leq S$ o número total de padrões T é menor que $(S+1)^{S-1} = O(S^S)$. Uma ocupação de m contêineres pode ser descrito por uma tupla $(n_1, \dots, n_T)$ com n_i sendo o número de contêineres que usam padrão i . O número de contêineres é no máximo n , logo $0 \leq n_i \leq n$ e o número de alocações diferentes é no máximo $(n+1)^T = O(n^T)$. Logo podemos enumerar todas possibilidades em tempo polinomial. ■

Proposição 3.5

Para um m fixo, EU pode ser resolvido em tempo pseudo-polinomial.

Prova. Seja $B(S_1, \dots, S_m, i) \in \{\text{falso}, \text{verdadeiro}\}$ a resposta se itens $i, i+1, \dots, n$ cabem em m contêineres com capacidades $S_1, \dots, S_m$. B satisfaz

$$B(S_1, \dots, S_m, i) = \begin{cases} \bigvee_{\substack{1 \leq j \leq m \\ s_i \leq S_j}} B(S_1, \dots, S_j - s_j, \dots, S_m, i+1), & i \leq n, \\ \text{verdadeiro}, & i > n, \end{cases}$$

e $B(S, \dots, S, 1)$ é a solução do EU². A tabela B possui no máximo $n(S+1)^m$ entradas, cada uma computável em tempo $O(m)$, logo o tempo total é no máximo $O(mn(S+1)^m)$. ■

²Observe que a disjunção vazia é falsa.

3. Algoritmos de aproximação

Observação 3.3

Com um fator adicional de $O(\log m)$ podemos resolver também MIN-EU, procurando o menor i tal que $B(\underbrace{S, \dots, S}_{i \text{ vezes}}, 0, \dots, 0, n)$ é verdadeiro. $\diamond$

A proposição 3.4 pode ser melhorada usando programação dinâmica.

Proposição 3.6

Para um número fixo k de tamanhos diferentes, min-EU pode ser resolvido em tempo $O(n^{2k})$.

Prova. Seja $B(i_1, \dots, i_k)$ o menor número de contêineres necessário para empacotar i_j itens do j -ésimo tamanho e T o conjunto de todas as padrões de alocação de um contêiner. B satisfaz

$$B(i_1, \dots, i_k) = \begin{cases} 1 + \min_{\substack{t \in T \\ t \leq i}} B(i_1 - t_1, \dots, i_k - t_k), & \text{caso } (i_1, \dots, i_k) \notin T, \\ 1, & \text{caso contrário,} \end{cases}$$

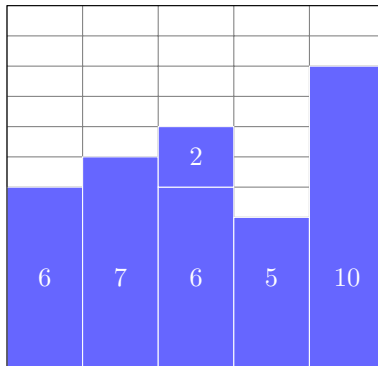
e $B(n_1, \dots, n_k)$ é a solução do EU, com n_i o número de itens de tamanho i na entrada. A tabela B tem no máximo n^k entradas. Como o número de itens em cada padrão de alocação é no máximo n , temos $|T| \leq n^k$ e logo o tempo total para preencher B é no máximo $O(n^{2k})$. $\blacksquare$

Corolário 3.1

Para um tamanho S fixo min-EU pode ser resolvido em tempo $O(n^{2S})$.

Abordagem prática?

- Ideia simples: Próximo que cabe (PrC).
- Por exemplo: Itens 6, 7, 6, 2, 5, 10 com limite 12.



Aproximação?

- Interessante: PrC é 2-aproximação.
- Observação: PrC é um algoritmo on-line.

Prova. Seja B o número de contêineres usados, $V = \sum_{i \in [n]} s_i$. Como dois contêineres consecutivos contêm uma soma > 1 , temos $\lfloor B/2 \rfloor < V$ e com $B/2 - 1/2 \leq \lfloor B/2 \rfloor$ ainda $B - 1 < 2V$ ou $B \leq 2V$. Mas precisamos pelo menos $\lceil V \rceil$ contêineres, logo $\lceil V \rceil \leq \text{OPT}(x)$. Portanto, $\varphi_{\text{PrC}}(x) \leq 2V \leq 2 \lceil V \rceil \leq 2\text{OPT}(x)$. ■

Aproximação melhor?

- Isso é a melhor estimativa possível para este algoritmo!
- Considere os $4n$ itens

$$\underbrace{1/2, 1/2n, 1/2, 1/2n, \dots, 1/2, 1/2n}_{2n \text{ vezes}}$$

- O que faz PrC? $\varphi_{\text{PrC}}(x) = 2n$: contêineres com

$1/(2n)$	$1/(2n)$	$1/(2n)$	$1/(2n)$		$1/(2n)$	$1/(2n)$
$1/2$	$1/2$	$1/2$	$1/2$	...	$1/2$	$1/2$

- Ótimo: n contêineres com dois elementos de $1/2$ + um com $2n$ elementos de $1/2n$. $\text{OPT}(x) = n = 1$.

$1/2$	$1/2$	$1/2$	$1/2$		$1/2$	$1/2$	$1/(2n)$
							$1/(2n)$
							$\vdots$
$1/2$	$1/2$	$1/2$	$1/2$	...	$1/2$	$1/2$	$1/(2n)$
							$1/(2n)$
							$1/(2n)$
							$1/(2n)$

- Portanto: Assintoticamente a taxa de aproximação 2 é estrito.

Melhores estratégias

- Primeiro que cabe (PiC), on-line, com “estoque” na memória

3. Algoritmos de aproximação

- Primeiro que cabe em ordem decrescente: PiCD, off-line.
- Taxa de aproximação?

$$\varphi_{\text{PiC}}(x) \leq \lceil 1.7\text{OPT}(x) \rceil$$

$$\varphi_{\text{PiCD}}(x) \leq 1.5\text{OPT}(x) + 1$$

Prova. (Da segunda taxa de aproximação.) Considere a partição $A \cup B \cup C \cup D = \{v_1, \dots, v_n\}$ com

$$A = \{v_i \mid v_i > 2/3\}$$

$$B = \{v_i \mid 2/3 \geq v_i > 1/2\}$$

$$C = \{v_i \mid 1/2 \geq v_i > 1/3\}$$

$$D = \{v_i \mid 1/3 \geq v_i\}$$

PiCD primeiro vai abrir $|A|$ contêineres com os itens do tipo A e depois $|B|$ contêineres com os itens do tipo B . Temos que analisar o que acontece com os itens em C e D .

Supondo que um contêiner contém somente itens do tipo D , os outros contêineres têm espaço livre menos que $1/3$, senão seria possível distribuir os itens do tipo D para outros contêineres. Portanto, nesse caso

$$B \leq \left\lceil \frac{V}{2/3} \right\rceil \leq 3/2V + 1 \leq 3/2\text{OPT}(x) + 1.$$

Caso contrário (nenhum contêiner contém somente itens tipo D), PiCD encontra a solução ótima. Isso pode ser justificado pelas seguintes observações:

- 1) O número de contêineres sem itens tipo D é o mesmo (eles são os últimos distribuídos em não abrem um novo contêiner). Logo é suficiente mostrar

$$\varphi_{\text{PiCD}}(x \setminus D) = \text{OPT}(x \setminus D).$$

- 2) Os itens tipo A não importam: Sem itens D , nenhum outro item cabe junto com um item do tipo A . Logo:

$$\varphi_{\text{PiCD}}(x \setminus D) = |A| + \varphi_{\text{PiCD}}(x \setminus (A \cup D)).$$

- 3) O melhor caso para os restantes itens são *pares* de elementos em B e C : Nessa situação, PiCD encontra a solução ótima.

Garantia ou aproximação melhor?

■

- Johnson (1973, Tese de doutorado)

$$\varphi_{\text{PiCD}}(x) \leq 11/9 \text{OPT}(x) + 4$$

- Baker (1985)

$$\varphi_{\text{PiCD}}(x) \leq 11/9 \text{OPT}(x) + 3$$

- Uma variante de PiCD (Johnson e Garey, 1985):

$$\varphi_{\text{PiCDM}}(x) \leq 71/60 \text{OPT}(x) + 31/6$$

3.8.1. Um esquema de aproximação assintótico para min-EU

Duas ideias permitem aproximar min-EU em $(1+\epsilon)\text{OPT}(I)+1$ para $\epsilon \in (0, 1]$.

Ideia 1: Arredondamento Para uma instância I , define uma instância R arredondada como segue:

1. Ordene os itens de forma não-decrescente e forme grupos de k itens.
2. Substitua o tamanho de cada item pelo tamanho do maior elemento no seu grupo.

Lema 3.1

Para uma instância I e a instância R arredondada temos

$$\text{OPT}(R) \leq \text{OPT}(I) + k$$

Prova. Suponha que temos uma solução ótima para I . Os itens do i -ésimo grupo de R cabem nos lugares dos itens do $i+1$ -ésimo grupo dessa solução. Para o último grupo de R temos que abrir no máximo k contêineres. ■

Ideia 2: Descartando itens menores

Lema 3.2

Suponha que temos um empacotamento para itens de tamanho maior que s_0 em B contêineres. Então existe um empacotamento de todos itens com no máximo

$$\max\left\{B, \sum_{i \in [n]} s_i / (S - s_0) + 1\right\}$$

contêineres.

3. Algoritmos de aproximação

Prova. Empacota os itens menores gulosamente no primeiro contêiner com espaço suficiente. Sem abrir um novo contêiner o limite é obviamente correto. Caso contrário, suponha que precisamos B' contêineres. $B' - 1$ contêineres contêm itens de tamanho total mais que $S - s_0$. A ocupação total W deles tem que ser menor que o tamanho total dos itens, logo

$$(B' - 1)(S - s_0) \leq W \leq \sum_{i \in [n]} s_i.$$

■

Juntando as ideias

Teorema 3.10

Para $\epsilon \in (0, 1]$ podemos encontrar um empacotamento usando no máximo $(1 + \epsilon)\text{OPT}(I) + 1$ contêineres em tempo $O(n^{16/\epsilon^2})$.

Prova. O algoritmo tem dois passos:

1. Empacota todos itens de tamanho maior que $s_0 = \lceil \epsilon/2 S \rceil$ usando arredondamento.
2. Empacota os itens menores depois.

Seja I' a instância com os $n' \leq n$ itens maiores. No primeiro passo, formamos grupos com $\lfloor n'\epsilon^2/4 \rfloor$ itens. Isso resulta em no máximo

$$\frac{n'}{\lfloor n'\epsilon^2/4 \rfloor} \leq \frac{2n'}{n'\epsilon^2/4} = \frac{8}{\epsilon^2}$$

grupos. (A primeira desigualdade usa $\lfloor x \rfloor \geq x/2$ para $x \geq 1$. Podemos supor que $n'\epsilon^2/4 \geq 1$, i.e. $n' \geq 4/\epsilon^2$. Caso contrário podemos empacotar os itens em tempo constante usando a proposição 3.6.)

Arredondando essa instância de acordo com lema 3.1 podemos obter uma solução em tempo $O(n^{16/\epsilon^2})$ pela proposição 3.6. Sabemos que $\text{OPT}(I') \geq n' \lceil \epsilon/2 S \rceil / S \geq n'\epsilon/2$. Logo temos uma solução com no máximo

$$\text{OPT}(I') + \lfloor n'\epsilon^2/4 \rfloor \leq \text{OPT}(I') + n'\epsilon^2/4 \leq (1 + \epsilon/2)\text{OPT}(I') \leq (1 + \epsilon/2)\text{OPT}(I)$$

contêineres.

O segundo passo, pelo lema 3.2, produz um empacotamento com no máximo

$$\max \left\{ (1 + \epsilon/2)\text{OPT}(I), \sum_{i \in [n]} s_i / (S - s_0) + 1 \right\}$$

contêineres, mas

$$\frac{\sum_{i \in [n]} s_i}{S - s_0} \leq \frac{\sum_{i \in [n]} s_i}{S(1 - \epsilon/2)} \leq \frac{\text{OPT}(I)}{1 - \epsilon/2} \leq (1 + \epsilon)\text{OPT}(I).$$

■

3.9. Aproximando problemas de sequenciamento

Problemas de sequenciamento recebem nomes da forma

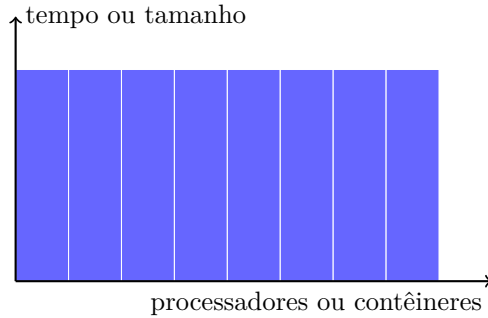
$$\alpha \mid \beta \mid \gamma$$

com campos

Máquina α	
1	Um processador
P	Processadores paralelos
Q	Processadores relacionados
R	Processadores arbitrários
Restrições β	
D_i	Prazo máximo (deadline)
d_i	Prazo previsto (due dates)
r_i	Tempo de liberação (release time)
$p_i = p$	Tempo uniforme p
prec	Precedências
Função objetivo γ	
$C_{\max}$	Maior tempo de término (maximum completion time)
$\sum_i C_i$	Tempo de término total (total completion time)
L_i	Atraso (lateness) $C_i - d_i$
T_i	Tardiness $\max\{L_i, 0\}$

Relação com empacotamento unidimensional:

3. Algoritmos de aproximação



- Empacotamento unidimensional: Dado $C_{\max}$ minimiza o número de processadores.
- $P \parallel C_{\max}$: Dado um número de contêineres, minimiza o tamanho dos contêineres.

SEQUENCIAMENTO EM PROCESSORES PARALELOS ($P \parallel C_{\max}$)

Entrada O número m de processadores e n tarefas com tempo de execução p_i , $i \in [n]$.

Solução Um *sequenciamento*, definido por uma alocação $M_1 \dot{\cup} \dots \dot{\cup} M_m = [n]$ das tarefas às máquinas.

Objetivo Minimizar o *makespan* (tempo de término) $C_{\max} = \max_{j \in [m]} C_j$, com $C_j = \sum_{i \in M_j} p_i$ o tempo de término da máquina j .

Fato 3.4

O problema $P \parallel C_{\max}$ é fortemente NP-completo.

Um limite inferior para $C_{\max}^* = \text{OPT}$ é

$$\text{LB} = \max \left\{ \max_{i \in [n]} p_i, \sum_{i \in [n]} p_i / m \right\}.$$

Uma classe de algoritmos gulosos para este problema são os algoritmos de *sequenciamento em lista* (inglês: list scheduling). Eles processam as tarefas em alguma ordem, e alocam a tarefa atual sempre à máquina de menor tempo de término atual.

Proposição 3.7

Sequenciamento em lista com ordem arbitrária permite uma $2-1/m$ -aproximação em tempo $O(n \log n)$.

Prova. Seja $C_{\max}$ o resultado do sequenciamento em lista. Considere uma máquina com tempo de término $C_{\max}$. Seja j a última tarefa alocada nessa máquina e C o término da máquina antes de alocar tarefa j . Logo,

$$\begin{aligned} C_{\max} = C + p_j &\leq \sum_{i \in [j-1]} p_i/m + p_j \leq \sum_{i \in [n]} p_i/m - p_j/m + p_j \\ &\leq \text{LB} + (1 - 1/m)\text{LB} = (2 - 1/m)\text{LB} \leq (2 - 1/m)C_{\max}^*. \end{aligned}$$

A primeira desigualdade é correta, porque alocando tarefa j a máquina tem tempo de término mínimo. Usando uma fila de prioridade a máquina com o menor tempo de término pode ser encontrada em tempo $O(\log n)$. ■

Observação 3.4

Pela prova da proposição 3.7 temos

$$\text{LB} \leq C_{\max}^* \leq 2\text{LB}.$$

◇

O que podemos ganhar com algoritmos off-line? Uma abordagem é ordenar as tarefas por tempo de execução não-crescente e aplicar o algoritmo guloso. Essa abordagem é chamada LPT (largest processing time).

Proposição 3.8

LPT é uma $4/3 - m/3$ -aproximação em tempo $O(n \log n)$.

Prova. Seja $p_1 \geq p_2 \geq \dots \geq p_n$ e suponha que esse é o menor contra-exemplo em que o algoritmo retorne $C_{\max} > (4/3 - m/3)C_{\max}^*$. Não é possível que a alocação do item $j < n$ resulta numa máquina com tempo de término $C_{\max}$, porque $p_1, \dots, p_j$ seria um contra-exemplo menor (mesmo $C_{\max}$, menor $C_{\max}^*$). Logo a alocação de p_n define o resultado $C_{\max}$.

Caso $p_n \leq C_{\max}^*/3$ pela prova da proposição 3.7 temos $C_{\max} \leq (4/3 - m/3)C_{\max}^*$, uma contradição. Mas caso $p_n > C_{\max}^*/3$ todas as tarefas possuem tempo de execução pelo menos $C_{\max}^*/3$ e no máximo duas podem ser executadas em cada máquina. Logo $C_{\max} \leq 2/3 C_{\max}^*$, outra contradição. ■

3.9.1. Um esquema de aproximação para $P \parallel C_{\max}$

Pela observação 3.4 podemos reduzir o $P \parallel C_{\max}$ para o empacotamento unidimensional via uma busca binária no intervalo $[\text{LB}, 2\text{LB}]$. Pela proposição 3.5 isso é possível em tempo $O(\log \text{LB} \cdot mn(2\text{LB} + 1)^m)$.

3. Algoritmos de aproximação

Com mais cuidado a observação permite um esquema de aproximação em tempo polinomial assintótico: similar com o esquema de aproximação para empacotamento unidimensional, vamos *remover elementos menores e arredondar* a instância.

Algoritmo 3.6 (Sequencia)

Entrada Uma instância I de $P \parallel C_{\max}$, um término máximo C e um parâmetro de qualidade ϵ .

```
1 Sequencia( $I, C, \epsilon$ ) :=  
2   remove as tarefas menores com  $p_j < \epsilon C$ ,  $j \in [n]$   
3   arredonda cada  $p_j \in [\epsilon C(1 + \epsilon)^i, \epsilon C(1 + \epsilon)^{i+1})$  para  
    algum  $i$  para  $p'_j = \epsilon C(1 + \epsilon)^i$   
4   resolve a instância arredondada com  
    programação dinâmica (proposição 3.6)  
5   empacota os itens menores gulosamente, usando  
    novas máquinas para manter o término  
     $(1 + \epsilon)C$ 
```

Lema 3.3

O algoritmo Sequencia gera um sequenciamento que termina em no máximo $(1 + \epsilon)C$ em tempo $O(n^{2 \lceil \log_{1+\epsilon} 1/\epsilon \rceil})$. Ele não usa mais máquinas que o mínimo necessário para executar as tarefas com término C

Prova. Para cada intervalo válido temos $\epsilon C(1 + \epsilon)^i \leq C$, logo o número de intervalos é no máximo $k = \lceil \log_{1+\epsilon} 1/\epsilon \rceil$. O valor k também é um limite para o número de valores p'_j distintos e pela proposição 3.6 o terceiro passo resolve a instância arredondada em tempo $O(n^{2k})$. Essa solução com os itens de tamanho original termina em no máximo $(1 + \epsilon)C$, porque $p_j/p'_j < 1 + \epsilon$. O número mínimo de máquinas para executar as tarefas em tempo C é o valor $m := \text{min-EU}(C, (p_j)_{j \in [n]})$ do problema de empacotamento unidimensional correspondente. Caso o último passo do algoritmo não usa novas máquinas ele precisa $\leq m$ máquinas, porque a instância arredondada foi resolvida exatamente. Caso contrário, uma tarefa com tempo de execução menor que ϵC não cabe nenhuma máquina, e todas máquinas usadas tem tempo de término mais que C . Logo o empacotamento ótimo com término C tem que usar pelo menos o mesmo número de máquinas. ■

Proposição 3.9

O resultado da busca binária usando o algoritmo Sequencia $C_{\max} = \min\{C \in [\text{LB}, 2\text{LB}] \mid \text{Sequencia}(I, C, \epsilon) \leq m\}$ é no máximo $C_{\max}^*$.

Prova. Com $\text{Sequencia}(I, C, \epsilon) \leq \text{min-EU}(C, (p_i)_{i \in [n]})$ temos

$$\begin{aligned} C_{\max} &= \min\{C \in [\text{LB}, 2\text{LB}] \mid \text{Sequencia}(I, C, \epsilon) \leq m\} \\ &\leq \min\{C \in [\text{LB}, 2\text{LB}] \mid \text{min-EU}(C, (p_i)_{i \in [n]}) \leq m\} \\ &= C_{\max}^* \end{aligned}$$

■

Teorema 3.11

A busca binária usando o algoritmo Sequencia para determinar um sequenciamento em tempo $O(n^{2\lceil \log_{1+\epsilon} 1/\epsilon \rceil} \log \text{LB})$ de término máximo $(1 + \epsilon)C_{\max}^*$.

Prova. Pelo lema 3.3 e proposição 3.9. ■

3.10. Exercícios

Exercício 3.1

Por que um subgrafo conexo de menor custo sempre é uma árvore?

Exercício 3.2

Mostre que o número de vértices com grau ímpar num grafo sempre é par.

Exercício 3.3

Um aluno propõe a seguinte heurística para o empacotamento unidimensional: Ordene os itens em ordem crescente, coloque o item com peso máximo junto com quantos itens de peso mínimo que é possível, e depois continue com o segundo maior item, até todos itens foram colocados em bins. Temos o algoritmo

```

1 ordene itens em ordem crescente
2  $m := 1$ ;  $M := n$ 
3 while ( $m < M$ ) do
4   abre novo contêiner, coloca  $v_M$ ,  $M := M - 1$ 
5   while ( $v_m$  cabe e  $m < M$ ) do
6     coloca  $v_m$  no contêiner atual
7      $m := m + 1$ 
8   end while
9 end while
```

Qual a qualidade desse algoritmo? É um algoritmo de aproximação? Caso sim, qual a taxa de aproximação dele? Caso não, por quê?

Exercício 3.4

Prof. Rapidez propõe o seguinte pré-processamento para o algoritmo SAT-R de aproximação para MAX-SAT (página 124): Caso a instância contém cláusulas com um único literal, vamos escolher uma delas, definir uma atribuição parcial que satisfazê-la, e eliminar a variável correspondente. Repetindo esse procedimento, obtemos uma instância cujas cláusulas tem 2 ou mais literais. Assim, obtemos $l \geq 2$ na análise do algoritmo, o podemos garantir que $E[X] \geq 3n/4$, i.e. obtemos uma 4/3-aproximação.

Esta análise está correta ou não?

4. Algoritmos randomizados

Um algoritmo randomizado usa *eventos aleatórios* na sua execução. Modelos computacionais adequados são máquinas de Turing probabilísticas – mais usadas na área de complexidade – ou máquinas RAM com um comando `random(S)` que retorne um elemento aleatório do conjunto S .

Veja alguns exemplos de probabilidades:

- Probabilidade morrer caindo da cama: $1/2 \times 10^6$ (Roach e Pieper, 2007).
- Morrer abanando a máquina de venda automática e ser espancado até a morte: 30 pessoas por ano.
- Probabilidade acertar 6 números de 60 na mega-sena: $1/50063860$.
- Probabilidade que a memória falha: em memória moderna temos 1000 FIT/MBit, i.e. 6×10^{-7} erros por segundo numa memória de 256 MB.¹
- Probabilidade que um meteorito destrói um computador em cada milissegundo: $\geq 2^{-100}$ (supondo que cada milênio ao menos um meteorito destrói uma área de $100 m^2$).

Portanto, um algoritmo que retorna uma resposta falsa com baixa probabilidade é aceitável. Em retorno um algoritmo randomizado frequentemente é

- mais simples;
- mais eficiente: para alguns problemas, um algoritmo randomizado é o mais eficiente conhecido;
- mais robusto: algoritmos randomizados podem ser menos dependentes da distribuição das entradas.
- a única alternativa: para alguns problemas, conhecemos só algoritmos randomizados.

¹FIT é uma abreviação de “failure-in-time” e é o número de erros cada 10^9 segundos. Para saber mais sobre erros em memória veja (Terrazon, 2004).

4.1. Teoria de complexidade

Classes de complexidade

Definição 4.1

Seja Σ algum alfabeto e $R(\alpha, \beta)$ a classe de linguagens $L \subseteq \Sigma^*$ tal que existe um algoritmo de decisão em tempo polinomial A que satisfaz

- $x \in L \Rightarrow \Pr(A(x) = \text{sim}) \geq \alpha$.
- $x \notin L \Rightarrow \Pr(A(x) = \text{não}) \geq \beta$.

(A probabilidade é sobre todas sequências de bits aleatórios r . Como o algoritmo executa em tempo polinomial no tamanho da entrada $|x|$, o número de bits aleatórios $|r|$ é polinomial em $|x|$ também.)

Com isso podemos definir

- a classe $\text{RP} := R(1/2, 1)$ (randomized polynomial), dos problemas que possuem um algoritmo com erro unilateral (no lado do “sim”); a classe $\text{co-RP} = R(1, 1/2)$ consiste dos problemas com erro no lado de “não”;
- a classe $\text{ZPP} := \text{RP} \cap \text{co-RP}$ (zero-error probabilistic polynomial) dos problemas que possuem algoritmo randomizado sem erro;
- a classe $\text{PP} := \bigcup_{\epsilon \in (0, 1/2]} R(1/2 + \epsilon, 1/2 + \epsilon)$ (probabilistic polynomial), dos problemas com erro $1/2 + \epsilon$ nos dois lados; e
- a classe $\text{BPP} := R(2/3, 2/3)$ (bounded-error probabilistic polynomial), dos problemas com erro $1/3$ nos dois lados.

Algoritmos que respondem corretamente somente com uma certa probabilidade também são chamados do tipo *Monte Carlo*, enquanto algoritmos que usam randomização somente internamente, mas respondem sempre corretamente são do tipo *Las Vegas*.

Exemplo 4.1 (Teste de identidade de polinômios)

Dado dois polinômios $p(x)$ e $q(x)$ de grau máximo d , como saber se $p(x) \equiv q(x)$? Caso tenhamos os dois na forma canônica $p(x) = \sum_{0 \leq i \leq d} p_i x^i$ ou na forma fatorada $p(x) = \prod_{1 \leq i \leq d} (x - r_i)$ isso é simples responder por comparação de coeficientes em tempo $\tilde{O}(n)$. E caso contrário? Converter para a forma canônica pode custar $\Theta(d^2)$ multiplicações. Uma abordagem randomizada é vantajosa, se podemos avaliar o polinômio mais rápido (por exemplo em $O(d)$):

```

1  identico(p,q) :=
2    Seleciona um número aleatório  $r$  no intervalo  $[1, 100d]$ .
3    Caso  $p(r) = q(r)$  retorne ``sim''.
4    Caso  $p(r) \neq q(r)$  retorne ``não''.
```

Caso $p(x) \equiv q(x)$, o algoritmo responde “sim” com certeza. Caso contrário a resposta pode ser errada, se $p(r) = q(r)$ por acaso. Qual a probabilidade disso? $p(x) - q(x)$ é um polinômio de grau d e possui no máximo d raízes. Portanto, a probabilidade de encontrar um r tal que $p(r) = q(r)$, caso $p \not\equiv q$ é $d/100d = 1/100$. Isso demonstra que o teste de identidade pertence à classe co-RP . $\diamond$

Observação 4.1

É uma pergunta em aberto se o teste de identidade pertence a P . $\diamond$

4.1.1. Amplificação de probabilidades

Caso não estejamos satisfeitos com a probabilidade de $1/100$ no exemplo acima, podemos repetir o algoritmo k vezes, e responder “sim” somente se todas k repetições responderam “sim”. A probabilidade erradamente responder “não” para polinômios idênticos agora é $(1/100)^k$, i.e. ela diminui exponencialmente com o número de repetições.

Essa técnica é uma *amplificação* da probabilidade de obter a solução correta. Ela pode ser aplicada para melhorar a qualidade de algoritmos em todas classes “Monte Carlo”. Com um número constante de repetições, obtemos uma probabilidade baixa nas classes RP , co-RP e BPP . Isso não se aplica a PP : é possível que ϵ diminua exponencialmente com o tamanho da instância. Um exemplo de amplificação de probabilidade encontra-se na prova do teorema 4.6.

Teorema 4.1

$\text{R}(\alpha, 1) = \text{R}(\beta, 1)$ para $0 < \alpha, \beta < 1$.

Prova. Sem perda de generalidade seja $\alpha < \beta$. Claramente $\text{R}(\beta, 1) \subseteq \text{R}(\alpha, 1)$. Suponha que A é um algoritmo que testemunha $L \in \text{R}(\alpha, 1)$. Execute A no máximo k vezes, respondendo “sim” caso A responda “sim” em alguma iteração e “não” caso contrário. Chama esse algoritmo A' . Caso $x \notin L$ temos $\Pr(A'(x) = \text{“não”}) = 1$. Caso $x \in L$ temos $\Pr(A'(x) = \text{“sim”}) \geq 1 - (1 - \alpha)^k$, logo para $k \geq \ln(1 - \beta) / \ln(1 - \alpha)$, $\Pr(A'(x) = \text{“sim”}) \geq \beta$. ■

Corolário 4.1

$\text{RP} = \text{R}(\alpha, 1)$ para $0 < \alpha < 1$.

Teorema 4.2

$\text{R}(\alpha, \alpha) = \text{R}(\beta, \beta)$ para $1/2 < \alpha, \beta$.

Prova. Sem perda de generalidade seja $\alpha < \beta$. Claramente $\text{R}(\beta, \beta) \subseteq \text{R}(\alpha, \alpha)$. Suponha que A é um algoritmo que testemunha $L \in \text{R}(\alpha, \alpha)$. Execute A k vezes, responde “sim” caso a maioria de respostas obtidas foi “sim”, e “não”

4. Algoritmos randomizados

caso contrário. Chama esse algoritmo A' . Para $x \in L$ temos

$$\Pr(A'(x) = \text{"sim"}) = \Pr(A(x) = \text{"sim"} \geq \lfloor k/2 \rfloor + 1 \text{ vezes}) \geq 1 - e^{-2k(\alpha-1/2)^2}$$

e para $k \geq \ln(\beta-1)/2(\alpha-1/2)^2$ temos $\Pr(A'(x) = \text{"sim"}) \geq \beta$. Similarmente, para $x \notin L$ temos $\Pr(A'(x) = \text{"não"}) \geq \beta$. Logo $L \in R(\beta, \beta)$. ■

Corolário 4.2

$BPP = R(\alpha, \alpha)$ para $1/2 < \alpha$.

Observação 4.2

Os resultados acima são válidos ainda caso o erro diminue polinomialmente com o tamanho da instância, i.e. $\alpha, \beta \geq n^{-c}$ no caso do teorema 4.1 e $\alpha, \beta \geq 1/2 + n^{-c}$ no caso do teorema 4.2 para uma constante c (ver por exemplo Arora e Barak (2009)). ◇

4.1.2. Relação entre as classes

Duas caracterizações alternativas de ZPP

Definição 4.2

Um algoritmo A é *honesto* se

- i) ele responde ou “sim”, ou “não” ou “não sei”,
- ii) $\Pr(A(x) = \text{não sei}) \leq 1/2$, e
- iii) no caso ele responde, ele não erra, i.e., para x tal que $A(x) \neq \text{“não sei”}$ temos $A(x) = \text{“sim”} \iff x \in L$.

Uma linguagem é honesta caso ela possui um algoritmo honesto. Com isso também podemos falar da classe das linguagens honestas.

► $ZPP \subseteq H$

Teorema 4.3

ZPP é a classe das linguagens honestas.

Lema 4.1

Caso $L \in ZPP$ existe um algoritmo honesto para L .

Prova. Para $L \in ZPP$ existem dois algoritmos $A_1 \in RP$ e $A_2 \in co-RP$. Vamos construir um algoritmo

```
1  if  $A_1(x) = A_2(x)$  then
2    return  $A_1(x)$ 
3  else if  $A_1(x) = \text{"não"}$  e  $A_2(x) = \text{"sim"}$  then
```



```

4   return ``não sei''
5   else if  $A_1(x) = \text{``sim''}$  e  $A_2(x) = \text{``não''}$  then
6       { caso impossível }
7   end if

```

O algoritmo responde corretamente “sim” e “não”, porque um dos dois algoritmos não erra. Qual a probabilidade do segundo caso? Para $x \in L$, $\Pr(A_1(x) = \text{“não”} \wedge A_2(x) = \text{“sim”}) \leq 1/2 \times 1 = 1/2$. Similarmente, para $x \notin L$, $\Pr(A_1(x) = \text{“não”} \wedge A_2(x) = \text{“sim”}) \leq 1 \times 1/2 = 1/2$. ■

Lema 4.2

► $H \subseteq ZPP$

Caso L possui um algoritmo honesto $L \in RP$ e $L \in co - RP$.

Prova. Seja A um algoritmo honesto. Construa outro algoritmo que sempre responde “não” caso A responde “não sei”, e senão responde igual. No caso de $co - RP$ analogamente construa um algoritmo que responde “sim” nos casos “não sei” de A . ■

Definição 4.3

Um algoritmo A é *sem falha* se ele sempre responde “sim” ou “não” corretamente em *tempo polinomial esperado*. Com isso podemos também falar de linguagens sem falha e a classe das linguagens sem falha.

Teorema 4.4

ZPP é a classe das linguagens sem falha.

Lema 4.3

► $ZPP \subseteq SF$

Caso $L \in ZPP$ existe um algoritmo sem falha para L .

Prova. Sabemos que existe um algoritmo honesto para L . Repete o algoritmo honesto até encontrar um “sim” ou “não”. Como o algoritmo honesto executa em tempo polinomial $p(n)$, o tempo esperado desse algoritmo ainda é polinomial:

$$\sum_{k \geq 0} k 2^{-k} p(n) \leq 2p(n)$$

■

Lema 4.4

► $SF \subseteq ZPP$

Caso L possui um algoritmo A sem falha, $L \in RP$ e $L \in co - RP$.

Prova. Caso A tem tempo esperado $p(n)$ executa ele para um tempo $2p(n)$. Caso o algoritmo responde, temos a resposta certa. Caso contrário, responde “não sei”. Pela desigualdade de Markov temos uma resposta com probabilidade $\Pr(T \geq 2p(n)) \leq p(n)/2p(n) = 1/2$. Isso mostra que existe um algoritmo honesto para L , e pelo lema 4.2 $L \in RP$. O argumento para $L \in co - RP$ é similar. ■

Mais relações**Teorema 4.5**

$$\text{RP} \subseteq \text{NP} \text{ e } \text{co-RP} \subseteq \text{co-NP}$$

Prova. Suponha que temos um algoritmo em RP para algum problema L . Podemos, não-deterministicamente, gerar todas sequências r de bits aleatórios e responder “sim” caso alguma execução encontra “sim”. O algoritmo é correto, porque caso para um $x \notin L$, não existe uma sequência aleatória r tal que o algoritmo responde “sim”. A prova do segundo caso é similar. ■

Teorema 4.6

$$\text{RP} \subseteq \text{BPP} \text{ e } \text{co-RP} \subseteq \text{BPP}.$$

Prova. Seja A um algoritmo para $L \in \text{RP}$. Constrói um algoritmo A'

```

1  if  $A(x) = \text{"não"}$  e  $A(x) = \text{"não"}$  then
2      return  $\text{"não"}$ 
3  else
4      return  $\text{"sim"}$ 
5  end if
```

Caso $x \notin L$, $\Pr(A'(x) = \text{"não"}) = \Pr(A(x) = \text{"não"} \wedge A(x) = \text{"não"}) = 1 \times 1 = 1$.
 1. Caso $x \in L$,

$$\begin{aligned} \Pr(A'(x) = \text{"sim"}) &= 1 - \Pr(A'(x) = \text{"não"}) = 1 - \Pr(A(x) = \text{"não"} \wedge A(x) = \text{"não"}) \\ &\geq 1 - 1/2 \times 1/2 = 3/4 > 2/3. \end{aligned}$$

(Observe que para k repetições de A obtemos $\Pr(A'(x) = \text{"sim"}) \geq 1 - 1/2^k$, i.e., o erro diminui exponencialmente com o número de repetições.) O argumento para co-RP é similar. ■

Relação com a classe NP e abundância de testemunhas Lembramos que a classe NP contém problemas que permitem uma verificação de uma solução em tempo polinomial. Não-deterministicamente podemos “chutar” uma solução e verificá-la. Se o número de soluções positivas de cada instância é mais que a metade do número total de soluções, o problema pertence a RP: podemos gerar uma solução aleatória e testar se ela possui a característica desejada. Um problema desse tipo possui uma *abundância de testemunhas*. Isso demonstra a importância de algoritmos randomizados. O teste de equivalência de polinômios acima é um exemplo de abundância de testemunhas.

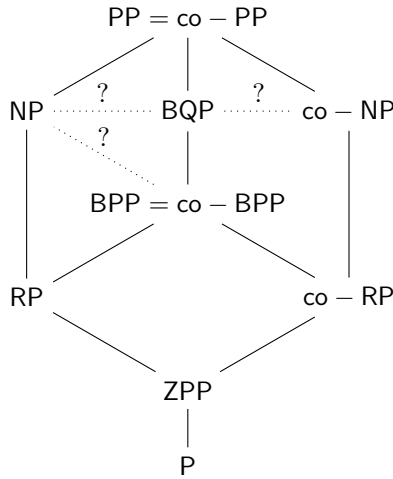


Figura 4.1.: Relações entre classes de complexidade para algoritmos randomizados.

4.2. Seleção

O algoritmo determinístico para selecionar o k -ésimo maior elemento de uma sequência não ordenada $x_1, \dots, x_n$ discutido na seção B.2 (página 187) pode ser simplificado usando randomização: escolheremos um elemento pivô $m = x_i$ aleatório. Com isso o algoritmo B.1 fica mais simples:

Algoritmo 4.1 (Seleção randomizada)

Entrada Números $x_1, \dots, x_n$, posição k .

Saída O k -ésimo maior número.

```

1   $S(k, \{x_1, \dots, x_n\}) :=$ 
2    if  $n \leq 1$ 
3      calcula e retorna o  $k$ -ésimo elemento
4    end if
5     $m := x_i$  para um  $i \in [n]$  aleatório
6     $L := \{x_i \mid x_i < m, 1 \leq i \leq n\}$ 
7     $R := \{x_i \mid x_i \geq m, 1 \leq i \leq n\}$ 
8     $i := |L| + 1$ 
```

4. Algoritmos randomizados

```

9      if  $i = k$  then
10         return  $m$ 
11     else if  $i > k$  then
12         return  $S(k, L)$ 
13     else
14         return  $S(k - i, R)$ 
15     end if

```

Para determinar a complexidade podemos observar que com probabilidade $1/n$ temos $|L| = i$ e $|R| = n - i$ e o caso pessimista é uma chamada recursiva com $\max\{i, n - i\}$ elementos. Logo, com custo cn para particionar o conjunto e os testes, escrevendo temos

$$\begin{aligned}
 T(n) &\leq cn + \sum_{i \in [0, n]} 1/n T(\max\{n - i, i\}) \\
 &\leq cn + 1/n \left(\sum_{i \in [0, k]} T(n - i) + \sum_{i \in [\lceil n/2 \rceil, n]} T(i) \right) \\
 &= cn + 2/n \sum_{i \in [0, k]} T(n - i),
 \end{aligned}$$

onde usamos $k = \lfloor n/2 \rfloor$. Separando o termo $T(n)$ do lado direito obtemos

$$\begin{aligned}
 (1 - 2/n)T(n) &\leq cn + 2/n \sum_{i \in [k]} T(n - i) \\
 \Leftrightarrow T(n) &\leq \frac{1}{n - 2} \left(cn^2 + 2 \sum_{i \in [k]} T(n - i) \right).
 \end{aligned}$$

Provaremos por indução que $T(n) \leq c'n$ para uma constante c' . Para um $n \leq n_0$ o problema pode ser claramente resolvido em tempo constante (por exemplo em $O(n_0 \log n_0)$ via ordenação). Logo, Suponha que $T(i) \leq c'i$ para $i < n$. Demonstraremos que $T(n) \leq c'n$. Temos

$$\begin{aligned}
 T(n) &\leq \frac{1}{n - 2} \left(cn^2 + 2 \sum_{i \in [k]} T(n - i) \right) \\
 &\leq \frac{1}{n - 2} \left(cn^2 + 2c' \sum_{i \in [k]} n - i \right) \\
 &= \frac{1}{n - 2} (cn^2 + 2c'(2n - k - 1)k/2)
 \end{aligned}$$

e com $2n - k - 1 = 2n - \lfloor n/2 \rfloor - 1 \leq 3/2n$

$$\leq \frac{1}{n-2}(cn^2 + 3/4c'n^2) = (c + 3/4c')\frac{n^2}{n-2}$$

Para $n \geq n_0 := 16$ temos $n/(n-2) \leq 8/7$ e com um $c' > 8c$ temos

$$T(n) \leq c'(1/8 + 3/4)8/7n = c'n.$$

4.3. Corte mínimo

CORTE MÍNIMO

Entrada Grafo não-direcionado $G = (V, A)$ com pesos $c : A \rightarrow \mathbb{Z}_+$ nas arestas.

Solução Uma partição $V = S \cup \bar{S}$ onde $\bar{S} = V \setminus S$.

Objetivo Minimizar o peso do corte $\sum_{a \in A(S, \bar{S})} c_a$.

Soluções determinísticas:

- Calcular a árvore de Gomory-Hu: a aresta de menor peso define o corte mínimo.
- Calcular o corte mínimo (via fluxo máximo) entre um vértice fixo $s \in V$ e todos outros vértices: o menor corte encontrado é o corte mínimo.

Custo em ambos casos: $O(n)$ aplicações de um algoritmo de fluxo máximo, i.e. $O(mn^2)$ usando o algoritmo de Orlin (ou $O(nm^{1+o(1)})$ com o algoritmo de Chen et al. (2022)).

Solução randomizada para pesos unitários No que segue supomos que os pesos são unitários, i.e. $c_a = 1$ para $a \in A$. Uma abordagem simples é baseada na seguinte observação: se escolhermos uma aresta que não faz parte de um corte mínimo, e contraímos-la (i.e. identificamos os vértices adjacentes), obtemos um grafo menor, que ainda contém o corte mínimo. Se escolhermos uma aresta aleatoriamente, a probabilidade de por acaso escolher uma aresta de um corte mínimo é baixa.

```

1  cmr(G) :=
2    while G possui mais que dois vértices
3      escolhe uma aresta {u,v} aleatoriamente
```

4. Algoritmos randomizados

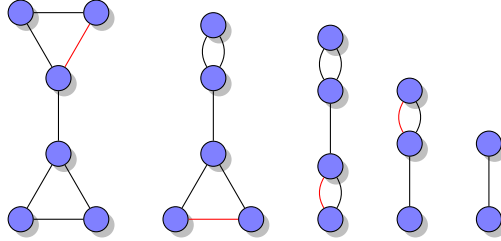
```

4     identifica  $u$  e  $v$  em  $G$ 
5     end while
6     return o corte definido pelos dois vértices em  $G$ 

```

Exemplo 4.2

Uma sequência de contrações (das arestas vermelhas).



◇

Dizemos que uma aresta “sobrevive” uma contração, caso ela não tenha sido contraída.

Lema 4.5

A probabilidade que os k arestas de um corte mínimo sobrevivem $n - n'$ contrações (de n para n' vértices) é $\Omega((n'/n)^2)$.

Prova. Como o corte mínimo é k , cada vértice possui grau pelo menos k , e portanto o número de arestas após a iteração $0 \leq i < n - n'$ é maior ou igual a $k(n - i)/2$ (com a convenção que a “iteração 0” produz o grafo inicial). Supondo que as k arestas do corte mínimo sobreviveram a iteração i , a probabilidade de não sobreviver a próxima iteração é pelo menos $k/(k(n - i)/2) = 2/(n - i)$. Logo, a probabilidade do corte sobreviver $n - n'$ iterações é pelo menos

$$\begin{aligned}
 \prod_{0 \leq i < n - n'} 1 - \frac{2}{n - i} &= \prod_{0 \leq i < n - n'} \frac{n - i - 2}{n - i} \\
 &= \frac{(n - 2)(n - 3) \cdots (n' - 1)}{n(n - 1) \cdots (n' + 1)} = \frac{n'(n' - 1)}{n(n - 1)} = \Omega((n'/n)^2).
 \end{aligned}$$

■

Teorema 4.7

Dado um corte mínimo C de tamanho k , a probabilidade do algoritmo cmr retornar C é $\Omega(n^{-2})$.

Prova. Caso o grafo possua n vértices, o algoritmo termina em $n-2$ iterações: podemos aplicar o lema acima com $n' = 2$. ■

Observação 4.3

O que acontece se repetimos o algoritmo algumas vezes? Seja C_i uma variável que indica se o corte mínimo foi encontrado na repetição i . Temos $\Pr(C_i = 1) \geq 2n^{-2}$ e portanto $\Pr(C_i = 0) \leq 1 - 2n^{-2}$. Para kn^2 repetições, vamos encontrar $C = \sum C_i$ cortes mínimos com probabilidade

$$\Pr(C \geq 1) = 1 - \Pr(C = 0) \geq 1 - (1 - 2n^{-2})^{kn^2} \geq 1 - e^{-2k}.$$

Para $k = \log n$ obtemos $\Pr(C \geq 1) \geq 1 - n^{-2}$. ◇

Logo, ao repetir o algoritmo $n^2 \log n$ vezes e retornar o menor corte encontrado, achamos o corte mínimo com probabilidade razoável. Se a implementação realiza uma contração em tempo $O(n)$ o algoritmo possui complexidade $O(n^2)$ e com as repetições em total $O(n^4 \log n)$.

Implementação de contrações Para garantir a complexidade acima, uma contração tem que ser implementada em $O(n)$. Isso é possível tanto na representação por uma matriz de adjacência, quanto na representação pela listas de adjacência. A contração de dois vértices adjacentes resulta em um novo vértice, que é adjacente aos vizinhos dos dois. Na contração arestas de um vértice com si mesmo são removidas. Múltiplas arestas entre dois vértices tem que ser mantidas para garantir o Lema 4.5.

Um algoritmo melhor (Karger e Stein, 1996) O problema principal com o algoritmo acima é que nas últimas iterações, a probabilidade de contrair uma aresta do corte mínimo é grande. Para resolver esse problema, executaremos o algoritmo duas vezes para instâncias menores, para aumentar a probabilidade de não contrair o corte mínimo. Define $f(n) = \lceil 1 + n/\sqrt{2} \rceil$.

```

1  cmr2(G) :=
2    if (G possui menos que 6 vértices)
3      determina o corte mínimo C por exaustão
4      return C
5    else
6      n' := f(n)
7      seja G1 o resultado de n - n' contrações em G
8      seja G2 o resultado de n - n' contrações em G
9      C1 := cmr2(G1)
10     C2 := cmr2(G2)

```

4. Algoritmos randomizados

11 **return** o menor dos dois cortes C_1 e C_2
 12 **end if**

Esse algoritmo possui complexidade de tempo $O(n^2 \log n)$ e encontra um corte mínimo com probabilidade $\Omega(1/\log n)$.

Lema 4.6

A probabilidade de um corte mínimo sobreviver $n - f(n)$ contrações é pelo menos $1/2$.

Prova. Pelo lema 4.5 a probabilidade é pelo menos

$$\frac{f(n)(f(n) - 1)}{n(n - 1)} \geq \frac{(1 + n/\sqrt{2})(n/\sqrt{2})}{n(n - 1)} = \frac{\sqrt{2} + n}{2(n - 1)} \geq \frac{n}{2n} = \frac{1}{2}.$$

Seja $P(n)$ a probabilidade que um corte com k arestas sobrevive caso o grafo possui n vértices. Temos

$$\Pr(\text{o corte sobrevive em } G_1) \geq 1/2 P(f(n))$$

$$\Pr(\text{o corte sobrevive em } G_2) \geq 1/2 P(f(n))$$

$$\Pr(\text{o corte não sobrevive em } G_1 \text{ nem } G_2) \leq (1 - 1/2 P(f(n)))^2$$

$$\begin{aligned} P(n) = \Pr(\text{o corte sobrevive em } G_1 \text{ ou } G_2) &\geq 1 - (1 - 1/2 P(f(n)))^2 \\ &= P(f(n)) - 1/4 P(f(n))^2 \end{aligned}$$

Para resolver essa recorrência, define $Q(k) = P(\sqrt{2}^k)$ com base $Q(0) = 1$ para obter a recorrência simplificada

$$\begin{aligned} Q(k + 1) &= P(\sqrt{2}^{k+1}) = P(\lceil 1 + \sqrt{2}^k \rceil) - 1/4 P(\lceil 1 + \sqrt{2}^k \rceil)^2 \\ &\approx P(\sqrt{2}^k) - P(\sqrt{2}^k)^2/4 = Q(k) - Q(k)^2/4 \end{aligned}$$

e depois $R(k) = 4/Q(k) - 1$ com base $R(0) = 3$ para obter

$$\frac{4}{R(k + 1) + 1} = \frac{4}{R(k) + 1} - \frac{4}{(R(k) + 1)^2} \iff R(k + 1) = R(k) + 1 + 1/R(k).$$

$R(k)$ satisfaz

$$k < R(k) < k + H_{k-1} + 3$$

Prova. Por indução. Para $k = 1$ temos $1 < R(1) = 13/3 < 1 + H_0 + 3 = 5$. Caso a HI esteja satisfeita, temos

$$R(k + 1) = R(k) + 1 + 1/R(k) > R(k) + 1 > k + 1$$

$$R(k + 1) = R(k) + 1 + 1/R(k) < k + H_{k-1} + 3 + 1 + 1/k = (k + 1) + H_k + 3$$

■

Logo, $R(k) = k + \Theta(\log k)$, e com isso $Q(k) = \Theta(1/k)$ e finalmente $P(n) = \Theta(1/\log n)$.

Para determinar a complexidade do algoritmo `cmr2` observe que temos $O(\log n)$ níveis de recursão e cada contração pode ser feita em tempo $O(n^2)$, portanto

$$T_n = 2T(f(n)) + O(n^2).$$

Aplicando o teorema de Akra-Bazzi obtemos a equação característica $2(1/\sqrt{2})^p = 1$ com solução $p = 2$ e

$$T_n \in \Theta(n^2(1 + \int_1^n \frac{cu^2}{u^3} du)) = \Theta(n^2 \log n).$$

4.4. Teste de primalidade

Um problema importante na criptografia é encontrar números primos grandes (p.ex. RSA). Escolhendo um número n aleatório, qual a probabilidade de n ser primo?

Teorema 4.8 (Hadamard (1896), Vallée Poussin (1896))

(Teorema dos números primos.)

Para $\pi(n) = |\{p \leq n \mid p \text{ primo}\}|$ temos

$$\lim_{n \rightarrow \infty} \frac{\pi(n)}{n/\ln n} = 1.$$

(Em particular $\pi(n) = \Theta(n/\ln n)$.)

Portanto, a probabilidade de um número aleatório no intervalo $[2, n]$ ser primo assintoticamente é somente $1/\ln n$. Então para encontrar um número primo, temos que testar se n é primo mesmo. Observe que isso não é igual a fatoração de n . De fato, temos testes randomizados (e determinísticos) em tempo polinomial, enquanto não sabemos fatorar nesse tempo. Uma abordagem simples é testar todos os divisores:

```

1 Primo1(n) :=
2   for i = 2, 3, 5, 7, ..., ⌊√n⌋ do
3     if i|n return ``Não''
4   end for
5   return ``Sim''

```

4. Algoritmos randomizados

O tamanho da entrada n é $t = \log n$ bits, portanto o número de iterações é $\Theta(\sqrt{n}) = \Theta(2^{t/2})$ e a complexidade $\Omega(2^{t/2})$ (mesmo contando o teste de divisão com $O(1)$) desse algoritmo é exponencial. Para testar a primalidade mais eficiente, usaremos uma característica particular dos números primos.

Teorema 4.9 (Fermat, Euler)

Para p primo e $a \geq 0$ temos

$$a^p \equiv a \pmod{p}.$$

Prova. Por indução sobre a . Base: evidente. Seja $a^p \equiv a$. Temos

$$(a+1)^p = \sum_{0 \leq i \leq p} \binom{p}{i} a^i$$

e para $0 < i < p$

$$p \mid \binom{p}{i} = \frac{p(p-1) \cdots (p-i+1)}{i(i-1) \cdots 1}$$

porque p é primo. Portanto $(a+1)^p \equiv a^p + 1 \equiv a + 1$

$$(a+1)^p \equiv a^p + 1 \equiv a + 1,$$

porque $a^p \equiv a$ por hipótese da indução. ■

Definição 4.4

Para $a, b \in \mathbb{Z}$ denotamos com (a, b) o maior divisor em comum (MDC) de a e b . No caso $(a, b) = 1$, a e b são números *coprimos*.

Teorema 4.10 (Divisão modulo n)

Caso $(b, n) = 1$

$$ab \equiv cb \pmod{n} \Rightarrow a \equiv c \pmod{n}.$$

(Em palavras: Numa identidade modulo n podemos dividir por números coprimos com n .)

Prova.

$$\begin{aligned} ab \equiv cb &\iff \exists k \, ab + kp = cb \\ &\iff \exists k \, a + kp/b = c \end{aligned}$$

Como $a, c \in \mathbb{Z}$, temos $kp/b \in \mathbb{Z}$ e $b \mid k$ ou $b \mid p$. Mas $(b, p) = 1$, então $b \mid k$. Definindo $k' := k/b$ temos $\exists k' \, a + k'p = c$, i.e. $a \equiv c$. ■

Logo, para p primo e $(a, p) = 1$ (em particular se $1 \leq a < p$)

$$a^{p-1} \equiv 1 \pmod{p}. \tag{4.1}$$

Um teste melhor então é

```

1 Primo2(n) :=
2   seleciona  $a \in [1, n-1]$  aleatoriamente
3   if  $(a, n) \neq 1$  return ``Não''
4   if  $a^{n-1} \equiv 1$  return ``Sim''
5   return ``Não''

```

Complexidade: Uma multiplicação e divisão com $\log n$ dígitos é possível em tempo $O(\log^2 n)$. Portanto, o primeiro teste (o algoritmo de Euclides em $\log n$ passos) pode ser feito em tempo $O(\log^3 n)$ e o segundo teste (exponenciação modular) é possível implementar com $O(\log n)$ multiplicações (exercício!).

Corretude: O caso de uma resposta “Não” é certo, porque n não pode ser primo. Qual a probabilidade de falhar, i.e. do algoritmo responder “Sim” para n composto? O problema é que o algoritmo falha no caso de *números Carmichael*.

► Thus:
co – RP.

Definição 4.5

Um número composto n que satisfaz $a^{n-1} \equiv 1 \pmod{n}$ é um *número pseudo-primo com base a* . Um *número Carmichael* é um número pseudo-primo para qualquer base a com $(a, n) = 1$.

Os primeiros números Carmichael são $561 = 3 \times 11 \times 17$, 1105 e 1729 (veja OEIS A002997). Existe um número infinito deles:

Teorema 4.11 (Alford et al. (1994))

Seja $C(n)$ o número de números Carmichael até n . Assintoticamente temos $C(n) > n^{2/7}$.

Exemplo 4.3

$C(n)$ até 10^{10} (OEIS A055553):

n	1	2	3	4	5	6	7	8	9	10	
$C(10^n)$	0	0	1	7	16	43	105	255	646	1547	◇
$\lceil (10^n)^{2/7} \rceil$	2	4	8	14	27	52	100	194	373	720	

Caso um número n não é primo, nem número de Carmichael, mais que $n/2$ dos $a \in [n-1]$ com $(a, n) = 1$ não satisfazem (4.1) ou seja, com probabilidade $> 1/2$ acharemos um testemunha que n é composto. O problema é que no caso de números Carmichael não temos garantia.

Teorema 4.12 (Raiz modular)

Para p primo temos

$$x^2 \equiv 1 \pmod{p} \Rightarrow x \equiv \pm 1 \pmod{p}.$$

4. Algoritmos randomizados

O teste de Miller-Rabin usa essa característica para melhorar o teste acima. Podemos escrever $n - 1 = 2^t u$ para um u ímpar. Temos $a^{n-1} = (a^u)^{2^t} \equiv 1$. Portanto, se $a^{n-1} \equiv 1$,

Ou $a^u \equiv 1$ ou existe um menor $i \in [t]$ tal que $(a^u)^{2^i} \equiv 1$

Caso p seja primo, $\sqrt{(a^u)^{2^i}} = (a^u)^{2^{i-1}} \equiv -1$ pelo teorema 4.12 e a minimalidade de i (que exclui o caso $\equiv 1$). Por isso:

Definição 4.6

Um número n é um *pseudo-primo forte com base a* caso

Ou $a^u \equiv 1 \pmod{p}$ ou existe um menor $i \in [0, t - 1]$ tal que $(a^u)^{2^i} \equiv -1 \pmod{p}$ (4.2)

```
1 Primo3(n) :=
2   if 2|n return "Não"
3   seleciona  $a \in [1, n - 1]$  aleatoriamente
4   if  $(a, n) \neq 1$  return "Não"
5   seja  $n - 1 = 2^t u$ 
6   if  $a^u \equiv 1$  return "Sim"
7   if  $(a^u)^{2^i} \equiv -1$  para um  $i \in [0, t - 1]$  return "Sim"
8   return "Não"
```

Teorema 4.13 (Monier (1980) e Rabin (1980))

Caso n seja composto e ímpar, mais que $3/4$ dos $a \in [1, n - 1]$ com $(a, n) = 1$ não satisfazem o critério (4.2) acima.

Portanto com k testes, a probabilidade de falhar $\Pr(\text{Sim} \mid n \text{ composto}) \leq (1/4)^k = 2^{-2k}$. De fato a probabilidade é menor:

Teorema 4.14 (Damgård et al., 1993)

A probabilidade de um único teste falhar para um número com k bits é $\leq k^2 4^{2-\sqrt{k}}$.

Exemplo 4.4

Para $n \in [2^{499}, 2^{500} - 1]$ a probabilidade de não detectar um n composto com um único teste é menor que

$$499^2 \times 4^{2-\sqrt{499}} \approx 2^{-22}.$$

◇

Teste determinístico O algoritmo pode ser convertido em um algoritmo determinístico, testando pelo menos $1/4$ dos a com $(a, n) = 1$. De fato, para o menor testemunho $w(n)$ de um número n ser composto temos

$$\text{Se o HGR é verdade: } w(n) < 2 \log^2 n \quad (4.3)$$

com HGR a hipótese generalizada de Riemann (uma conjectura aberta). Supondo HGR, obtemos um algoritmo determinístico com complexidade $O(\log^5 n)$. Em 2002, Agrawal et al. (2004) descobriram um algoritmo determinístico (sem a necessidade da HGR) em tempo $\tilde{O}(\log^{12} n)$ que depois foi melhorado para $\tilde{O}(\log^6 n)$.

4.5. Notas

Um applet com uma implementação do teste de Miller e Rabin [se encontra aqui](#).

4.6. Exercícios

Exercício 4.1

Encontre um primo p e um valor b tal que a identidade do teorema 4.10 não é correta.

Exercício 4.2

Encontre um número p não primo tal que a identidade do teorema 4.12 não é correta.

5. Complexidade e algoritmos parametrizados

A complexidade de um problema geralmente é resultado de diversos elementos. Um *algoritmo parametrizado* separa explicitamente os elementos que tornam um problema difícil, dos que são simples de tratar. A análise da *complexidade parametrizada* quantifica essas partes separadamente. Por isso, a complexidade parametrizada é chamada uma “complexidade de duas dimensões”.

Exemplo 5.1

O problema de satisfatibilidade (SAT) é NP-completo, i.e. não conhecemos um algoritmo cuja complexidade cresce somente polinomialmente com o tamanho da entrada. Porém, a complexidade deste problema cresce principalmente com o número de variáveis, e não com o tamanho da entrada: com k variáveis e entrada de tamanho n solução trivial resolve o problema em tempo $O(2^k n)$. Em outras palavras, para *parâmetro* k fixo, a complexidade é linear no tamanho da entrada. $\diamond$

Definição 5.1

Um problema que possui um parâmetro $k \in \mathbb{N}$ (que depende da instância) e permite um algoritmo de complexidade $f(k)|x|^{O(1)}$ para entrada x e com f uma função arbitrária, se chama *tratável por parâmetro fixo* (ingl. fixed-parameter tractable, fpt). A classe de complexidade correspondente é FPT.

Um problema tratável por parâmetro fixo se torna tratável na prática, se o nosso interesse são instâncias com parâmetro pequeno. É importante observar que um problema permite diferentes parametrizações. O objetivo do projeto de algoritmos parametrizados consiste em descobrir para quais parâmetros pequenos na prática o problema possui um algoritmo parametrizado. Neste sentido, o algoritmo parametrizado para SAT não é interessante, porque o número de variáveis na prática é grande.

A seguir consideramos o problema NP-completo de *cobertura de vértices*. Uma versão parametrizada é

k -COBERTURA DE VÉRTICES

Instância Um grafo não-direcionado $G = (V, A)$ e um número k^1 .

Solução Uma cobertura C , i.e. um conjunto $C \subseteq V$ tal que $\forall a \in A :$

5. Complexidade e algoritmos parametrizados

$$a \cap C \neq \emptyset.$$

Parâmetro O tamanho k da cobertura.

Objetivo Minimizar $|C|$.

Abordagem com força bruta:

```
1 mvc( $G = (V, A)$ ) :=  
2   if  $A = \emptyset$  return  $\emptyset$   
3   seleciona aresta  $\{u, v\} \in A$  não coberta  
4    $C_1 := \{u\} \cup \text{mvc}(G \setminus \{u\})$   
5    $C_2 := \{v\} \cup \text{mvc}(G \setminus \{v\})$   
6   return a menor entre as coberturas  $C_1$  e  $C_2$ 
```

Supondo que a seleção de uma aresta e a redução dos grafos é possível em $O(n)$, a complexidade desta abordagem é dada pela recorrência

$$T_n = 2T_{n-1} + O(n)$$

com solução $T_n = O(2^n)$. Para achar uma solução com no máximo k vértices, podemos podar a árvore de busca definida pelo algoritmo mvc na profundidade k . Isso resulta em

Teorema 5.1

O problema k -cobertura de vértices é tratável por parâmetro fixo em $O(2^k n)$.

Prova. Até o nível k vamos visitar $O(2^k)$ vértices na árvore de busca, cada um com complexidade $O(n)$. ■

O projeto de algoritmos parametrizados frequentemente consiste em

- achar uma parametrização tal que a parte super-polinomial da complexidade é limitada a uma parte do problema que depende de um parâmetro k que é pequeno na prática;
- encontrar o melhor algoritmo possível para a parte super-polinomial.

Exemplo 5.2

Considere o algoritmo direto (via uma árvore de busca, ou backtracking) para SAT.

```
1 BT-SAT( $\varphi, \alpha$ ) :=  
2   if  $\alpha$  é atribuição completa: return  $\varphi(\alpha)$ 
```

¹Introduzimos k na entrada, porque k mede uma característica da solução. Para evitar complexidades artificiais, entende-se que k nestes casos é codificado em *unário*.

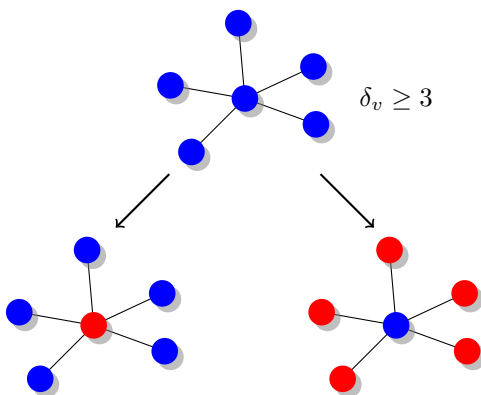


Figura 5.1.: Subproblemas geradas pela decisão da inclusão de um vértice v .
Vermelho: vértices selecionados para a cobertura.

```

3   if alguma cláusula não é satisfeita: return false
4   if BT-SAT( $\varphi, \alpha 1$ ) return true
5   return BT-SAT( $\varphi, \alpha 0$ )

```

($\alpha 0$ e $\alpha 1$ denotam extensões de uma atribuição parcial das variáveis.)

Aplicado a 3SAT, das 8 atribuições por cláusula podemos excluir uma que não a satisfaz. Portanto a complexidade de BT-SAT é $O(7^{n/3}) = O(\sqrt[3]{7^n}) = O(1.9129^n)$. (Exagerando – mas não mentindo – podemos dizer que isso é uma aceleração exponencial sobre a abordagem trivial que testa todas 2^n atribuições.)

O melhor algoritmo para 3-SAT possui complexidade $O(1.324^n)$. $\diamond$

Um algoritmo melhor para cobertura de vértices Consequência: O projeto cuidadoso de uma árvore de busca pode melhorar a complexidade. Vamos aplicar isso para o problema de cobertura de vértices.

Um melhor algoritmo para a k -cobertura de vértices pode ser obtido pelas seguintes observações

- Caso o grau máximo Δ de G é 2, o problema pode ser resolvido em tempo $O(n)$, porque G é uma coleção de caminhos simples e ciclos.
- Caso contrário, temos pelo menos um vértice v de grau $\delta_v \geq 3$. Ou esse vértice faz parte da cobertura mínima, ou todos seus vizinhos $N(v)$ (veja figura 5.1).

5. Complexidade e algoritmos parametrizados

```

1  mvc'(G) :=
2    if  $\Delta(G) \leq 2$  then
3      determina a cobertura mínima  $C$  em tempo  $O(n)$ 
4      return  $C$ 
5    end if
6    seleciona um vértice  $v$  com grau  $\delta_v \geq 3$ 
7     $C_1 := \{v\} \cup \text{mvc}'(G \setminus \{v\})$ 
8     $C_2 := N(v) \cup \text{mvc}'(G \setminus N(v))$ 
9    return a menor cobertura entre  $C_1$  e  $C_2$ 

```

O algoritmo resolve o problema de cobertura de vértices mínima de forma exata. Se podamos a árvore de busca após selecionar k vértices obtemos um algoritmo parametrizado para k -cobertura de vértices. O número de vértices nessa árvore é

$$V_i \leq V_{i-1} + V_{i-4} + 1.$$

Lema 5.1

A solução dessa recorrência é $V_i = O(1.3803^i)$.

Teorema 5.2

O problema k -cobertura de vértices é tratável por parâmetro fixo em $O(1.3803^k n)$.

Prova. Considerações acima com trabalho limitado por $O(n)$ por vértice na árvore de busca. ■

Prova. (Do lema acima.) Com o ansatz $V_i \leq c^i$ obtemos uma prova por indução se para um $i \geq i_0$

$$\begin{aligned}
 V_i &\leq V_{i-1} + V_{i-4} + 1 \leq c^{i-1} + c^{i-4} + 1 \leq c^i \\
 \iff c^{i-4}(c^4 - c^3 - 1) &\geq 1 \\
 \iff c^4 - c^3 - 1 &\geq 0
 \end{aligned}$$

(O último passo é justificado porque para $c > 1$ e i_0 suficientemente grande o produto vai ser ≥ 1 .) $c^4 - c^3 - 1$ possui uma única raiz positiva ≈ 1.32028 e para $c \geq 1.3803$ temos $c^4 - c^3 - 1 \geq 0$. ■

6. Outros algoritmos

6.1. O problema de soma de intervalos

No *problema de soma de intervalos* (ingl. range-sum problem) queremos manter números $a_1, \dots, a_n$ sobre duas operações: $\text{add}(i, v)$ aumenta a_i por v e $\text{get}(k)$ retorna $\sum_{i \in [k]} a_i$. Note que a soma sobre qualquer intervalo $[j, k]$ contíguo, $\sum_{i \in [j, k]} a_i$, é $\text{get}(k) - \text{get}(j - 1)$. Numa implementação direta por um vetor essas operações possuem complexidade $O(1)$ e $O(n)$.

Para uma operação $O : \mathbb{N} \rightarrow \mathbb{N}$ seja $Oi = \{i, O(i), O(O(i)), \dots\} \cap [n]$ a *órbita* de i sobre O .

Teorema 6.1

Caso operações O e P satisfaçam

$$|Ox \cap Py| = [x \leq y] \quad (\odot)$$

as operações

- 1 $\text{add}(i, v) := a_j := a_j + v$ para todo $j \in Oi$
- 2 $\text{get}(k) := \text{return } \sum_{i \in Pk} a_i$

resolvem o problema da soma de intervalos.

Prova. Por indução sobre as operações add . Suponha que $\text{get}(k) = \sum_{i \in [k]} a_i$. Depois de uma operação $\text{add}(i, v)$ temos: (i) Caso $i > k$: $\text{get}(k) = \sum_{i \in Pk} a'_i = \sum_{i \in Pk} a_i = \sum_{i \in [k]} a_i$ porque $|Oi \cap Pk| = 0$. (ii) Caso $i \leq k$: $\text{get}(k) = \sum_{i \in Pk} a'_i = v + \sum_{i \in Pk} a_i = v + \sum_{i \in [k]} a_i$ porque $|Ox \cap Py| = 1$. ■

Exemplo 6.1

A solução por um vetor que armazena os a_i diretamente corresponde com $O(i) = i$ e $P(i) = i - 1$. Operações add e get têm complexidade $O(1)$ e $O(n)$, respectivamente. (Critério $(\odot)$ é satisfeito porque $Oi = \{i\}$, $Pi = [i]$.) ◇

Exemplo 6.2

Com $O(i) = i + 1$ e $P(i) = i$ obtemos uma solução em que a_i armazena as somas parciais. As operações agora têm complexidade $O(n)$ e $O(1)$. (Critério $(\odot)$ é satisfeito porque $Oi = \{i, i + 1, \dots, n\}$ e $Pi = \{i\}$.) ◇

Exemplo 6.3

Seja $O(i) = i + 2^{r(i)}$ e $P(i) = i - 2^{r(i)}$ com $r(i)$ o índice do bit menos significativo (LSB) na representação binária de i . Pela definição é claro que a órbita de O cresce, i.e. $O(i) > i$, e o de P decresce, i.e. $P(i) < i$.

6. Outros algoritmos

Proposição 6.1

Cr terio $(\odot)$   satisfeito.

Prova. Se $x > y$, temos $|Ox \cap Py| = 0$, pois a  rbita de O cresce e a de P decresce. Pelo mesmo motivo, se $x = y$, ent o $|Ox \cap Py| = |\{x, y\}| = 1$   v lido.

Agora, suponha que $x < y$. Podemos escrever $x = h + s_x$, $y = h + 2^b + s_y$, onde b   o bit mais significativo diferente de x e y , $h \geq 2^{b+1}$ e $0 \leq s_x, s_y < 2^b$. Considere primeiramente $s_x = 0$. Ent o, $x = h \in Py$, j  que P remove repetidamente bit menos significativo (least significant bit, LSB) e, portanto, $x \in Ox \cap Py$. Para qualquer outro $o \in Ox$, $o \neq x$, temos $o \geq x + 2^{r(x)} \geq x + 2^{b+1}$, mas para $p \in Py$, $p \leq h + 2^b + s_y = x + 2^b + s_y < x + 2^b + 2^b = x + 2^{b+1}$. Portanto, $|Ox \cap Py| = |\{x\}| = 1$.

Agora considere $s_x > 0$. Afirmamos que $Ox \cap Py = \{m\}$, onde $m = h + 2^b$. Novamente,   f cil ver que $m \in Py$, pois P remove repetidamente o LSB. Para ver que $m \in Ox$, considere as itera  es $s_i = O^i(s_x)$, $i = 0, 1, 2, \dots$. Se $s_i < 2^b$, ent o $s_i \leq (2^b - 1) - (2^{r(s_i)} - 1) = 2^b - 2^{r(s_i)}$, j  que $r(s_i) < b$   o LSB. Assim, para o primeiro iterado tal que $s_i \geq 2^b$, temos $s_i = s_{i-1} + 2^{r(s_{i-1})} \leq 2^b$, portanto $s_i = 2^b$ e, assim, $m = h + 2^b \in Ox$.

Agora considere $e \in Ox \cap Py$. Caso $e < m$, $e \in Ox$ implica $e \geq x = h + s_x > h$, mas $e \in Py$ implica $p \leq m - 2^{r(m)} = h$, uma contradi  o. Caso $e > m$, $e \in Ox$ implica $o \geq m + 2^{r(m)} = m + 2^b = h + 2^b + 2^b = h + 2^{b+1}$ mas $e \in Py$ $p \leq y = h + 2^b + s_y < h + 2^b + 2^b = h + 2^{b+1}$, novamente uma contradi  o. Logo m   o  nico elemento em $Ox \cap Py$. ■

Proposi  o 6.2

As opera  es **add** e **get** t m complexidade $O(\log n)$.

Prova. Por indu  o, $O^i(x) \geq x + \sum_{0 \leq j < i} 2^j \geq 2^i$, de modo que a  rbita de O tem no m ximo $\log_2 n$ elementos. Da mesma forma, $P^i(y) \leq n - \sum_{0 \leq j < i} 2^j = n - 2^i + 1$ e a  rbita de P tamb m tem no m ximo $\log_2 n$ elementos. As duas opera  es podem ser implementadas de forma eficiente por $O(i) = (i \mid (i - 1)) + 1$ e $P(i) = i \& (i - 1)$ em tempo $O(1)$. ■ ◇

Exerc cio 6.1

Mostre que as opera  es $O(i) = i \mid i + 1$ e $P(i) = (i \& (i + 1)) - 1$ satisfazem o crit rio do teorema 6.1. Qual a interpreta  o das opera  es na representa  o bin ria? Voc  consegue dar uma defini  o aritm tica equivalente? Qual a complexidade de **add** e **get** usando essas opera  es?

6.2. Amostragem aleatória

Vamos discutir primeiro o caso mais simples de *amostragem com reposição* e depois a *amostragem sem reposição*.

6.2.1. Amostragem com reposição

Assume que podemos amostrar uniformemente um número real no intervalo $[0, 1]$, ou seja, $x \sim U[0, 1]$. Com isso $(u-l)x+l \sim U[l, u]$. Ainda, por subdividir o intervalo regularmente, obtemos uma amostragem discreta. Por exemplo, $\lceil nx \rceil$ e $\lfloor nx \rfloor$ produzem um número aleatório uniforme em $[n]$ e $[0, n-1]$, respectivamente¹

Mais interessante é o caso não uniforme. Considere uma função de densidade de probabilidade $\pi(x)$. Caso o domínio seja limitado, digamos $[l, u]$ e temos um limite superior L , uma solução simples é *amostragem por rejeição*: gera $x \sim U[l, u]$, aceita x com probabilidade $\pi(x)/L$, senão rejeita, e repete até aceitar uma amostra. Isso ainda funciona caso tenhamos uma função limite $l(x)$ tal que $\pi(x) \leq l(x)$ e sabemos como amostrar $l(x)$, por gerar $x \sim l(x)$ e depois aceitar com probabilidade $\pi(x)/l(x)$.

Uma abordagem melhor é por inversão. Caso $F(x)$ seja a função distribuição acumulada e F^{-1} a sua inversa, uma amostra aleatória pode ser obtida por $F^{-1}(y)$ com $y \sim U[0, 1]$. Por exemplo, para uma variável distribuída exponencialmente $x \sim \lambda e^{-\lambda x}$, com domínio $[0, \infty]$ temos

$$F(x) = \int_0^x \lambda e^{-\lambda t} dt = [-e^{-\lambda t}]_0^x = 1 - e^{-\lambda x}, \quad (6.1)$$

logo $F^{-1}(y) = -\ln(1-y)/\lambda$. Neste caso particular, com $y \sim U[0, 1]$ também $1-y \sim U[0, 1]$, logo

$$x = \frac{-\ln U[0, 1]}{\lambda} \quad (6.2)$$

é distribuído exponencial.

Agora vamos focar no caso discreto com probabilidades $p_1, p_2, \dots, p_n$. Novamente uma abordagem muito simples é amostragem por rejeição. Seja $\bar{p} = \max_{i \in [n]} p_i$. Seleccionamos um item $i \in [n]$ e um número em $q \sim U[0, \bar{p}]$ e rejeitamos se $q > p_i$. A taxa de aceitação é $1/(n\bar{p})$.

Uma ideia melhor é *tower sampling*, uma versão discreta da inversão discutido acima. Aqui, armazenamos as somas parciais $q_i = \sum_{j \in [i]} p_j$, $i \in [n]$, amostamos um número aleatório uniforme $r \in U[0, 1]$ seguido por busca binária

¹Isso, junto com algoritmos de *ranking* e *unranking*, por exemplo, é suficiente para amostrar vários objetos discretos.

6. Outros algoritmos

pelo menor i , de modo que $r \geq q_i$. O pré-processamento leva tempo $O(n)$, a amostragem apenas $O(\log n)$.

A solução para o problema de soma de intervalos discutido na seção 6.1 permite atualizar as somas de prefixo no tempo $O(\log n)$. Portanto, podemos aplicar a amostragem de torre dinamicamente com tempo de atualização de $O(\log n)$ e tempo de amostragem de $O(\log^2 n)$, já que temos no máximo $\log n$ consultas, cada uma de custo $O(\log n)$.

Uma ideia ainda melhor é *alias sampling*. Primeiro, subdivida todos os p_i em itens de baixa probabilidade $L = \{i \mid p_i < 1/n\}$, boa probabilidade $G = \{i \mid p_i = 1/n\}$ e alta probabilidade $H = \{i \mid p_i > 1/n\}$. Logo, se $L = H = \emptyset$, podemos fazer uma amostragem uniforme de G . Em seguida, observamos que $L = \emptyset$ sse $H = \emptyset$, pois as probabilidades dos n itens devem somar 1. Portanto, ou somos bons ou temos um par L-H. Para esse par, crie um compartimento “bom” combinando o item L com uma parte adequada do item H. Lembre-se dos compartimentos de origem e realoque a parte restante do item H para L, G ou H. Agora ainda temos n compartimentos, mas um compartimento bom (tipo G) a mais. Repita até que tenhamos apenas compartimentos bons. Isso leva no máximo $O(n)$ tempo, pois podemos ter no máximo n compartimentos bons.

Para amostragem, armazene em $s_1, s_2, \dots, s_{2n}$ números de itens de modo que o compartimento i represente os itens s_{2i} e s_{2i+1} . (Para compartimentos puramente bons, $s_{2i} = s_{2i+1}$.) Armazene também a massa de probabilidade do primeiro item s_{2i} em cada compartimento em $q_1, q_2, \dots, q_n$. (Novamente, para compartimentos puramente bons, $q_i = 1$).

Agora podemos amostrar em tempo $O(1)$ da seguinte forma:

```
1       $x = U(0, 1]$ 
2       $b = \lceil nx \rceil$  // localizar o compartimento
3       $r = \lceil (nx \bmod 1) \rceil > q_b$  // localizar o item no compartimento
4      retornar  $s_{2b+r}$ 
```

6.2.2. Amostragem sem reposição

Queremos selecionar k números de $[n]$ sem reposição. Uma forma simples de conseguir isso é definir um vetor $s_i = i$, $i \in [n]$ e para $j \in [k]$ trocar um elemento aleatório em $s_{[j, n]}$ com s_j . No final o prefixo $s_{[k]}$ contém a amostra desejada. O custo é $O(n)$ tempo e espaço, porque usa um vetor de tamanho n . Uma abordagem melhor usa uma tabela hash mapeando índices para valores, sem armazenar os valores default $i \mapsto i$. Com isso o custo de tempo e espaço é reduzido para $O(k)$ que é essencialmente ótimo.

Uma outra forma de conseguir isso é a *amostragem de reservatório* (ingl. reservoir sampling). Conceitualmente atribuir uma chave $U_i \sim U[0, 1]$ a cada

elemento $i \in [n]$ e depois selecionar os k elementos de menor chave produz um prefixo aleatório de k elementos de uma permutação aleatória e uniforme, e logo uma amostra uniforme de k elementos. Logo o Algoritmo 6.1 produz tal amostra.

Algoritmo 6.1 (Amostragem de reservatório (uniforme))

```

1   $S := \{(1, U_1), \dots, (k, U_k)\}$  onde  $U_i = U[0, 1]$  // amostra inicial
2  mantém  $U = \max\{U \mid (i, U) \in S\}$ 
3  for  $i = k + 1$  to  $n$ 
4       $U_i := U[0, 1]$ 
5      if  $U_i \leq U$  then
6          substitui o elemento de valor  $U$  por  $(i, U_i)$ 
7      end
8  end

```

(O algoritmo mantém o k menores elementos. Alternativamente podemos manter os k maiores.) Uma implementação usando um *max heap* precisa tempo $O(n \log n)$, n números aleatórios, e memória $O(k)$. O número esperado de atualizações do conjunto S é menor: ao processar elemento i , i.e. o prefixo $[i]$, a probabilidade do elemento i ser entre os k menores é k/i . Logo, o número esperado de atualizações do conjunto S é $\sum_{i \in [k+1, n]} k/i = k(H_n - H_k) = k(\ln n/k) + o(1)$. Note que esse algoritmo também funciona de forma *online*, passando pela sequência somente uma vez.

O caso $k = 1$ possui uma solução particularmente fácil. Mantenha um item selecionado, inicialmente o primeiro, e substitua-o pelo item i com probabilidade $1/i$. A correção dessa abordagem é facilmente demonstrada por indução. Suponha que, para n itens, tenhamos $p_i = 1/n$. Então, para $n + 1$, escolhemos $n + 1$ com probabilidade $1/(n + 1)$, ou qualquer um dos outros itens com probabilidade $p_i \cdot n/(n + 1) = 1/(n + 1)$, conforme necessário.

Talvez surpreendentemente o Algoritmo 6.1 pode ser melhorado. Note que o laço principal repetidamente gera uma amostra $U[0, 1]$ até obter um valor menor que U . Logo o número de iterações até a próxima amostra é distribuído geometricamente com probabilidade de sucesso U (a saber a probabilidade da próxima amostra precisar i iterações é $(1 - U)^{i-1}U$.) Para realizar isso vamos usar o fato que caso $x \sim \lambda e^{-\lambda x}$ é distribuído exponencialmente, então $\lceil x \rceil \sim \text{Geo}(1 - e^{-\lambda})$, i.e. temos uma distribuição geométrica sobre $1, 2, 3, \dots$ ² com probabilidade de sucesso $1 - e^{-\lambda x}$. Logo para amostrar de uma distribuição

²Com $\lfloor x \rfloor$ temos uma distribuição geométrica sobre $0, 1, \dots$

6. Outros algoritmos

geométrica com probabilidade de sucesso U , temos $U = 1 - e^{-\lambda}$ para obter $\lambda = -\ln 1 - U$, e podemos amostrar de acordo com Eq. (6.2) via $\frac{\ln U[0,1]}{\ln 1-U}$. Com isso é possível amostrar o *intervalo* até a próxima amostra diretamente, sem analisar elemento por elemento. Isso é *amostragem de reservatório com saltos exponenciais*.

Algoritmo 6.2 (AR de saltos exponenciais (uniforme))

```
1   $S := \{(1, U_1), \dots, (k, U_k)\}$  with  $U_i = U[0, 1]$  // initial sample
2  mantém  $U = \max\{U \mid (i, U) \in S\}$ 
3   $i = k$ 
4  repeat
5     $i := i + \left\lceil \frac{\ln U[0,1]}{\ln 1-U} \right\rceil$ 
6    if  $i > n$ : para
7      substitui o elemento de valor  $U$  por  $(i, U[0, U])$ ,
8    end
```

(Note como no algoritmo a nova amostra já é em $U[0, U]$.) Memória e complexidade pessimista permanecem iguais (porque o salto poderia ser 1 cada vez), mas o número esperado de iterações e atualizações do conjunto S , e logo números aleatórios gerados, é somente $m \ln n / m$.

Distribuições não uniformes. Considere agora o problema de escolher $k \leq n$ elementos da sequência $1, 2, \dots, n$ com probabilidades p_i sem reposição. Um algoritmo simples (Algoritmo SI) para resolver o problema é como segue. Começa com $S = \emptyset$, e repetidamente seleciona um item em $[n] \setminus S$ com probabilidade $w_i/w(\bar{S})$, onde $w(S) = \sum_{i \in S} w_i$. O problema é uma implementação eficiente dessa ideia. É claro que poderíamos ler toda a sequência, repetidamente amostrar como no caso com reposição e atualizar a probabilidades. Usando a versão dinâmica da amostragem não uniforme um algoritmo simples de tempo $O(n \log n)$ e memória $\Theta(n)$ é possível. Portanto, a restrição aqui é que temos $O(k)$ de memória.

Na solução desse problema, vamos supor pesos $w_1, \dots, w_n > 0$ tal que $p_i = w_i/w([n])$, i.e. os pesos não precisam ser normalizados. Uma solução do problema é o seguinte Algoritmo K. Para cada item, calcule o valor $U[0, 1]^{1/w_i}$ e mantenha os m itens de maior valor. Podemos ver facilmente por que isso é correto no caso especial de amostragem uniforme, porque com pesos $w_1 = \dots = w_n = 1$ obtemos o Algoritmo 6.1.

Algoritmo K gera n números aleatórios, e o número esperado de atualizações do conjunto escolhido é $O(m \log n/m)$, igual o caso uniforme. Há uma versão que precisa de apenas $O(m \log n/m)$ amostras aleatórias, aplicando a técnicas de saltos exponenciais discutido anteriormente. Esses algoritmos também podem ser usados para criar uma amostra aleatória de tamanho k com reposição, executando k instâncias paralelas que selecionam $m = 1$ item cada.

Vamos demonstrar a correção do Algoritmo K por demonstrar que ele é idêntico ao Algoritmo SI.

Seja $x \in U[0, 1]$, $X = x^{1/w}$ para $w > 0$. Note que $0 \leq X \leq 1$. Primeiro, temos para $\alpha \in [0, 1]$

$$\Pr(X \leq \alpha) = \Pr(x^{1/w} \leq \alpha) = \Pr(x \leq \alpha^w) = \alpha^w,$$

e logo

$$\Pr(X = \alpha) = d\Pr(X \leq \alpha)/d\alpha = w\alpha^{w-1}. \quad (6.3)$$

Considere, para $i \in [n]$, pesos $w_i > 0$, $x_i \in U[0, 1]$, $X_i = x_i^{1/w_i}$. Temos

Proposição 6.3 (Efraimidis e Spirakis (2005))

Para cada $\alpha \in [0, 1]$

$$\Pr(X_1 \leq \dots \leq X_n \leq \alpha) = \alpha^{w([n])} \prod_{i \in [n]} \frac{w_i}{w([i])}.$$

Prova. Por indução sobre n . Para $n = 1$ temos

$$\Pr(X_1 \leq \alpha) = \alpha^{w_1} = \alpha^{w([1])} \frac{w_1}{w([1])}.$$

Agora assume que a proposição vale para k . Temos

$$\begin{aligned} \Pr(X_1 \leq \dots \leq X_{k+1} \leq \alpha) &= \int_0^\alpha \Pr(X_{k+1} = t) \Pr(X_1 \leq \dots \leq X_k \leq t) dt \\ &= \int_0^\alpha w_{k+1} t^{w_{k+1}-1} t^{w([k])} \prod_{i \in [k]} \frac{w_i}{w([i])} dt \\ &= w_{k+1} \prod_{i \in [k]} \frac{w_i}{w([i])} \int_0^\alpha t^{w([k+1])-1} dt \\ &= w_{k+1} \prod_{i \in [k]} \frac{w_i}{w([i])} \alpha^{w([k+1])} / w([k+1]) \\ &= \alpha^{w([k+1])} \prod_{i \in [k+1]} \frac{w_i}{w([i])}. \end{aligned}$$



Agora vamos demonstrar que o algoritmo simples, conceitual produz o mesmo resultado que o algoritmo melhor.

Proposição 6.4

Algoritmos SI e K produzem o mesmo resultado.

Prova. Note que ambos produzem uma permutação π , caso $m = n$. É suficiente demonstrar $\Pr_{SI}(\pi) = \Pr_K(\pi)$, porque a probabilidade de produzir uma amostra que contém um item i é igual a probabilidade total de todas as permutações onde i está em uma das primeiras k posições.

Sem perda de generalidade seja $\pi = (12 \cdots n)$. Temos

$$\Pr_{SI}(\pi) = \prod_{i \in [n]} \frac{w_i}{w([i])}$$

considerando os elementos na ordem inversa. Mas pela proposição 6.3 com $\alpha = 1$ temos

$$\Pr_K(\pi) = \prod_{i \in [n]} \frac{w_i}{w([i])} = \Pr_{SI}(\pi).$$



Notas Uma boa fonte sobre amostragem são os livros de Devroye (1986) e Krauth (2006). Para amostragem sem reposição ver também Ting (2021) e Bentley e Floyd (1987). Alias sampling é bem explicada por Pătraşcu (2011). A amostragem de reservatório ponderada é de Efraimidis e Spirakis (2005).

6.3. Problemas de contagem

Consideramos o problema de contar o número de elementos distintos em uma sequência $a_1, a_2, \dots, a_n$ de algum universo U . Suponha que existam u elementos únicos. Duas abordagens exatas padrão são as seguintes. Podemos inserir todos os elementos em um conjunto de hash em tempo esperado $O(n)$ e espaço $\Theta(u)$, ou em um vetor do qual, após ordenação, os elementos repetidos são removidos, o que custa tempo $O(n \log n)$ e espaço $\Theta(n)$ ³.

Exercício 6.2

O tamanho do vetor pode ser reduzido para $O(u)$?

³A segunda abordagem é mais rápida na prática (Lemire, 2017).

Analisando cada elemento uma vez, precisamos de memória de $O(u)$, que pode ser tão grande quanto $O(n)$. Portanto, recorremos a uma solução aproximada (Flajolet e Martin, 1985), como segue. Usamos um número de L bits $b = b_0 b_1 \dots b_{L-1}$, inicialmente 0 (little endian), e uma função hash $h : U \rightarrow [0, 2^L)$. Para qualquer número $x \in [0, 2^L)$, deixe $\rho(x) = \min\{i \geq 0 \mid x \& 2^i = 0\}$ ser o índice do primeiro bit 1 (portanto, $\rho(2^L - x)$ é o índice do primeiro bit 0), ou L se todos os bits forem 0. Em seguida, simplesmente definimos, para $i \in [n]$, $b_{\rho(h(a_i))} = 1$ e estimamos $u \approx 2^{\rho(2^L - b)}/\varphi$, onde $\varphi \approx 0,77351$.

Por que isso funciona? Como estamos fazendo hashing, a probabilidade de ver o padrão $0^{i-1}1x$ (ou seja, o 1 mais à esquerda, em *little endian*, está na posição i) é 2^{-i} . Portanto, se virmos $u = 2^l$ elementos diferentes, o número esperado de observações O de $0^{i-1}1x$ é

$$E[O] = E[O_1 + \dots + O_u] = \sum_u E[O_u] = \sum_u 2^{-i} = 2^{l-i}.$$

Então, se $i \gg l$, a probabilidade é muito pequena, e se $i \ll l$, a probabilidade é muito alta de que o bit i esteja setado. Portanto, esperamos que o primeiro bit seja setado em torno de l . Uma estimativa mais exata leva à constante acima.

Proposição 6.5 (Flajolet e Martin (1985))

Para a posição do bit 0 mais à esquerda R , temos

$$\begin{aligned} E[R] &\approx \log_2 \varphi n \\ \sigma(R) &\approx \sigma_\infty \approx 1.12127 \end{aligned}$$

Isso também mostra o principal problema dessa abordagem: temos um erro típico de 1 de ordem binária de magnitude. Isso pode ser melhorado de várias maneiras.

Se usarmos m bitmaps e o estimador $A = (R_1 + \dots + R_m)/m$, teremos

$$\begin{aligned} E[A] &= \log_2 \varphi n, \\ \sigma(A) &= \sigma_\infty / \sqrt{m} \end{aligned}$$

(como sempre, o erro diminui pela raiz quadrada do número de amostras).

Uma abordagem melhor é *stochastic averaging*: usar a função hash apenas uma vez com m bitmaps. O bitmap usado é $\alpha = h(x) \bmod m$, e usamos o restante $\lfloor h(x)/m \rfloor$ dentro do bitmap escolhido. Quando cerca de n/m elementos se enquadram em cada um dos m compartimentos, então $2^A/\varphi$ é um bom estimador para n/m , com precisão de $0,78/\sqrt{m}$.

Observe que, para um limite superior de $\bar{N}$ na cardinalidade, cada bitmap ocupa cerca de $\log_2 \bar{N}$ bits. Aprimoramentos:

6. Outros algoritmos

- LogLog (Durand e Flajolet, 2003) comprime stochastic averaging, de modo que cada um dos bitmaps usa apenas $\log_2 \log_2 \bar{N}$ bits e atinge o erro padrão $1,30/\sqrt{m}$. (O algoritmo e a estimativa são essencialmente os mesmos).
- SuperLogLog (Durand e Flajolet, 2003): é LogLog com truncamento: antes de calcular a média, usamos apenas os $\lfloor 0.7m \rfloor$ menores registros. Isso reduz o erro padrão para $1,05/\sqrt{m}$.
- HyperLogLog (Flajolet, Fusy et al., 2007): é novamente o mesmo algoritmo, mas usando (uma versão corrigida de) a média geométrica $m / \sum_{j \in [m]} 2^{-R_j}$ como um estimador. Isso reduz o erro padrão para $1,04/\sqrt{m}$.
- HyperLogLog++ (Heule et al., 2013) adiciona hashes de 64 bits, corrige um viés para cardinalidades pequenas e introduz uma representação esparsa. Isso melhora as estimativas para cardinalidades muito pequenas e remove um pico no HLL, que vem da mudança imprecisa de Linear-Counting para HLL.

A. Conceitos matemáticos

Definição A.1

Uma relação binária R é *polinomialmente limitada* se

$$\exists p \in \text{poly} : \forall (x, y) \in R : |y| \leq p(|x|)$$

Definição A.2 (Pisos e tetos)

Para $x \in \mathbb{R}$ o *piso* $\lfloor x \rfloor$ é o maior número inteiro menor ou igual a x e o *teto* $\lceil x \rceil$ é o menor número inteiro maior ou igual a x . Formalmente

$$\lfloor x \rfloor = \max\{y \in \mathbb{Z} \mid y \leq x\}$$

$$\lceil x \rceil = \min\{y \in \mathbb{Z} \mid y \geq x\}$$

A *parte fracionária* de x é $\{x\} = x - \lfloor x \rfloor$.

Observe que a parte fracionária sempre é positiva, por exemplo $\{-0.3\} = 0.7$.

Proposição A.1 (Regras para pisos e tetos)

Pisos e tetos satisfazem

$$x \leq \lceil x \rceil < x + 1 \tag{A.1}$$

$$x - 1 < \lfloor x \rfloor \leq x \tag{A.2}$$

Definição A.3

Uma função f é *convexa* se ela satisfaz a desigualdade de Jensen

$$f(\Theta x + (1 - \Theta)y) \leq \Theta f(x) + (1 - \Theta)f(y). \tag{A.3}$$

Similarmente uma função f é *côncava* caso $-f$ é convexo, i.e., ela satisfaz

$$f(\Theta x + (1 - \Theta)y) \geq \Theta f(x) + (1 - \Theta)f(y). \tag{A.4}$$

Exemplo A.1

Exemplos de funções convexas são x^{2k} , $1/x$. Exemplos de funções côncavas são $\log x$, $\sqrt{x}$. $\diamond$

Proposição A.2

Para $\sum_{i \in [n]} \Theta_i = 1$ e pontos $x_i, i \in [n]$ uma função convexa satisfaz

$$f\left(\sum_{i \in [n]} \Theta_i x_i\right) \leq \sum_{i \in [n]} \Theta_i f(x_i) \quad (\text{A.5})$$

e uma função côncava

$$f\left(\sum_{i \in [n]} \Theta_i x_i\right) \geq \sum_{i \in [n]} \Theta_i f(x_i) \quad (\text{A.6})$$

Prova. Provaremos somente o caso convexo por indução, o caso côncavo sendo similar. Para $n = 1$ a desigualdade é trivial, para $n = 2$ ela é válida por definição. Para $n > 2$ define $\bar{\Theta} = \sum_{i \in [2, n]} \Theta_i$ tal que $\Theta + \bar{\Theta} = 1$. Com isso temos

$$f\left(\sum_{i \in [n]} \Theta_i x_i\right) = f\left(\Theta_1 x_1 + \sum_{i \in [2, n]} \Theta_i x_i\right) = f(\Theta_1 x_1 + \bar{\Theta} y)$$

onde $y = \sum_{j \in [2, n]} (\Theta_j / \bar{\Theta}) x_j$, logo

$$\begin{aligned} f\left(\sum_{i \in [n]} \Theta_i x_i\right) &\leq \Theta_1 f(x_1) + \bar{\Theta} f(y) \\ &= \Theta_1 f(x_1) + \bar{\Theta} f\left(\sum_{j \in [2, n]} (\Theta_j / \bar{\Theta}) x_j\right) \\ &\leq \Theta_1 f(x_1) + \bar{\Theta} \sum_{j \in [2, n]} (\Theta_j / \bar{\Theta}) f(x_j) = \sum_{i \in [n]} \Theta_i f(x_i) \end{aligned}$$

■

A.1. Probabilidade discreta

Probabilidade: Noções básicas

- Espaço amostral finito Ω de eventos elementares $e \in \Omega$.
- Distribuição de probabilidade $\Pr(e)$ tal que

$$\Pr(e) \geq 0; \quad \sum_{e \in \Omega} \Pr(e) = 1$$

- *Eventos* (compostos) $E \subseteq \Omega$ com probabilidade

$$\Pr(E) = \sum_{e \in E} \Pr(e)$$

Exemplo A.2

Para um dado sem bias temos $\Omega = \{1, 2, 3, 4, 5, 6\}$ e $\Pr(i) = 1/6$. O evento $\text{Par} = \{2, 4, 6\}$ tem probabilidade $\Pr(\text{Par}) = \sum_{e \in \text{Par}} \Pr(e) = 1/2$. $\diamond$

Probabilidade: Noções básicas

- *Variável aleatória*

$$X : \Omega \rightarrow \mathbb{N}$$

- Escrevemos $\Pr(X = i)$ para $\Pr(X^{-1}(i))$.
- Variáveis aleatórias *independentes*

$$P[X = x \text{ e } Y = y] = P[X = x]P[Y = y]$$

- *Valor esperado*

$$E[X] = \sum_{e \in \Omega} \Pr(e)X(e) = \sum_{i \geq 0} i \Pr(X = i)$$

- Linearidade do valor esperado: Para variáveis aleatórias X, Y

$$E[X + Y] = E[X] + E[Y]$$

Prova. (Das fórmulas equivalentes para o valor esperado.)

$$\begin{aligned} \sum_{0 \leq i} \Pr(X = i)i &= \sum_{0 \leq i} \Pr(X^{-1}(i))i \\ &= \sum_{0 \leq i} \sum_{e \in X^{-1}(i)} \Pr(e)X(e) = \sum_{e \in \Omega} \Pr(e)X(e) \end{aligned}$$

■

Prova. (Da linearidade.)

$$\begin{aligned} E[X + Y] &= \sum_{e \in \Omega} \Pr(e)(X(e) + Y(e)) \\ &= \sum_{e \in \Omega} \Pr(e)X(e) + \sum_{e \in \Omega} \Pr(e)Y(e) = E[X] + E[Y] \end{aligned}$$

■

Exemplo A.3

(Continuando exemplo A.2.)

Seja X a variável aleatória que denota o número sorteado, e Y a variável aleatória tal que $Y = [a \text{ face em cima do dado tem um ponto no meio}]$.

$$E[X] = \sum_{i \geq 0} \Pr(X = i)i = 1/6 \sum_{1 \leq i \leq 6} i = 21/6 = 7/2$$

$$E[Y] = \sum_{i \geq 0} \Pr(Y = i)i = \Pr(Y = 1) = 1/2$$

$$E[X + Y] = E[X] + E[Y] = 4$$

◇

Lema A.1 (Forma alternativa da expectativa)

Para uma variável aleatória X que assume somente valores não-negativos inteiros $E[X] = \sum_{k \geq 1} P[X \geq k] = \sum_{k \geq 0} P[X > k]$.

Prova.

$$E[X] = \sum_{k \geq 1} kP[X = k] = \sum_{k \geq 1} \sum_{j \in [k]} P[X = k] = \sum_{j \geq 1} \sum_{j \leq k} P[X = k] = \sum_{j \geq 1} P[X \geq j].$$

■

Lema A.2

Para tentativas repetidas com probabilidade de sucesso p , o número esperado de passos para o primeiro sucesso é $1/p$.

Prova. Seja X o número de passos até o primeiro sucesso. Temos $P[X > k] = (1 - p)^k$ e logo pelo lema A.1

$$E[X] = \sum_{k \geq 0} (1 - p)^k = 1/p.$$

■

Proposição A.3

Para φ convexo $\varphi(E[X]) \leq E[\varphi(X)]$ e para φ côncavo $\varphi(E[X]) \geq E[\varphi(X)]$.

Prova. Pela proposição A.2.

■

Proposição A.4 (Desigualdade de Markov)

Seja X uma variável aleatória com valores não-negativos. Então, para todo $a > 0$

$$\Pr[X \geq a] \leq E[X]/a.$$

Prova. Seja $I = [X \geq a]$. Como $X \geq 0$ temos $I \leq X/a$. O valor esperado de I é $E[I] = \Pr(I = 1) = \Pr[X \geq a]$, logo

$$\Pr(X \geq a) = E[I] \leq E[X/a] = E[X]/a.$$

■

Proposição A.5 (Limites de Chernoff (ingl. Chernoff bounds))

Sejam $X_1, \dots, X_n$ indicadores independentes com $\Pr(X_i) = p_i$. Para $X = \sum_i X_i$ temos para todo $\delta > 0$

$$\Pr(X \geq (1 + \delta)\mu) \leq \left(\frac{e^\delta}{(1 + \delta)^{(1 + \delta)}} \right)^\mu$$

para todo $\delta \in (0, 1)$

$$\Pr(X \leq (1 - \delta)\mu) \leq \left(\frac{e^{-\delta}}{(1 - \delta)^{(1 - \delta)}} \right)^\mu$$

para todo $\delta \in (0, 1]$

$$\Pr(X \geq (1 + \delta)\mu) \leq e^{-\mu\delta^2/3}$$

e para todo $\delta \in (0, 1)$

$$\Pr(X \leq (1 - \delta)\mu) \leq e^{-\mu\delta^2/2}.$$

Exemplo A.4

Sejam $X_1, \dots, X_k$ indicadores com $\Pr(X_i = 1) = \alpha$ e $X = \sum_i X_i$. Temos $\mu = E[X] = \sum_i E[X_i] = \alpha k$. Qual a probabilidade de ter menos que a metade dos $X_i = 1$?

$$\Pr(X \leq \lfloor k/2 \rfloor) \leq \Pr(X \leq k/2) = \Pr(X \leq \mu/(2\alpha)) =$$

$$\Pr(X \leq \mu(1 - (1 - 1/(2\alpha)))) \leq e^{-\mu\delta^2/2} = e^{-k/(2\alpha)(\alpha - 1/2)^2},$$

onde usamos

$$-\mu\delta^2/2 = -\alpha k/2 \left(1 - \frac{1}{2\alpha}\right)^2 = -\frac{k}{2\alpha}(\alpha - 1/2)^2.$$

◇

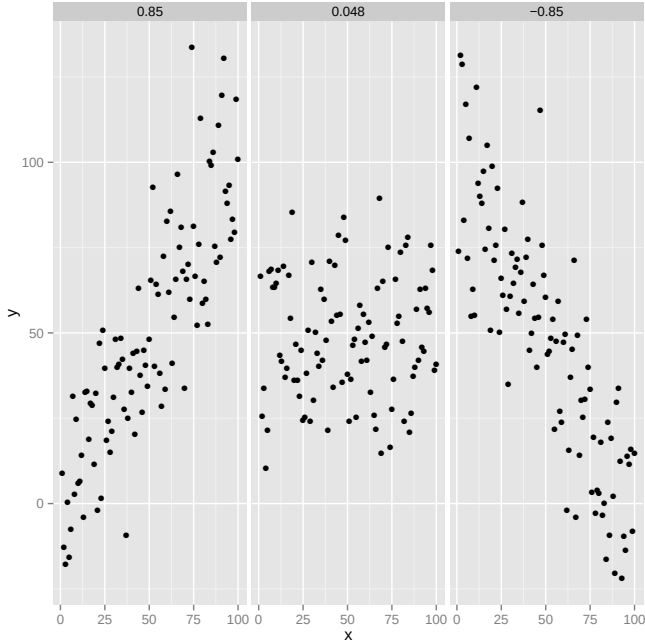


Figura A.1.: Três conjuntos de dados com correlação alta, quase zero, e negativa.

Medidas básicas A covariância de duas variáveis aleatórias X e Y é

$$\text{cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y].$$

A variância de uma variável aleatória X é a covariância com si mesmo

$$\sigma(X) = \text{cov}(X, X) = E[X^2] - E[X]^2 \quad (\text{A.7})$$

e o seu *desvio padrão* é $\sigma(X) = \sqrt{\text{cov}(X, X)}$. A *correlação* entre duas variáveis aleatórias é a covariância normalizada

$$\rho(X, Y) = \text{cov}(X, Y) / (\sigma(X)\sigma(Y)). \quad (\text{A.8})$$

A figura A.1 mostra exemplos de dados com correlações diferentes.

Um estimador não-viés da covariância dado amostras $x_1, \dots, x_n$ e $y_1, \dots, y_n$ é

$$S^2 = 1/(n-1) \sum_{i \in [n]} (x_i - \bar{x})(y_i - \bar{y})$$

para médias $\bar{x}$ e $\bar{y}$.

Todos conceitos podem ser generalizados para vetores. Neste caso temos a matriz de covariância

$$\text{cov}(X, Y) = E[(X - E[X])(Y - E[Y])^t] = E[XY] - E[X]E[Y], \quad (\text{A.9})$$

e um estimador

$$\Sigma = 1/(n-1) \sum_{i \in [n]} (x_i - \bar{x})(y_i - \bar{y})^t. \quad (\text{A.10})$$

B. Algoritmos adicionais

B.1. Soluções do problema da mochila com Programação Dinâmica

Seja $S^*(k, v)$ a solução de tamanho menor entre todas soluções que usam somente itens $S \subseteq [1, k]$ e tem valor exatamente v . Temos casos base

$$S^*(k, v) = \begin{cases} \emptyset, & \text{caso } v = 0, \\ \{1\}, & \text{caso } k = 1 \text{ e } v = v_1, \\ \text{undef}, & \text{caso } k = 1 \text{ e } v \neq v_1. \end{cases}$$

S^* obedece a recorrência

$$S^*(k, v) = \min_{\text{tamanho}} \begin{cases} S^*(k-1, v-v_k) \cup \{k\}, & \text{se } v_k \leq v \text{ e } S^*(k-1, v-v_k) \text{ definido} \\ S^*(k-1, v) \end{cases}$$

Para seleccionar o menor tamanho entre os dois compare

$$\sum_{i \in S^*(k-1, v-v_k)} t_i + t_k \leq \sum_{i \in S^*(k-1, v)} t_i.$$

Para obter o melhor valor escolhe $S^*(n, v)$ com o valor máximo de v definido. O custo de tempo e espaço é $O(n \sum_{i \in [n]} v_i)$.

B.2. Seleção

Dado um conjunto de números, o problema da seleção consiste em encontrar o k -ésimo maior elemento. Com ordenação o problema possui solução em tempo $O(n \log n)$. Mas existe um outro algoritmo mais eficiente. Podemos determinar o mediano de grupos de cinco elementos, e depois o recursivamente o mediano m desses medianos. Com isso, o algoritmo particiona o conjunto de números em um conjunto L de números menores que m e um conjunto R de números maiores que m . O mediano m é na posição $i := |L| + 1$ desta sequência. Logo, caso $i = k$ m é o k -ésimo elemento. Caso $i > k$ temos que procurar o k -ésimo elemento em L , caso $i < k$ temos que procurar o $k-i$ -ésimo elemento em R (ver figura [B.1](#)).

B. Algoritmos adicionais

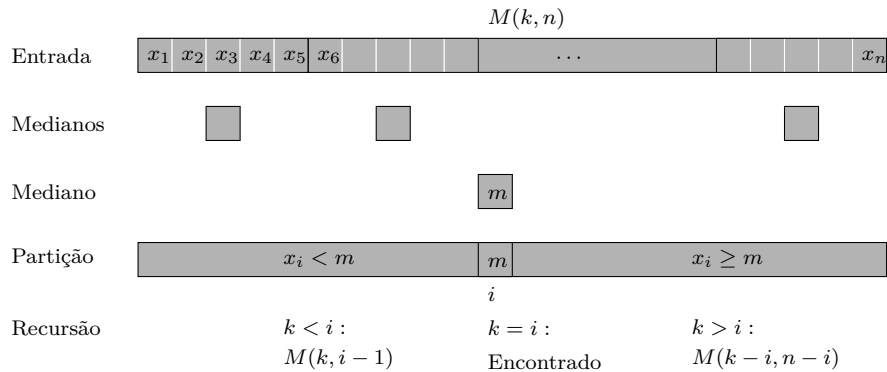


Figura B.1.: Funcionamento do algoritmo recursivo para seleção.

O algoritmo é eficiente, porque a seleção do elemento particionador m garante que o subproblema resolvido na segunda recursão é no máximo um fator $7/10$ do problema original. Mais preciso, o número de medianos é maior que $n/5$, logo o número de medianos antes de m é maior que $n/10 - 1$, o número de elementos antes de m é maior que $3n/10 - 3$ e com isso o número de elementos depois de m é menor que $7n/10 + 3$. Por um argumento similar, o número de elementos antes de m é também menor que $7n/10 + 3$. Portanto temos um custo no caso pessimista de

$$T(n) = \begin{cases} \Theta(1), & \text{se } n \leq 5 \\ T(\lceil n/5 \rceil) + \Theta(7n/10 + 3) + \Theta(n), & \text{caso contrário} \end{cases}$$

e com $5^{-p} + (7/10)^p = 1$ temos $p = \log_2 7 \approx 0.84$ e

$$\begin{aligned} T(n) &= \Theta \left(n^p \left(1 + \int_1^n u^{-p} du \right) \right) \\ &= \Theta(n^p(1 + (n^{1-p}/(1-p) - 1/(1-p)))) \\ &= \Theta(c_1 n^p + c_2 n) = \Theta(n). \end{aligned}$$

Algoritmo B.1 (Seleção)

Entrada Números $x_1, \dots, x_n$, posição k .

Saída O k -ésimo maior número.

```

1   $S(k, \{x_1, \dots, x_n\}) :=$ 
2    if  $n \leq 5$ 
3      calcula e retorne o  $k$ -ésimo elemento
4    end if
5     $m_i := \text{median}(x_{5i+1}, \dots, x_{\min(5i+5, n)})$  para  $0 \leq i < \lceil n/5 \rceil$ .
6     $m := S(\lceil \lceil n/5 \rceil / 2 \rceil, m_1, \dots, m_{\lceil n/5 \rceil - 1})$ 
7     $L := \{x_i \mid x_i < m, 1 \leq i \leq n\}$ 
8     $R := \{x_i \mid x_i \geq m, 1 \leq i \leq n\}$ 
9     $i := |L| + 1$ 
10   if  $i = k$  then
11     return  $m$ 
12   else if  $i > k$  then
13     return  $S(k, L)$ 
14   else
15     return  $S(k - i, R)$ 
16   end if

```


C. Técnicas para a análise de algoritmos

C.1. Análise de recorrências

Teorema C.1 (Akra-Bazzi e Leighton)

Dada a recorrência

$$T(x) = \begin{cases} \Theta(1), & \text{se } x \leq x_0, \\ \sum_{1 \leq i \leq k} a_i T(b_i x + h_i(x)) + g(x), & \text{caso contrário,} \end{cases}$$

com constantes $a_i > 0$, $0 < b_i < 1$ e funções g , h , tal que

$$|g'(x)| \in O(x^c); \quad |h_i(x)| \leq x / \log^{1+\epsilon} x$$

para um $\epsilon > 0$ e a constante x_0 e suficientemente grande

$$T(x) \in \Theta \left(x^p \left(1 + \int_1^x \frac{g(u)}{u^{p+1}} du \right) \right)$$

com p tal que $\sum_{1 \leq i \leq k} a_i b_i^p = 1$.

Teorema C.2 (Graham et al. (1988))

Dada a recorrência

$$T(n) = \begin{cases} \Theta(1), & n \leq \max_{1 \leq i \leq k} d_i, \\ \sum_i \alpha_i T(n - d_i), & \text{caso contrário,} \end{cases}$$

seja α a raiz com o maior valor absoluto com multiplicidade l do *polinômio característico*

$$z^d - \alpha_1 z^{d-d_1} - \dots - \alpha_k z^{d-d_k}$$

com $d = \max_k d_k$. Então

$$T(n) = \Theta(n^l \alpha^n) = \Theta^*(\alpha^n).$$

Bibliografia

- [1] Manindra Agrawal, Neeraj Kayal e Nitin Saxena. “PRIMES is in P”. Em: *Annals of Mathematics* 160.2 (2004), pp. 781–793.
- [2] W. R. Alford, A. Granville e C. Pomerance. “There are infinitely many Carmichael numbers”. Em: *Annals Math.* 140 (1994).
- [3] *Algorithm Engineering*. <http://www.algorithm-engineering.de>. Deutsche Forschungsgemeinschaft.
- [4] H. Alt et al. “Computing a maximum cardinality matching in a bipartite graph in time $O(n^{1.5}\sqrt{m \log n})$ ”. Em: *Information Processing Letters* 37 (1991), pp. 237–240.
- [5] June Andrews e J. A. Sethian. “Fast marching methods for the continuous traveling salesman problem”. Em: *Proc. Natl. Acad. Sci. USA* 104.4 (2007). DOI: [10.1073/pnas.0609910104](https://doi.org/10.1073/pnas.0609910104).
- [6] Sanjeev Arora e Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [7] Brenda S. Baker. “A new proof for the first fit decreasing bin packing algorithm”. Em: *J. Alg.* 6 (1985), pp. 49–70. DOI: [10.1016/0196-6774\(85\)90018-5](https://doi.org/10.1016/0196-6774(85)90018-5).
- [8] Jon Bentley e Bob Floyd. “Programming pearls: a sample of brilliance”. Em: *Communications of the ACM* 30.9 (set. de 1987), pp. 754–757. ISSN: 1557-7317. DOI: [10.1145/30401.315746](https://doi.org/10.1145/30401.315746).
- [9] Claude Berge. “Two theorems in graph theory”. Em: *Proc. National Acad. Science* 43 (1957), pp. 842–844.
- [10] John R. Black Jr. e Charles U. Martel. *Designing Fast Graph Data Structures: An Experimental Approach*. Rel. téc. Department of Computer Science, University of California, Davis, 1998.
- [11] G. S. Brodal, R. Fagerberg e R. Jacob. *Cache Oblivious Search Trees via Binary Trees of Small Height*. Rel. téc. RS-01-36. BRICS, 2001.
- [12] Andrei Broder e Michael Mitzenmacher. “Network applications of Bloom filter: A survey”. Em: *Internet Mathematics* 1.4 (2003), pp. 485–509.
- [13] Bernhard Chazelle. “A Minimum Spanning Tree Algorithm with Inverse-Ackermann Type Complexity”. Em: *Journal ACM* 47 (2000), pp. 1028–1047.

- [14] Li Chen et al. “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time”. Em: *tbd* (2022). DOI: [10.48550/arxiv.2203.00671](https://doi.org/10.48550/arxiv.2203.00671). arXiv: [2203.00671](https://arxiv.org/abs/2203.00671) [cs.DS].
- [15] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. The MIT Press, 2009.
- [16] Ivan Damgård, Peter Landrock e Carl Pomerance. “Average case error estimates for the strong probable prime test”. Em: *Mathematics of computation* 61.203 (1993), pp. 177–194.
- [17] Brian C. Dean, Michel X. Goemans e Nicole Immorlica. “Finite termination of ”augmenting path”algorithms in the presence of irrational problem data”. Em: *ESA’06: Proceedings of the 14th conference on Annual European Symposium*. Zurich, Switzerland: Springer-Verlag, 2006, pp. 268–279. DOI: [10.1007/11841036_26](https://doi.org/10.1007/11841036_26).
- [18] R. Dementiev et al. “Engineering a Sorted List Data Structure for 32 Bit Keys”. Em: *Workshop on Algorithm Engineering & Experiments*. 2004, pp. 142–151.
- [19] Luc Devroye. *Non-Uniform Random Variate Generation*. Springer New York, 1986. ISBN: 9781461386438. DOI: [10.1007/978-1-4613-8643-8](https://doi.org/10.1007/978-1-4613-8643-8).
- [20] Ran Duan, Seth Pettie e Hsin-Hao Su. “Scaling algorithms for approximate and exact maximum weight matching”. Em: *CoRR* abs/1112.0790 (2011).
- [21] Ran Duan, Seth Pettie e Hsin-Hao Su. “Scaling Algorithms for Weighted Matching in General Graphs”. Em: *ACM Trans. Algorithms* 14.1 (2018), pp. 225–231. DOI: [10.1145/3155301](https://doi.org/10.1145/3155301).
- [22] Marianne Durand e Philippe Flajolet. “Loglog Counting of Large Cardinalities”. Em: *Algorithms - ESA 2003*. Springer Berlin Heidelberg, 2003, pp. 605–617. ISBN: 9783540396581. DOI: [10.1007/978-3-540-39658-1_55](https://doi.org/10.1007/978-3-540-39658-1_55).
- [23] J. Edmonds. “Paths, Trees, and Flowers”. Em: *Canad. J. Math* 17 (1965), pp. 449–467.
- [24] J. Edmonds e R. Karp. “Theoretical improvements in algorithmic efficiency for network flow problems”. Em: *JACM* 19.2 (1972), pp. 248–264.
- [25] Pavlos S. Efrimidis e Paul G. Spirakis. “Weighted Random Sampling”. Em: *Encyclopedia of Algorithms*. 2005.
- [26] Jenő Egerváry. “Matrixok kombinatorius tulajdonságairól (On combinatorial properties of matrices)”. Em: *Matematikai és Fizikai Lapok* 38 (1931), pp. 16–28.

- [27] T. Feder e R. Motwani. “Clique Partitions, Graph Compression and Speeding-Up Algorithms”. Em: *Journal of Computer and System Sciences* 51.2 (out. de 1995), pp. 261–272. ISSN: 0022-0000. DOI: [10.1006/jcss.1995.1065](https://doi.org/10.1006/jcss.1995.1065).
- [28] T. Feder e R. Motwani. “Clique partitions, graph compression and speeding-up algorithms”. Em: *Proceedings of the Twenty Third Annual ACM Symposium on Theory of Computing (23rd STOC)*. 1991, pp. 123–133.
- [29] Philippe Flajolet, Éric Fusy et al. “HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm”. Em: *Discrete Mathematics & Theoretical Computer Science DMTCS Proceedings* vol. AH,...Proceedings (jan. de 2007). ISSN: 1365-8050. DOI: [10.46298/dmtcs.3545](https://doi.org/10.46298/dmtcs.3545).
- [30] Philippe Flajolet e G. Nigel Martin. “Probabilistic counting algorithms for data base applications”. Em: *Journal of Computer and System Sciences* 31.2 (out. de 1985), pp. 182–209. ISSN: 0022-0000. DOI: [10.1016/0022-0000\(85\)90041-8](https://doi.org/10.1016/0022-0000(85)90041-8).
- [31] L. R. Ford e D. R. Fulkerson. “Maximal flow through a network”. Em: *Canadian Journal of Mathematics* 8 (1956), pp. 399–404.
- [32] C. Fremuth-Paege e D. Jungnickel. “Balanced network flows VIII: a revised theory of phase-ordered algorithms and the $O(\sqrt{nm} \log(n^2/m)/\log n)$ bound for the nonbipartite cardinality matching problem”. Em: *Networks* 41 (2003), pp. 137–142.
- [33] Martin Fürer e Balaji Raghavachari. “Approximating the minimum-degree steiner tree to within one of optimal”. Em: *Journal of Algorithms* (1994).
- [34] H. N. Gabow. “Data structures for weighted matching and nearest common ancestors with linking”. Em: *Proc. of the 1st Annual ACM-SIAM Symposium on Discrete Algorithms* (1990), pp. 434–443.
- [35] Ashish Goel, Michael Kapralov e Sanjeev Khanna. “Perfect Matchings in $O(n \log n)$ Time in Regular Bipartite Graphs”. Em: *STOC 2010*. 2010.
- [36] A. V. Goldberg e A. V. Karzanov. “Maximum skew-symmetric flows and matchings”. Em: *Mathematical Programming A* 100 (2004), pp. 537–568.
- [37] Olivier Goldschmidt e Dorit S. Hochbaum. “Polynomial Algorithm for the k -Cut Problem”. Em: *Proc. 29th FOCS*. 1988, pp. 444–451.
- [38] Ronald Lewis Graham, Donald Ervin Knuth e Oren Patashnik. *Concrete Mathematics: a foundation for computer science*. Addison-Wesley, 1988.
- [39] J. Hadamard. “Sur la distribution des zéros de la fonction zeta(s) et ses conséquences arithmétiques”. Em: *Bull. Soc. math. France* 24 (1896), pp. 199–220.

- [40] Bernhard Haeupler, Siddharta Sen e Robert E. Tarjan. “Heaps simplified”. Em: (*Preprint*) (2009). arXiv:0903.0116.
- [41] Stefan Heule, Marc Nunkesser e Alexander Hall. “HyperLogLog in practice: algorithmic engineering of a state of the art cardinality estimation algorithm”. Em: *Proceedings of the 16th International Conference on Extending Database Technology*. EDBT/ICDT '13. ACM, mar. de 2013, pp. 683–692. DOI: [10.1145/2452376.2452456](https://doi.org/10.1145/2452376.2452456).
- [42] Carl Hierholzer. “Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren”. Em: *Mathematische Annalen* 6 (1873), pp. 30–32. DOI: [10.1007/bf01442866](https://doi.org/10.1007/bf01442866).
- [43] J. E. Hopcroft e R. Karp. “An $n^{5/2}$ algorithm for maximum matching in bipartite graphs”. Em: *SIAM J. Comput.* 2 (1973), pp. 225–231.
- [44] David S. Johnson. “Near-optimal bin packing algorithms”. Tese de doutoramento. Massachusetts Institute of Technology. Dept. of Mathematics, 1973. URL: <http://hdl.handle.net/1721.1/57819>.
- [45] David S. Johnson e Michael R. Garey. “A 71/60 theorem for bin packing”. Em: *J. Complex.* 1.1 (1985), pp. 65–106. DOI: [10.1016/0885-064X\(85\)90022-6](https://doi.org/10.1016/0885-064X(85)90022-6).
- [46] Michael J. Jones e James M. Rehg. *Statistical Color Models with Application to Skin Detection*. Rel. téc. CRL 98/11. Cambridge Research Laboratory, 1998.
- [47] Haim Kaplan e Uri Zwick. “A simpler implementation and analysis of Chazelle’s soft heaps”. Em: *SODA '09: Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. New York, New York: Society for Industrial e Applied Mathematics, 2009, pp. 477–485.
- [48] David R. Karger e Clifford Stein. “A new approach to the minimum cut problem”. Em: *Journal of the ACM* 43.4 (1996), pp. 601–640. DOI: [10.1145/234533.234534](https://doi.org/10.1145/234533.234534).
- [49] Werner Krauth. *Statistical Mechanics: Algorithms and Computation*. OUP, 2006.
- [50] H. W. Kuhn. “The Hungarian Method for the assignment problem”. Em: *Naval Research Logistic Quarterly* 2 (1955), pp. 83–97.
- [51] Daniel Lemire. *Counting exactly the number of distinct elements: sorted arrays vs. hash sets?* 23 de mai. de 2017. URL: <https://lemire.me/blog/2017/05/23/counting-exactly-the-number-of-distinct-elements-sorted-arrays-vs-hash-sets/>.

- [52] Jerry Li e John Peebles. “Replacing Mark Bits with Randomness in Fibonacci Heaps”. Em: *Int. Colloq. Automata, Languages, and Progr.* Ed. por Magnús Halldórsson et al. Vol. 9134. LNCS. 2015, pp. 886–897.
- [53] S. Micali e Vijay V. Vazirani. “An $O(\sqrt{|v||E|})$ algorithm for finding maximum matching in general graphs”. Em: *Proc. 21th FOCS*. 1980, pp. 17–27.
- [54] L. Monier. “Evaluation and comparison of two efficient probabilistic primality testing algorithms”. Em: *Theoret. Comp. Sci.* 12 (1980), pp. 97–108.
- [55] J. Munkres. “Algorithms for the assignment and transporation problems”. Em: *J. Soc. Indust. Appl. Math* 5.1 (1957), pp. 32–38.
- [56] K. Noshita. “A theorem on the expected complexity of Dijkstra’s shortest path algorithm”. Em: *Journal of Algorithms* 6 (1985), pp. 400–408.
- [57] Joon-Sang Park, Michael Penner e Viktor K. Prasanna. “Optimizing Graph Algorithms for Improved Cache Performance”. Em: *IEEE Trans. Par. Distr. Syst.* 15.9 (2004), pp. 769–782.
- [58] Mihai Pătraşcu. *Follow-up: Sampling a discrete distribution*. 19 de set. de 2011. URL: <http://infowEEKLY.blogspot.com/2011/09/follow-up-sampling-discrete.html>.
- [59] Michael O. Rabin. “Probabilistic algorithm for primality testing”. Em: *J. Number Theory* 12 (1980), pp. 128–138.
- [60] Emma Roach e Vivien Pieper. “Die Welt in Zahlen”. Em: *Brand eins* 3 (2007).
- [61] J.R. Sack e J. Urrutia, eds. *Handbook of computational geometry*. Elsevier, 2000.
- [62] Alexander Schrijver. *Combinatorial optimization. Polyhedra and efficiency*. Vol. A. Springer, 2003.
- [63] J. A. Sethian. *Level Set Methods and Fast Marching Methods: Evolving Interfaces in Computational Geometry, Fluid Mechanics, Computer Vision and Materials Science*. Cambridge Monographs on Applied and Computational Mathematics. Cambridge University Press, 1999.
- [64] Terrazon. *Soft Errors in Electronic Memory – A White Paper*. Rel. téc. Terrazon Semiconductor, 2004.
- [65] Daniel Ting. “Simple, Optimal Algorithms for Random Sampling Without Replacement”. Em: (abr. de 2021). DOI: [10.48550/ARXIV.2104.05091](https://doi.org/10.48550/ARXIV.2104.05091). arXiv: [2104.05091](https://arxiv.org/abs/2104.05091) [[cs.DS](#)].

- [66] C.-J. de la Vallée Poussin. “Recherches analytiques la théorie des nombres premiers”. Em: *Ann. Soc. scient. Bruxelles* 20 (1896), pp. 183–256.
- [67] Norman Zadeh. “Theoretical Efficiency of the Edmonds-Karp Algorithm for Computing Maximal Flows”. Em: *J. ACM* 19.1 (1972), pp. 184–192.
- [68] Uri Zwick. “The smallest networks on which the Ford-Fulkerson maximum flow procedure may fail to terminate”. Em: *Theoretical Computer Science* 148.1 (1995), pp. 165–170. DOI: [DOI : 10 . 1016 / 0304 - 3975 \(95\) 00022 - 0](https://doi.org/10.1016/0304-3975(95)00022-0).

Índice

A

- admissível, 15
- Akra, Louay, 191
- Akra-Bazzi
 - método de, 191
- algoritmo
 - ϵ -aproximativo, 119
 - de aproximação, 117
 - guloso, 119
 - parametrizado, 165
 - primal-dual, 125
 - randomizado, 147
 - r -aproximativo, 119
- algoritmo A^* , 14
- alternante, *ver* caminho, alternante
- amostragem
 - com reposição, 171
 - de reservatório, 172
 - de reservatório com saltos exponenciais, 174
 - por rejeição, 171
 - sem reposição, 171
- aproximação
 - absoluta, 119
 - relativa, 119
- APX, 119
- arredondamento randomizado, 125
- árvore
 - binomial, 24
 - van Emde Boas, 47–56
 - vEB, *ver* árvore, van Emde Boas
- árvore geradora mínima, 8
 - algoritmo de Prim, 8

árvore Steiner mínima, 128

B

- Baker, Brenda S., 139
- Bazzi, Mohamad, 191
- bin packing
 - empacotamento unidimensional, 135
- Bloom, Burton Howard, 113
- busca informada, 13

C

- caminho
 - alternante, 86
 - Euleriano, 7
 - mais curto, 21, 57
 - algoritmo de Dijkstra, 21, 57
- caminho mais gordo
 - algoritmo de, 65, 66
- cascading cut, *ver* corte, em cascatas
- Chernoff bounds, *ver* limites de Chernoff
- circulação, 58
- cobertura de vértices, 120, 165
 - aproximação, 120
- complexidade
 - amortizada, 26
 - parametrizada, 165
- consistente, 15
- coprimos, 160
- corte
 - em cascatas, 29

cuco hashing, 111

D

desigualdade

de Jensen, 179

de Markov, 182

desigualdade triangular, 128

dicionário, 105

Dijkstra

algoritmo de, 13, 21, 57

Dijkstra, Edsger Wybe, 21

distribuição, 181

E

Edmonds, Jack R., 63

Edmonds-Karp

algoritmo de, 63–65

empacotamento unidimensional, 135

emparelhado, *ver* vértice, emparelhado

emparelhamento, 81

de peso máximo, 81

máximo, 81

perfeito, 81

de peso mínimo, 81

endereçamento aberto, 109

equação Eikonal, 12

espaço amostral, 181

evento, 181

elementar, 181

excesso, 66

F

fator de ocupação, 106

fecho métrico, 128

fila de prioridade, 21–57

com lista ordenada, 9

com vetor, 9

filtro de Bloom, 113

fluxo, 58

com fontes e destinos múltiplos, 70

de menor custo, 80

formulação linear, 60

s - t máximo, 59

Ford, Lester Randolph, 60

Ford-Fulkerson

algoritmo de, 60–63

forward star, 5

Fulkerson, Delbert Ray, 60

função

côncava, 179

convexa, 179

função de otimização, 117

função hash, 105

com divisão, 107

com multiplicação, 107

universal, 107, 108

função objetivo, 117

G

grafo

Euleriano, 7

grafo residual, 61

H

hashing

com endereçamento aberto, 109

com listas encadeadas, 105

cuco, 111

perfeito, 105, 108

universal, 107

heap, 21–57

binário, 21, 56

implementação, 24

binomial, 24, 37, 56

custo amortizado, 28

Fibonacci, 29

hollow, *ver* heap, oco

oco, 41

rank-pairing, 33, 39

rp, *ver* heap, rank-pairing *ver* head, rank-pairing

- Hierholzer
 - algoritmo de, 8
- Hierholzer, Carl, 8
- hollow heap, *ver* heap, oco
- J**
- Jensen
 - desigualdade de, 179
- Johnson, David Stifler, 139
- K**
- Karp, Richard Manning, 63
- Knapsack, 122
- L**
- limites de Chernoff, 183
- linearidade do valor esperado, 182
- livre, *ver* vértice, livre
- M**
- maxheap, *ver* heap, máximo
- método de divisão, 107
- método de multiplicação, 107
- minheap, *ver* heap, mínimo
- N**
- NPO, 118
- O**
- ordem
 - van Emde Boas, 48
 - vEB, *ver* ordem, van Emde Boas
- P**
- $P \parallel C_{\max}$, 142
- permutação, 109
- piso, 179
- PO, 118
- pré-fluxo, 66
- Prim
 - algoritmo de, 8
- Prim, Robert Clay, 8
- probabilidade, 181
- problema
 - da mochila, 187
 - de avaliação, 118
 - de construção, 118
 - de decisão, 118
 - de otimização, 117
- problema da mochila, 122, 187
- problema de soma de intervalos, 169
- pseudo-primo forte, 162
- R**
- rank-pairing heap, *ver* heap, rank-pairing
- relação
 - polinomialmente limitada, 118
- S**
- SAT, 165
- satisfatibilidade
 - de fórmulas booleanas, 165
- semi-árvore, 34
- sequenciamento
 - em processadores paralelos, 142
- T**
- terminal, 128
- teto, 179
- torneio, 33
- tower sampling, 171
- tratável por parâmetro fixo, 165
- U**
- uniforme, 109
- V**
- valor esperado, 182
- valor hash, 105
- van Emde Boas, Peter, 49
- variável aleatória, 181, 182
 - independente, 181
- vertex cover, 120

Índice

aproximação, [120](#)
vértice
ativo, [66](#)
emparelhado, [86](#)
livre, [86](#)

W

Williams, J. W. J., [21](#)